

Digital Pendulum System

TEACHING MANUAL

33-005-4M5

MATLAB 5 Version



Feedback

Feedback Instruments Ltd, Park Road, Crowborough, E. Sussex, TN6 2QR, UK.

Telephone: +44 (0) 1892 653322, Fax: +44 (0) 1892 663719.

email: feedback@fdbk.demon.co.uk World Wide Web Homepage: <http://www.fbk.com>

Manual: 33-005-4M5 Ed01 981130

Printed in England by FI Ltd, Crowborough

Feedback Part No. 1160-330054M5



DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

PREFACE

THE HEALTH AND SAFETY AT WORK ACT 1974

We are required under the Health and Safety at Work Act 1974, to make available to users of this equipment certain information regarding its safe use.

The equipment, when used in normal or prescribed applications within the parameters set for its mechanical and electrical performance, should not cause any danger or hazard to health or safety if normal engineering practices are observed and they are used in accordance with the instructions supplied.

If, in specific cases, circumstances exist in which a potential hazard may be brought about by careless or improper use, these will be pointed out and the necessary precautions emphasised.

While we provide the fullest possible user information relating to the proper use of this equipment, if there is any doubt whatsoever about any aspect, the user should contact the Product Safety Officer at Feedback Instruments Limited, Crowborough.

This equipment should not be used by inexperienced users unless they are under supervision.

We are required by European Directives to indicate on our equipment panels certain areas and warnings that require attention by the user. These have been indicated in the specified way by yellow labels with black printing, the meaning of any labels that may be fixed to the instrument are shown below:



CAUTION -
RISK OF
DANGER

Refer to accompanying documents



CAUTION -
RISK OF
ELECTRIC SHOCK



CAUTION -
ELECTROSTATIC
SENSITIVE DEVICE

PRODUCT IMPROVEMENTS

We maintain a policy of continuous product improvement by incorporating the latest developments and components into our equipment, even up to the time of dispatch.

All major changes are incorporated into up-dated editions of our manuals and this manual was believed to be correct at the time of printing. However, some product changes which do not affect the instructional capability of the equipment, may not be included until it is necessary to incorporate other significant changes.

COMPONENT REPLACEMENT

Where components are of a 'Safety Critical' nature, i.e. all components involved with the supply or carrying of voltages at supply potential or higher, these must be replaced with components of equal international safety approval in order to maintain full equipment safety.

In order to maintain compliance with international directives, all replacement components should be identical to those originally supplied.

Any component may be ordered direct from Feedback or its agents by quoting the following information:

- | | |
|------------------------|----------------------------|
| 1. Equipment type | 2. Component value |
| 3. Component reference | 4. Equipment serial number |

Components can often be replaced by alternatives available locally, however we cannot therefore guarantee continued performance either to published specification or compliance with international standards.



PREFACE

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1



DECLARATION CONCERNING ELECTROMAGNETIC COMPATIBILITY

Should this equipment be used outside the classroom, laboratory study area or similar such place for which it is designed and sold then Feedback Instruments Ltd hereby states that conformity with the protection requirements of the European Community Electromagnetic Compatibility Directive (89/336/EEC) may be invalidated and could lead to prosecution.

This equipment, when operated in accordance with the supplied documentation, does not cause electromagnetic disturbance outside its immediate electromagnetic environment.

COPYRIGHT NOTICE

© Feedback Instruments Limited

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without the prior permission of Feedback Instruments Limited.

ACKNOWLEDGEMENTS

Feedback Instruments Ltd acknowledge all trademarks.

IBM, IBM - PC are registered trademarks of International Business Machines.

MICROSOFT, WINDOWS 95, WINDOWS 3.1 are registered trademarks of Microsoft Corporation.

MATLAB is a registered trademark of Mathworks Inc.



Table of Contents

1	INTRODUCTION	1-1
2	MODELLING AND CONTROL PROBLEMS	2-3
2.1	The system model in SIMULINK	2-13
3	IDENTIFICATION.	3-1
3.1	Start of the identification experiment.	3-1
3.2	Identification of the pendulum friction.	3-3
3.2.1	Test of encoder quality.	3-7
3.3	Minimum force needed to move the cart.	3-8
3.4	Cart friction identification.	3-12
3.5	Dead zones identification DZcv, DZpv	3-16
3.6	Impulse excitation	3-20
3.7	Results of pendulum identification process.	3-23
3.8	Identified parameters.	3-26
3.9	Verification and tuning.	3-27
3.10	Optimisation of J_p and f_p	3-32
4	PID CONTROLLERS.	4-1
4.1	SIMULINK models for PID double controller	4-4
4.2	Crane mode (2 PID for Pendulum Angle And Cart Position)	4-5
4.3	Self erecting mode	4-9
4.4	Examples of PID controllers, used in the simulation model and real system.	4-14



DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

CONTENTS

5	LINEAR-QUADRATIC CONTROLLERS	5-1
5.1	LQ control principles	5-1
5.2	Design of LQ controllers (crane problem)	5-4
5.3	LQ control experiments.	5-12
5.4	Discrete LQ controllers	5-22
6	LIST OF DIRECTORIES AND FILES :	6-1



1 INTRODUCTION

In this manual you will find the following sections:

- Modelling and control problems
- Identification
- PID control
- LQ control.

When you use the manual for the first time, you can start experiments from the demonstrations provided on the software installation disk. However, before exploring the examples, you need to know if the system model is compatible with reality.

Default parameters are stored in the model but it is recommended to pass through the identification procedure, as it cannot be guaranteed that they are well identified.

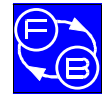
At the beginning of this manual you find the model of the cart-pendulum system. It contains several dedicated blocks added to obtain the best compatibility with the real laboratory system. The model is equipped with a set of default parameter values. The parameters can be divided into three groups:

- Constant (not to be identified)
- Well identified
- Poorly identified.

For example the pendulum and the cart are produced with production tolerances and so the masses are assumed to be identical.

A different case is with the moment of inertia of the rotating pendulum. It is slightly different in different sets. The reason is simple: the pendulum is assembled by the user in the laboratory and so small differences in the dimensions of the assembled set may occur. Unfortunately, in a real system there is usually a group of poorly identified parameters, characterised by random behaviour. In the pendulum-cart system, parameters related to the cart friction belong to this group.

The identification procedures generate experiments to obtain friction parameters to be used in the *Friction* block of the model. We must note that the cart friction function vs. cart velocity is an approximation of the real friction force. It is not sufficient to identify it once, but must be verified and usually tuned by signals exciting the simulation model and the real system.



CHAPTER 2

MODELLING AND CONTROL PROBLEMS

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

Verification and tuning is an obligatory step at the end of identification. If the time responses of the model and the real system are close to each other, as far as shape and amplitude of signals are concerned, then we can trust in our modelling.

The PID and LQ controllers presented in sections 4 and 5 are fortunately not very sensitive to model errors. Nevertheless, they also require an appropriate model of the system. The user is equipped with the simulation tools to build PID or LQ controllers. The parameters of simulation can be verified on the real system in two ways. First the existing demo files can be used where both controllers are invoked from the main control window *pendulum* m-file. Second the *pd_2ddn.m* can be accessed from the MATLAB Command Window, where the external interface file for the PID control is stored. This file is also used for both practicals concerning PID and LQ controllers.

Matlab toolboxes necessary in order to execute the experiments shown in this manual:

- Control System version 4.0.1
- Signal Processing version 4.0.1
- Optimization version 1.5.1
- Non-linear Control version 1.1.1
- Simulink version 2.1
- Matlab version 5.1

Typographical conventions

The following conventions are used throughout this manual.

Italics *Variables, SIMULINK blocks, files, function and button names.*

Grey background Text typed in the MATLAB Command Window

Courier New Results shown in the MATLAB Command Window

Installation Instructions

Ensure that Windows and the basic Digital Pendulum software are installed and running properly. (Refer to Manual 33000M5 - Feedback MATLAB Based Control Products – Software Installation).

After installation your \Pendulum directory contains all the required files and subdirectories, which are listed in Section 6.



2 MODELLING AND CONTROL PROBLEMS

The pendulum-cart system can illustrate several complex and nontrivial problems of control theory. The following characterisation is essential for this system:

- four state variables and one control variable
- the dynamics are described by one ordinary differential equation (ode) of the fourth order or by four ordinary differential equations of the first order.
- there are an infinite number of equilibrium points for the uncontrolled system; there are also an infinite number of equilibrium points for the system with linear feedback.

For our practicals we choose three equilibrium points:

- The first point is stable - the cart at a point close to the left end of the rail and the pendulum in its down position
- The second point is also stable - the cart at a point close to the right end of the rail and the pendulum in its down position
- The third point is unstable - the cart at the centre of the rail and the pendulum in its upright position.

- the system is non-linear
- the control is bounded
- the length of the rail is limited and the cart position is bounded

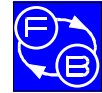
A number of key words can define in short the content of topics related to the pendulum-cart system. The underlined key words below refer to the material presented in this Teaching Manual 1.

Control problems

- crane steering from point to point
- pendulum swinging up to the vicinity of the upright position and stabilisation in the upright position

Quality criteria

- minimisation of the Integral Absolute Error (IAE)



CHAPTER 2

MODELLING AND CONTROL PROBLEMS

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

- minimum cost (energy)
- time-optimal

Controllers

- PID
- LQ

Tools applied

- mathematical model

Programming environment

- MS Windows
- unique Real Time Kernel
- MATLAB & SIMULINK
- MATLAB toolboxes: CONTROL, SIGNAL PROCESSING, NONLINEAR CONTROL DESIGN, etc.

There are two essential components of virtually any controller: a mechanism for knowledge representation and a mechanism for decision-making. With conventional controllers knowledge is represented by a mathematical model (such as a set of differential equations) and decision-making is based on optimisation.

When the system is complex or less precisely known, it becomes difficult to characterise the knowledge adequately with conventional means. Alternative ways must be found to encode the available knowledge about the system and can result in the development of intelligent controllers.

Note also however, that a control algorithm can be built without any model. This is the so called “Black box” technique.

The closed-loop responses with different controllers are more or less sensitive to variation of the model parameters. The LQ and PID controllers are not especially sensitive, but in most cases, however, a precise identification of the dynamics of the laboratory model is essential for the generation of an appropriate control algorithm.



The position measurement signals from the encoders play a crucial role in achieving the control goal. Measurement technique is another factor influencing the quality of the control. The velocities in the system are not measured, they are observed and poor observation algorithms may be a source of serious errors.

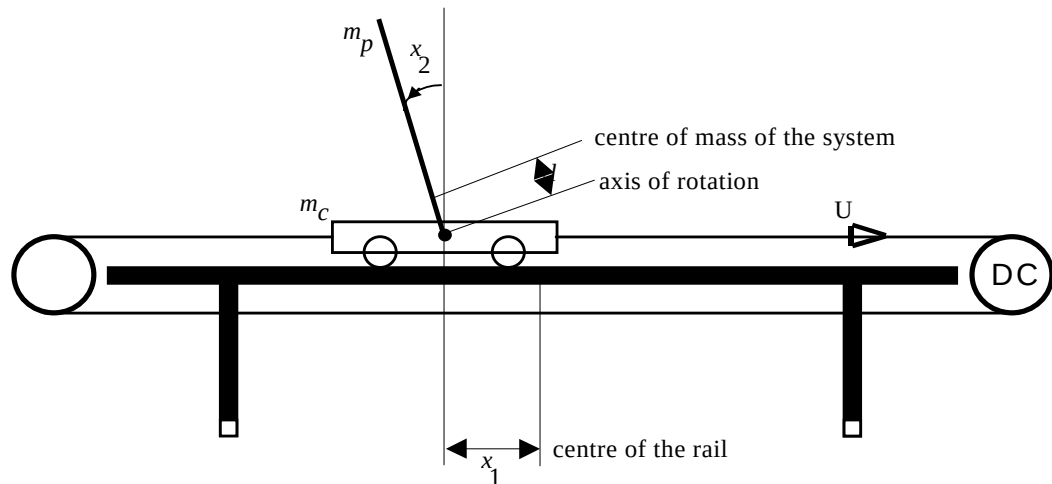


Figure 2-1: Laboratory model of Pendulum-Cart System

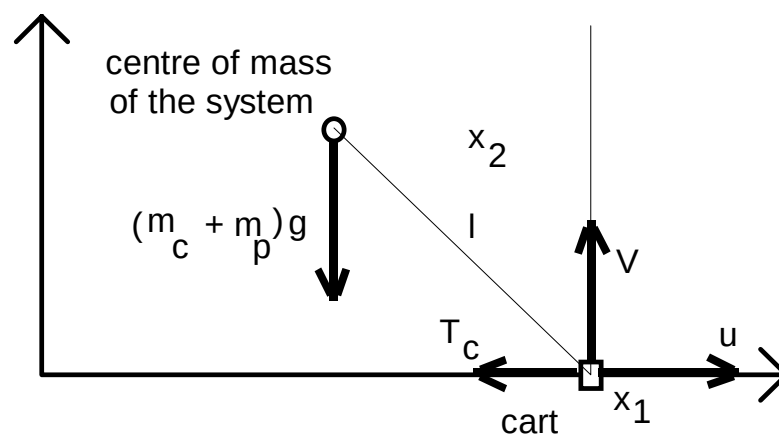
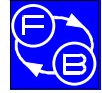


Figure 2-2: Showing the forces acting on the cart



CHAPTER 2

MODELLING AND CONTROL PROBLEMS

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

Figure 2-1 shows a schematic of the Pendulum-Cart system, whilst Figure 2-2 shows the forces acting.

The pendulum rotates in a vertical plane around an axis located on a cart. The cart can move along a horizontal rail whose long axis lies in the plane of rotation. A control force U , parallel to the rail, is applied to the cart. The mass of the cart is denoted by m_c and the mass of the pendulum, by m_p . l is the distance from the axis of rotation to the centre of mass of the pendulum-cart system. J is the moment of inertia of the pendulum-cart system with respect to the centre of mass.

The state of the system is the vector $x = \text{col}(x_1, x_2, x_3, x_4)$ where:

x_1 is the cart position (distance from the centre of the rail)

x_2 is the angle between the upward vertical and the ray pointing at the centre of mass, measured counter-clockwise from the cart ($x_2 = 0$ for the upright position of the pendulum)

x_3 is the cart velocity

x_4 is the pendulum angular velocity

T_c denotes the friction in the motion of the cart

D_p is the moment of friction in the angular motion of the pendulum, proportional to the angular velocity: $D_p = f_p x_4$

The force of reaction of the rail V acts vertically on the cart. As the horizontal co-ordinate of the centre of mass is equal to $x_1 - l \sin x_2$ and the vertical to $l \cos x_2$, the equations of motion are as follows:

$$(m_c + m_p) (x_1 - l \sin x_2)'' = F - T_c,$$

$$(m_c + m_p) (l \cos x_2)'' = V - (m_c + m_p)g,$$

$$J x_2'' = (F - T_c) l \cos x_2 + V l \sin x_2 - D_p.$$

$(.)''$ denotes the second derivative with respect to time t and $(.)'$ denotes the first derivative with respect to time t . The first two equations describe the translation of the centre of mass, while the third describes the rotation of the whole system around the centre of mass. After the elimination of V and simple calculations we obtain the state equations (for $t \geq 0$)

$$x_1' = x_3,$$

$$x_3' = \frac{a(F - T_c - \mu x_4^2 \sin x_2) + l \cos x_2 (\mu g \sin x_2 - f_p x_4)}{J + \mu l \sin^2 x_2}$$



$$\dot{x}_2' = x_4,$$

$$\dot{x}_4' = \frac{l \cos x_2 (F - T_c - \mu x_4^2 \sin x_2) + \mu g \sin x_2 - f_p x_4}{J + \mu l \sin^2 x_2} \quad (2.1)$$

where

$$a = l^2 + \frac{J}{m_c + m_p}, \quad \mu = (m_c + m_p)l. \quad (2.2)$$

The admissible controls are bounded

$$|F(t)| \leq M. \quad (2.3)$$

Cart friction

The cart friction T_c in the model is a non linear function of the cart velocity x_3 , as shown in Figure 2-3.

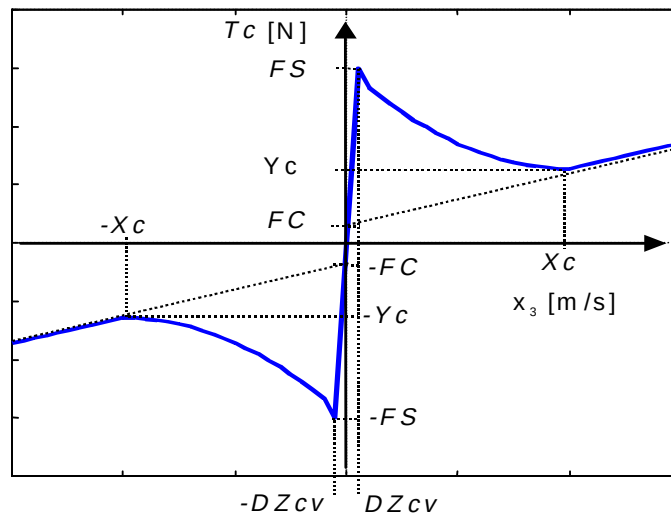


Figure 2-3: **Cart friction T_c vs. cart velocity x_3 (for a large range of x_3)**

The following notation is used in Figure 2-3.

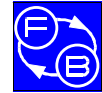
FS - static friction,

FC - dynamic or Coulomb friction ,

X_c - cart velocity - beginning of the linear dependence zone,

Y_c - friction value for X_c point,

DZ_{cv} - dead zone of cart velocity.



CHAPTER 2

MODELLING AND CONTROL PROBLEMS

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

Consider Figure 2-3. The diagram represents the experimental relationship between cart friction and cart velocity.

Static friction exists in the dead zone of the cart, and this requires a force to be applied in order to start movement ($0 < x_3 < DZ_{CV}$). Having overcome this initial friction, however, the static friction reduces, with an approximately quadratic relation to velocity, but the dynamic friction increases almost linearly with velocity.

The static friction eventually reaches a constant value of zero (at $x_3 = x_C$), at which point the total friction becomes equal to the dynamic or Coulomb friction.

Since friction always acts in a direction opposite to the direction of motion, the function is mirrored for negative values of x_3 with the friction force changed in sign.

In order to obtain some analytical results for the total friction force acting on the cart, we divide the friction function into 5 zones.

- | | | |
|----|----------------------------|--------|
| 1) | $x_3 < -x_C$ | Zone 1 |
| 2) | $-x_C < x_3 < -DV_{CV}$ | Zone 2 |
| 3) | $-DV_{CV} < x_3 < DV_{CV}$ | Zone 3 |
| 4) | $DV_{CV} < x_3 < x_C$ | Zone 4 |
| 5) | $x_3 > x_C$ | Zone 5 |

We shall only consider cases 3) 4) and 5) since 1) and 2) are mirror images of 3) and 4)

Method of deriving the equations for the linear and quadratic parts of the friction function

The relationship for the linear and quadratic parts of T_C , the friction function, are derived as follows:



Linear

Two points are needed for a solution:

The standard equation for a line passing through the points (x_1, y_1) and (x_2, y_2) is

$$y - y_1 = \frac{(y_2 - y_1)}{(x_2 - x_1)} \cdot (x - x_1) \quad 2.4$$

Quadratic

Either three points are needed for a solution, **or** two points **and** a known gradient at a point. Use the following assumptions:

The equation is of the form $y = ax^2 + bx + c$. i)

The curve passes through (x_1, y_1) and (x_2, y_2) ii)

The gradient at (x_2, y_2) is G . iii)

From i) $y' = 2ax + b$

From iii) $2ax_2 + b = G$, and therefore $b = G - 2ax_2$ iv)

Therefore from i) $y = ax^2 + (G - 2ax_2)x + c$ v)

From ii) and iv) $y_1 = ax_1^2 + (G - 2ax_2)x_1 + c$ and vi)

$$y_2 = ax_2^2 + (G - 2ax_2)x_2 + c \quad \text{vii)}$$

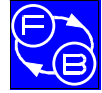
Subtracting vi) from vii) gives

$$y_2 - y_1 = a(x_2^2 - x_1^2) + (G - 2ax_2)(x_2 - x_1) \quad \text{viii)}$$

Hence a can be found from viii), b from iv) and c from vi)

From Figure 2-3 replace x_1, y_1 by $0, F_S$. Replace x_2, y_2 by x_C, y_C and note that the gradient at this point is zero ($G=0$ at $x=x_C$). Also at $x=0$ the friction is the static friction only (F_S) as the speed (x_3) is zero, - hence $c=F_S$, and from iv) $b=-2ax_C$

$$\text{From viii) } a = \frac{(y_2 - y_1)}{(x_1 - x_2)^2} \quad \text{which is equivalent to } \frac{(y_C - F_S)}{(x_C)^2}$$



CHAPTER 2

MODELLING AND CONTROL PROBLEMS

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

Zone 3

The Friction linearly increases in the range $DV_{cv} < x_3 < DV_{cv}$

From Figure 2-3 the value of the Friction at $x_3 = \pm DV_{cv}$ is $\pm F_S$

Using the linear method and equation 2.4, and noting that this linear part passes through the origin, we derive the relationship

$$T_C = \frac{F_S \cdot X_3}{DV_{cv}}$$

Zone 4

The static friction has a quadratic relation with velocity, decreasing as the velocity increases and eventually reaching zero, and the dynamic friction has a quadratic relation with velocity. The total friction, which is the sum of the static and dynamic elements, therefore has a quadratic relation with velocity.

We note that the static friction reaches a constant value of zero at $x_3 = x_c$. This implies that its gradient at this point is zero. Using the quadratic method described above, with $G=0$, gives the following relation for T_C

$$T_C = \frac{(F_S - Y_C)}{X_c^2} \cdot (X_3)^2 - \frac{2(F_S - Y_C)}{X_c} \cdot X_3 + F_S$$

Zone 5

The static friction has reached zero and the dynamic or Coulomb friction increases linearly with velocity.

Using the linear method and equation 2.4, and noting that this linear part passes through the point (x_c, y_c) we derive the relationship

$$T_C = \frac{(Y_C - F_C)}{X_c} \cdot x_3 - F_C$$



Moment of Inertia

The figure depicted in Figure 2-4 is used to illustrate calculations of the Moment of Inertia of the rotating pendulum

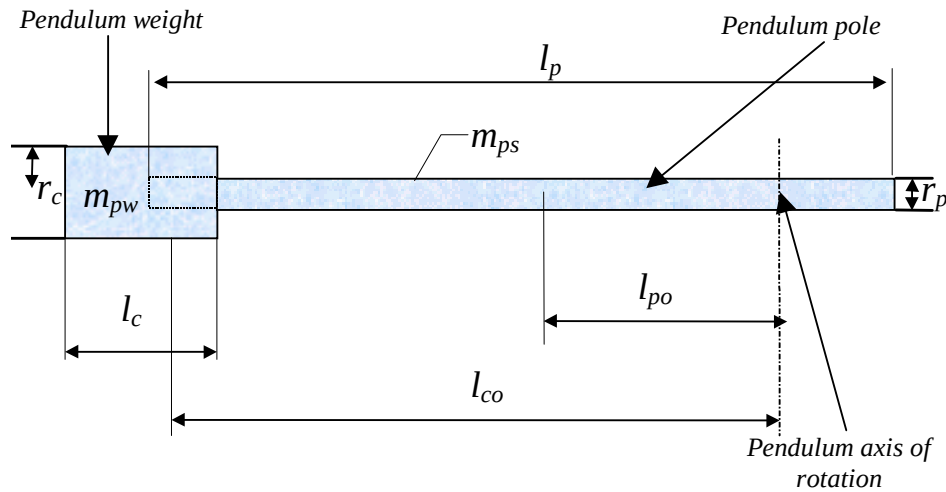


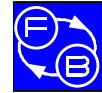
Figure 2-4: View of the pendulum

The following notation is used in Figure 2-4.

- m_{pw} - mass of the load pendulum weight[kg],
- m_{ps} - mass of the pendulum pole [kg],
- l_p - length of the pole [m],
- l_{po} - distance between centre of the pole mass and the pendulum rotation axis [m],
- l_c - length of load [m],
- l_{co} - distance between centre of the load mass and the pendulum rotation axis [m],
- r_p - radius of the pole [m],
- r_c - radius of the load [m].

Moment of inertia of pendulum related to the pendulum rotation axis:

$$J_p = \left[\frac{1}{12} m_{pw} l_c^2 + \frac{1}{4} m_{pw} r_c^2 + m_{pw} l_{co}^2 \right] + \left[\frac{1}{12} m_{ps} l_p^2 + \frac{1}{4} m_{ps} r_p^2 + m_{ps} l_{po}^2 \right] \quad (2.4)$$



CHAPTER 2

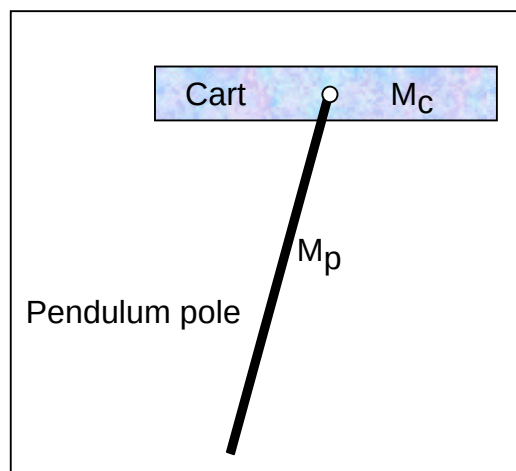
MODELLING AND CONTROL PROBLEMS

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

We shall make the following assumptions in order to simplify the model.

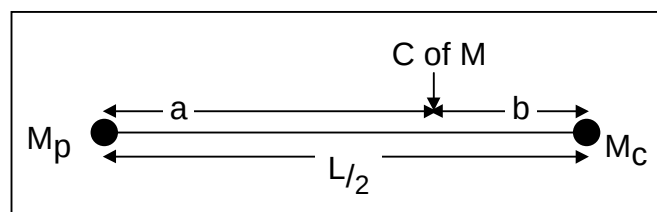
- The pendulum is a uniform **thin** cylinder of length L metres and mass m_p
- The pivot position is at the mass centre of the cart and at one end of the pole
- The cart behaves as a point mass M_c , at the pivot point

Consider the pendulum cart system and consider the moment of inertia about the mass centre of the system



Now it is a well known principle that if J is the Moment of Inertia about a first axis through the mass centre and J_p is the Moment of Inertia about a second parallel axis separated by d from the first, then $J_p = J + Md^2$ (2.5)

The centre of mass (C of M) of the system is such that $M_p a = M_c b$, and $a + b = L/2$... (2.6)



Hence the total Moment of Inertia about the mass centre (from (2.5))

is given by $J = M_p a^2 + M_c b^2 + \frac{M_p L^2}{12}$ (2.7)

And so $a = \frac{L \cdot M_c}{2(M_p + M_c)}$ and $b = \frac{L \cdot M_p}{2(M_p + M_c)}$ from (2.6)..... (2.8)



From (2.7) and (2.8) the Moment of Inertia is

$$J = \frac{m_p(4m_c + m_p)L^2}{12(m_c + m_p)} \quad (2.9)$$

J can be expressed in terms of J_p , since there is no rotation of the cart about its own mass centre

$$J = J_p - l^2(m_c + m_p) \quad (2.10)$$

The distance from the centre of mass to the axis of rotation equals

$$l = \frac{0.5m_p L}{m_c + m_p} \quad (\text{from 2.8}) \quad (2.11)$$

2.1 THE SYSTEM MODEL IN SIMULINK

The simulation model of the real pendulum-cart system (*model.m in Simulink*) is presented in Figure 2-1, Figure 2-2 and Figure 2-3.

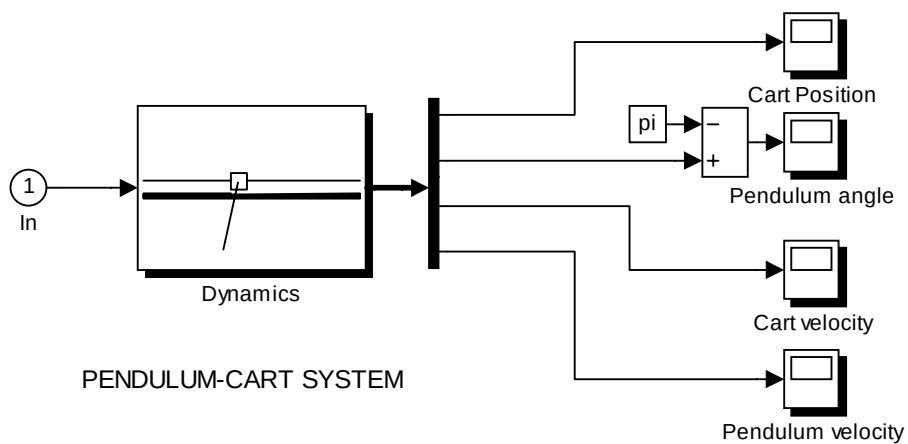


Figure 2-1: Pendulum-cart model *model.m*

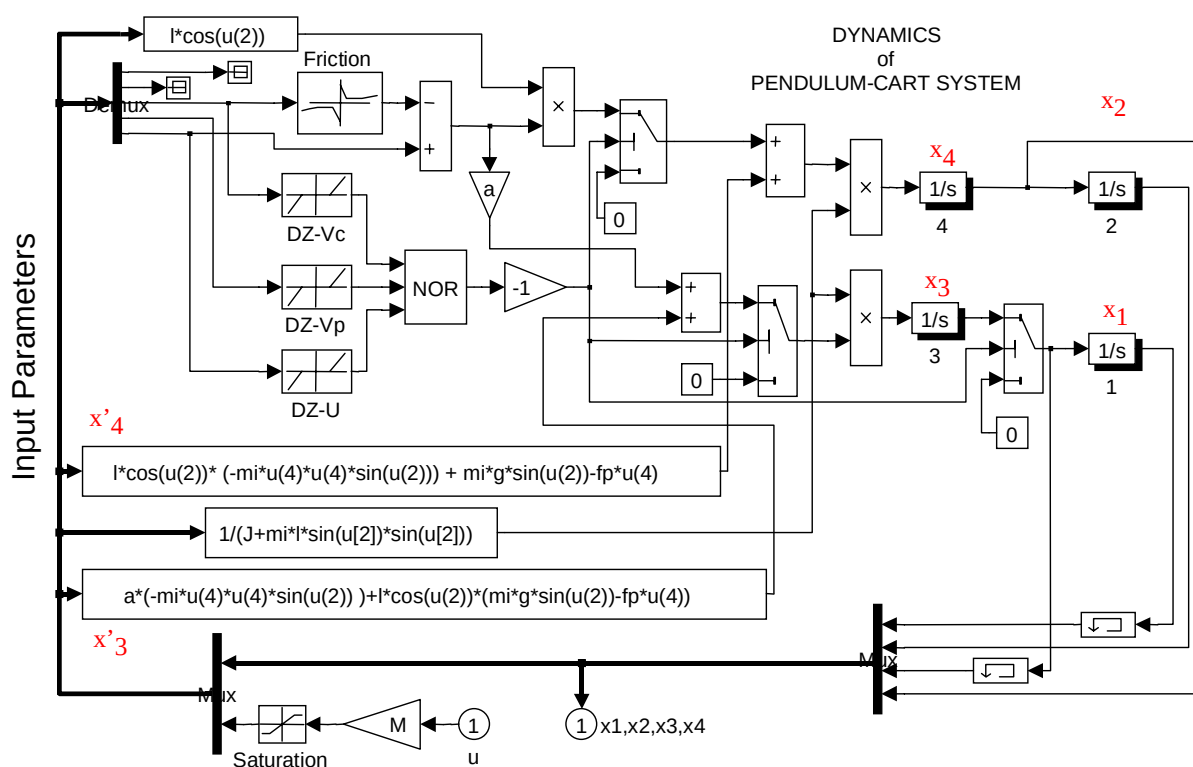


Figure 2-2: Dynamics block

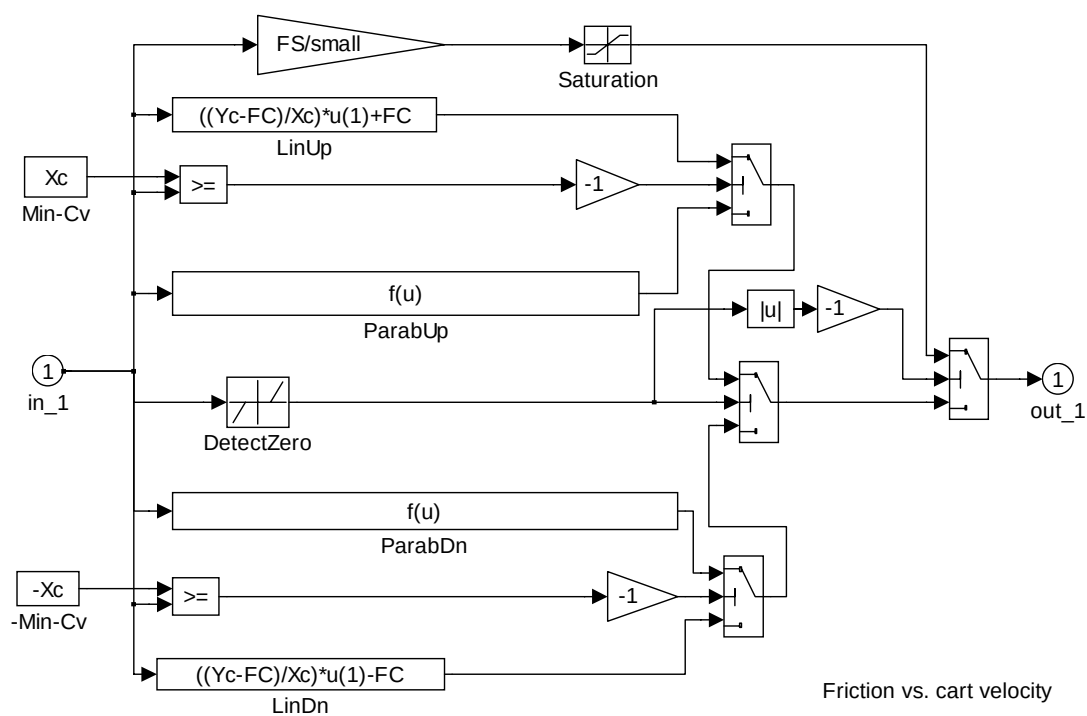


Figure 2-3: Friction block



The cart-pendulum dynamics (equations from (2.1) to (2.5)) are modelled in the masked block *Dynamics*. The unmasked block *Dynamics* is given in Figure 2-2. It contains several specialised blocks from the SIMULINK library such as *Dead zone* and *Saturation*, and the *Friction* block dedicated to friction representation.

Friction is the most important block for the identification procedure. The interior of this block is shown in Figure 2-3. Omitting or simplifying the *Friction* block results in poor compatibility between the real and simulation models responses. Two memory blocks are introduced to avoid the generation of algebraic loops. There are three *Dead zone* blocks to make the model sensitive to small values of velocities and control.

The default simulation parameters are given in Figure 2-4 and Figure 2-5.

Notice, that the fifth order Runge Kutta integration method with the constant step 0.01 [s] is used.

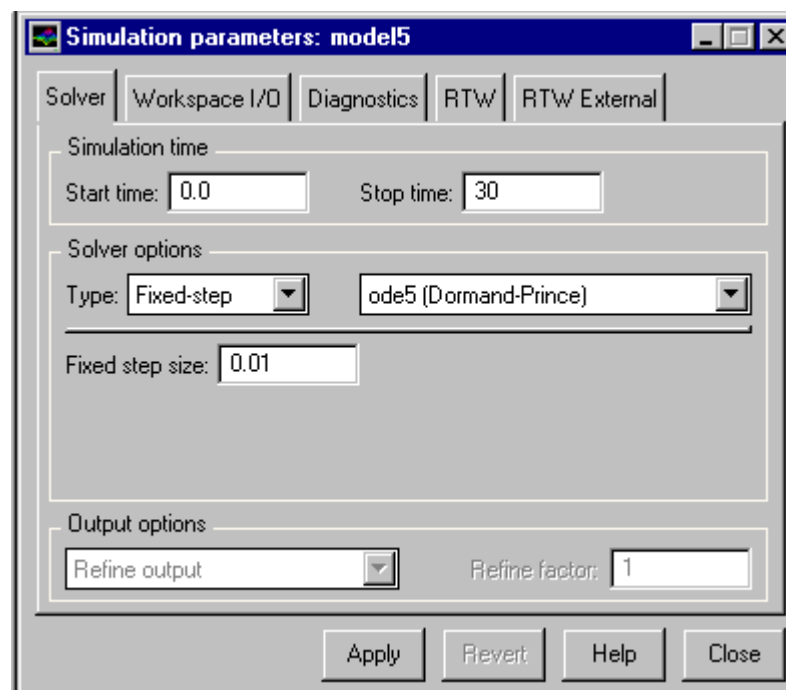
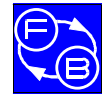


Figure 2-4: Parameters of simulation - Solver



CHAPTER 2

MODELLING AND CONTROL PROBLEMS

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

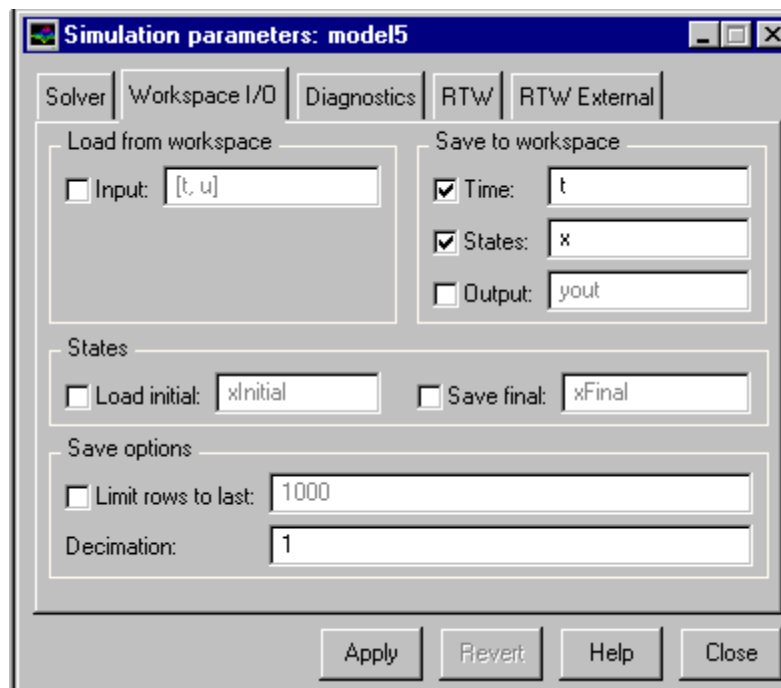


Figure 2-5: Parameters of simulation - Workspace I/O



3 IDENTIFICATION.

To start identification call *p_id* file in the MATLAB Command Window. The main window of the identification process is shown in Figure 3-1.

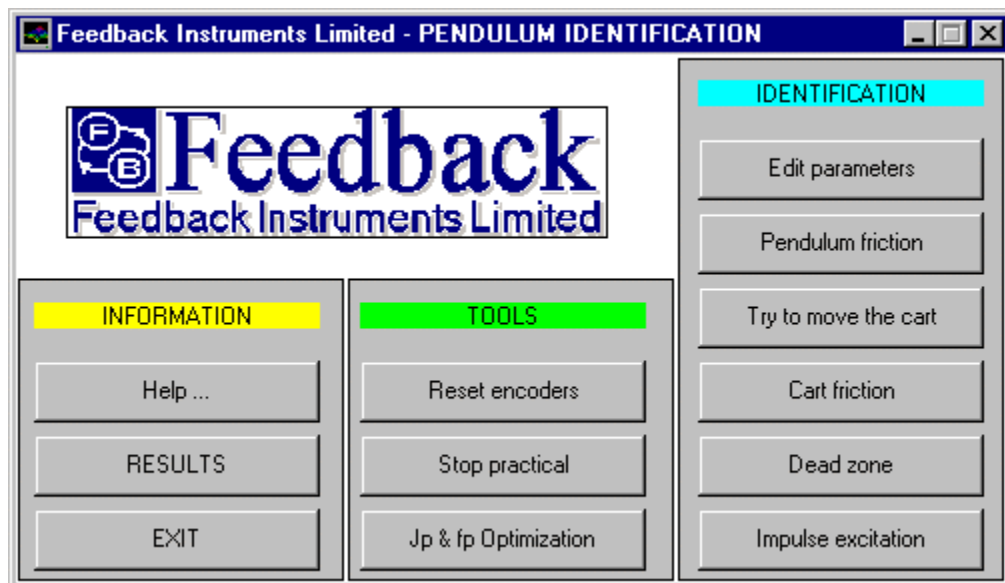


Figure 3-1: Pendulum identification - Main window

To get important information about the identification process click on the **Help ...** button. To close the identification main window click on the **EXIT** button.

3.1 START OF THE IDENTIFICATION EXPERIMENT.

Bring the cart to the centre of the rail. No motion of cart and pendulum should be observed! If you had used the pendulum before you started *p_id.m* file, double click on the **Stop practical** followed by a double click on the **Reset encoders**¹ button from TOOLS section.

¹See the Getting Started Manual 33-005-1



CHAPTER 3

IDENTIFICATION

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

The pendulum-cart parameters are loaded automatically when *p_id.m* is started. You can edit all constant parameters by clicking the **Edit parameters** button in the IDENTIFICATION section. The parameters of the pendulum are loaded from the *pendpar.mat* file and displayed in the window (Figure 3-3). When the file does not exist in the Feedback Pendulum Identification Toolbox directory the error message is displayed (Figure 3-1). If the *pendpar.mat* file is absent in your directory it will be created after accepting model parameters.

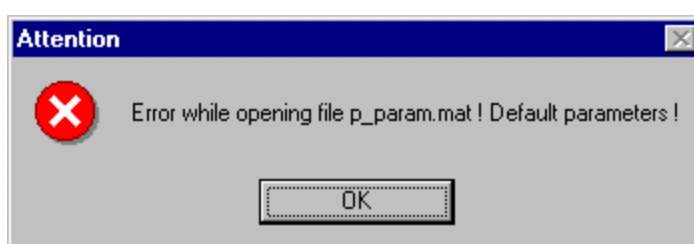


Figure 3-1: Parameters file open error

Next you will be asked to accept the parameters, or to change them by editing in the parameters window (Figure 3-3). When you edit one of the parameters click **OK** to store the to the *pendpar.mat* file. Click **Cancel** button to start the identification procedure.

Each parameter contains date and time, to let you better recognise the identification time.

PARAMETER DESCRIPTION	SYMBOL	VALUE	ACTUALISED
Cart mass [kg] :	mc	1.12	1997-09-15 13:27
Cart mass without positioners [kg] :	mc_	1.045	1997-09-15 13:27
Pendulum weight mass [kg] :	mpw	0.095	1997-09-15 13:27
Pendulum stick mass [kg] :	mps	0.025	1997-09-15 13:27
Rail length [m] :	RI	1	1997-09-15 13:27
Pendulum stick length [m] :	lp	0.402	1997-09-15 13:27
Distance lpo [m] :	lpo	0.146	1997-09-15 13:27
Length of load[m] :	lc	0.041	1997-09-15 13:27
Distance lco [m] :	lco	0.347	1997-09-15 13:27
Radius of the load [m] :	rc	0.02	1997-09-15 13:27
Radius of the stick [m] :	rp	0.006	1997-09-15 13:27

Figure 3-3: Model parameters measured and set by Feedback Ltd.



3.2 IDENTIFICATION OF THE PENDULUM FRICTION.

Matlab program p_idpf.m

Block the cart at an arbitrary point of the rail using the special stoppers supplied and which are shown in Figure 3-4.

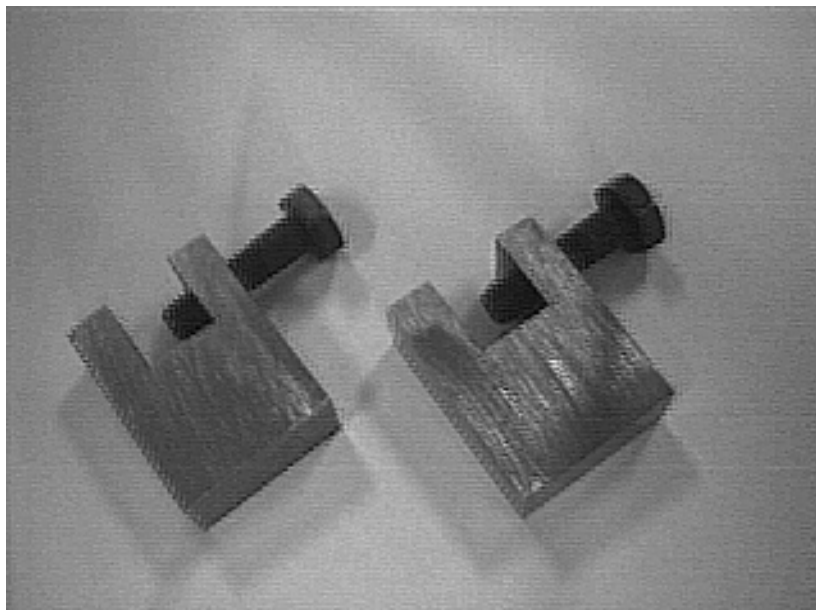


Figure 3-4: Blocking stoppers

The cart is mechanically blocked. Click on the **Pendulum friction** button placed in the IDENTIFICATION section of the main identification window. The identification procedure guides you step by step to perform the experiment.

First raise the pendulum to $\pi \pm \pi/9$ (20 degrees up from the stable down position). If you click on the **Check Angle** button in Figure 3-5, you can see the current value of the pendulum angle (assuming that you hold the pendulum in the raised position).



CHAPTER 3

IDENTIFICATION

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

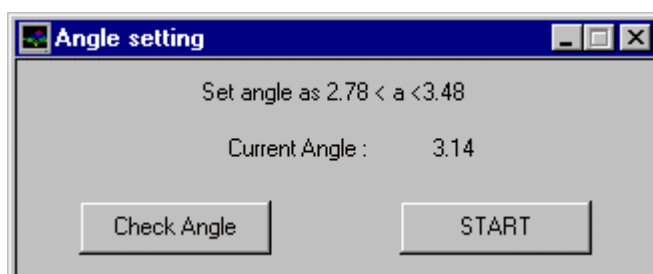


Figure 3-5: Angle checking window

Click the START button to continue identification of the pendulum angle. The following window appears on the screen (Figure 3-6).

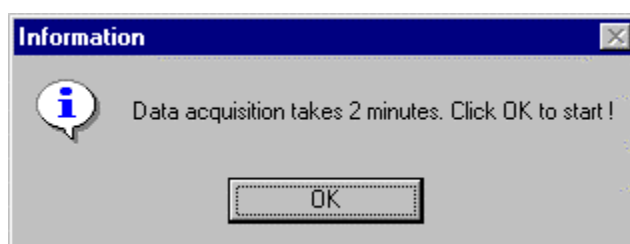


Figure 3-6: Start of the algorithm

You are informed that data acquisition is in progress. The algorithm starts using the default model parameters. The algorithm measures the angle for a defined time period in order to calculate the friction of the pendulum, based on an exponential decay. The pendulum is oscillating and data are being gathered which will be used to calculate friction coefficients. The following window containing progress bar will appear on the screen (Figure 3-7).



Figure 3-7: Identification in progress



At least eight oscillations are required to start the data acquisition in the algorithm. If the pendulum does not move (oscillations are not observed), the identification process breaks and you see the message in Figure 3-8, and you will have to start this step again.



Figure 3-8: Oscillations not observed

The algorithm divides its operation time between data acquisition and calculations. The following actions are performed:

- calculation of the mean value of the pendulum period T , (i.e. the time between two peaks)
- calculation of the pendulum friction (damping) coefficient,

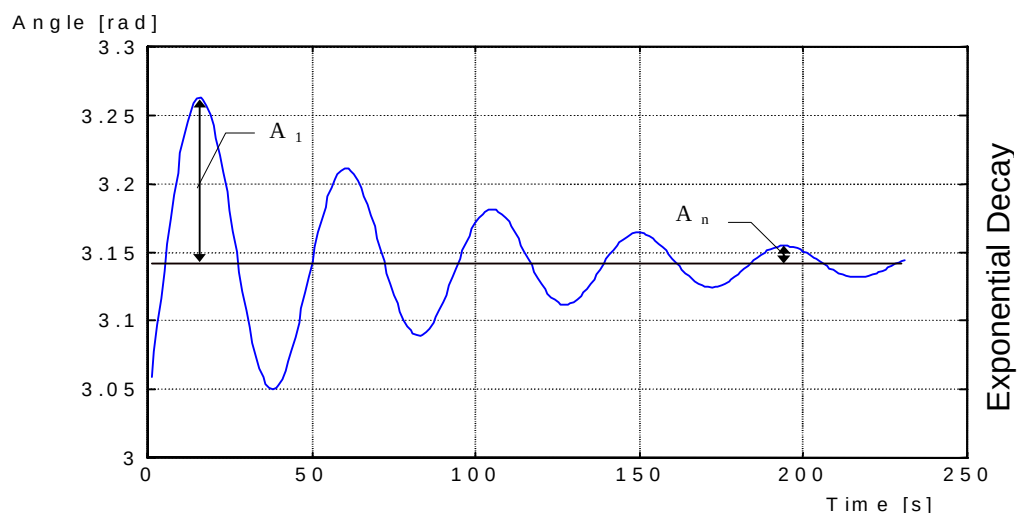


Figure 3-9: Damped oscillations of the pendulum.



CHAPTER 3

IDENTIFICATION

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

The damping coefficient is calculated on the basis of local maximum values A_1 , and A_n (Figure 3-9) with the following formula.

$$f_p = \frac{J_p}{nT} \log \left(\frac{A_1}{A_n} \right)$$

where :

- f_p - damping coefficient [$\text{kg}\cdot\text{m}^2/\text{s}$]
- J_p - moment of inertia of pendulum [$\text{kg}\cdot\text{m}^2$]
- n - number of local maximums upper the • value,
- T - pendulum period [s], 200 seconds by default
- A_1, A_n - local maximum values, after 1 oscillation and n oscillations respectively

When the data acquisition is finished, the information appears on the screen (Figure 3-10).

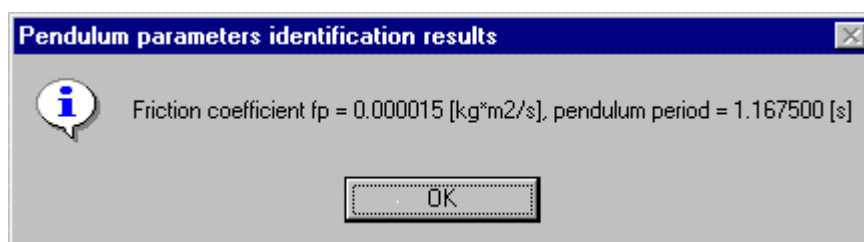


Figure 3-10: Pendulum parameters - identification results

The plot of the most important data is then automatically created (Figure 3-11).

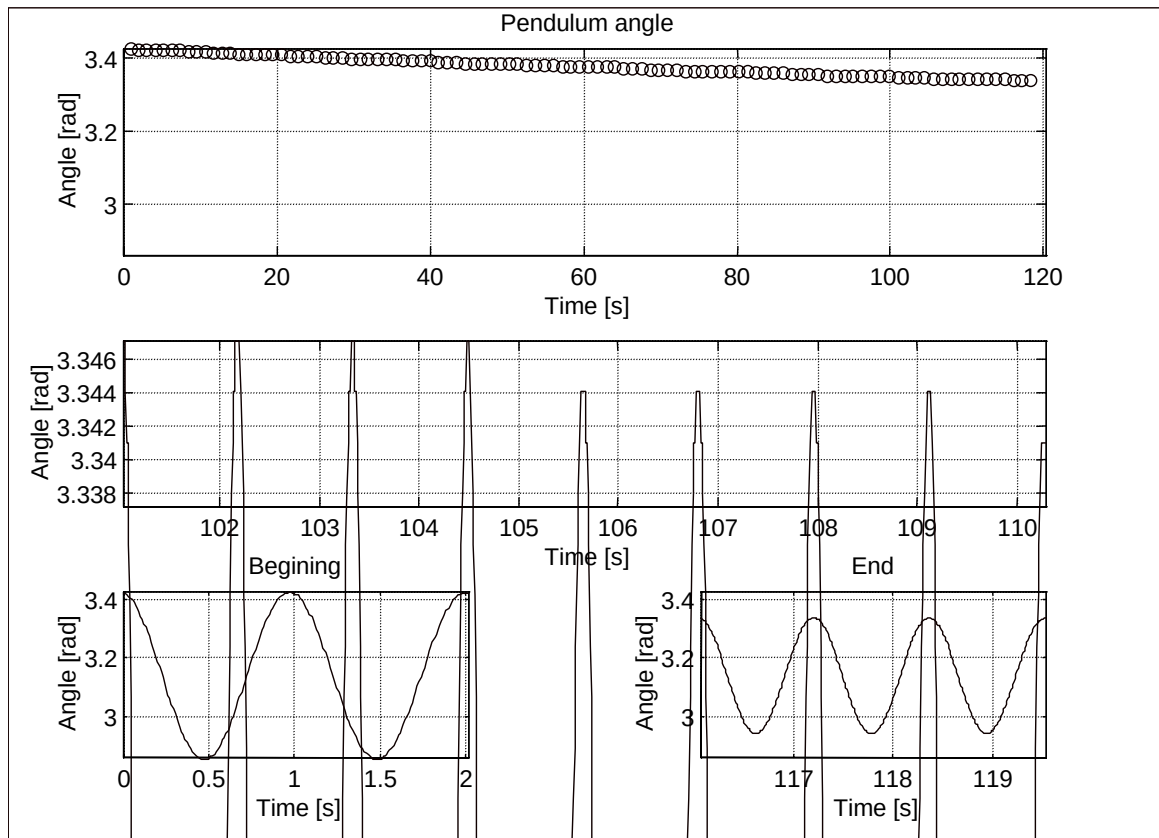


Figure 3-11: Graphical window with pendulum data.

At the end of pendulum identification the following message is displayed (Figure 3-12). Click on the **Yes** button to store identified parameters to *pendpar.mat* file.

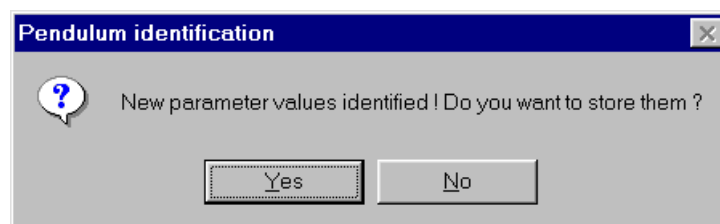
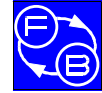


Figure 3-12: End of pendulum parameters identification

3.2.1 Test of encoder quality.

The detected amplitudes of oscillations are shown in Figure 3-11. Due to the finite accuracy of the measuring device (encoder), the collected local maximum values are only distinguished after 4 - 5 oscillations.



CHAPTER 3

IDENTIFICATION

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

This can be clearly seen in the second plot of Figure 3-11. The angle measuring device has an accuracy of 2048 [bits] per 2π [rad]. The minimum resolution is therefore:

$$\Delta\varphi = \frac{2\pi}{2048} = 0.00306796 \text{ [rad]}$$

The resolution is readable from the plot. Two plots entitled "Beginning" and "End" show local maxima A_1 , A_n which are required for further calculations.

3.3 MINIMUM FORCE NEEDED TO MOVE THE CART.

Matlab program p_idtmc.m

This part of identification is used to detect the minimum force DZu needed to move the cart. To run this part of identification procedure remove blocking stoppers (Figure 3-13) followed by click on the **Try to move the cart** button in the main identification window. The following dialog box appears.

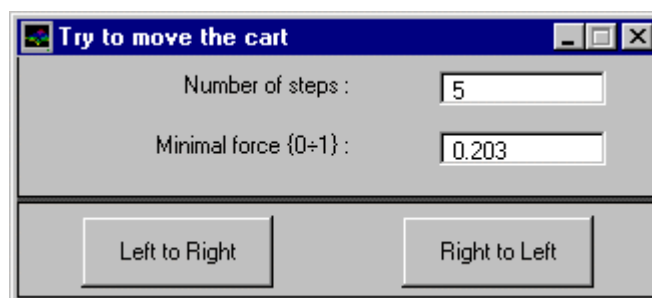


Figure 3-13: Set parameters for cart movement

Enter the force value being the initial value in each step and the number of steps (cart movements). To push the cart from left to right you should click on the **Left to Right** button and for the opposite direction, the **Right to Left** button.

Next you will be asked about motor supply. You will see the following window (Figure 3-14).

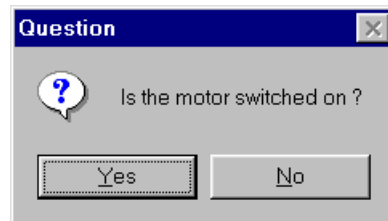


Figure 3-14

If you choose the **No** button you must turn motor on and accept the **OK** button in the following window (Figure 3-15) and next you should answer correct the previous question.

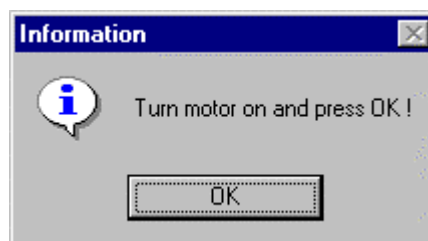


Figure 3-15

The message corresponding to your choice of motion direction (see Figure 3-16 and Figure 3-17).

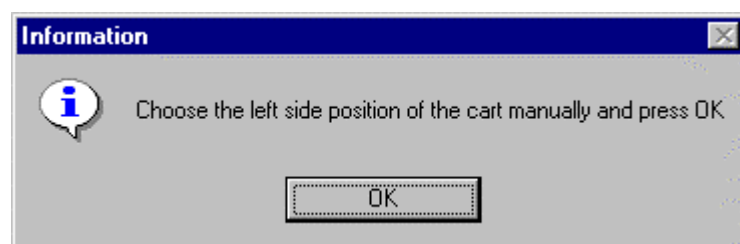


Figure 3-16 Information for *Left to Right* choice

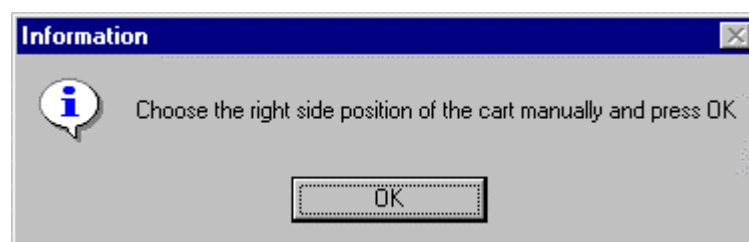


Figure 3-17 Information for *Right to Left* choice

Click the **OK** button to continue.



CHAPTER 3

IDENTIFICATION

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

If you see the message box (Figure 3-18) you can manually move the cart to a new position or the motor moves the cart.

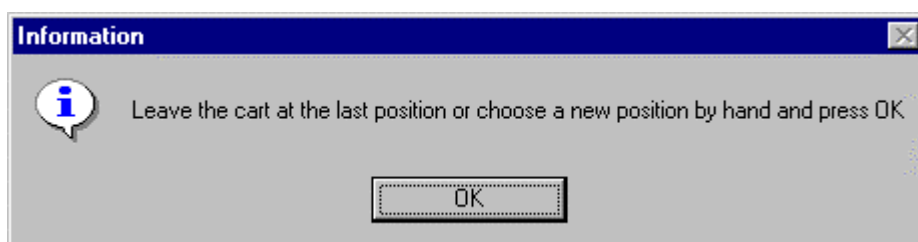


Figure 3-18: Positioning the cart

Next you see Figure 3-19 and you should wait for a while until the algorithm increases the steering force and detects the cart position. If the position has been changed by 5 millimetres the algorithm stops and this message window disappears.



Figure 3-19: Trying to move the cart

After the determined number of steps defined in the window (Figure 3-13) the procedure is complete and the result is displayed. Bars in Figure 3-20 represent the force required to move the cart at different locations on the rail.

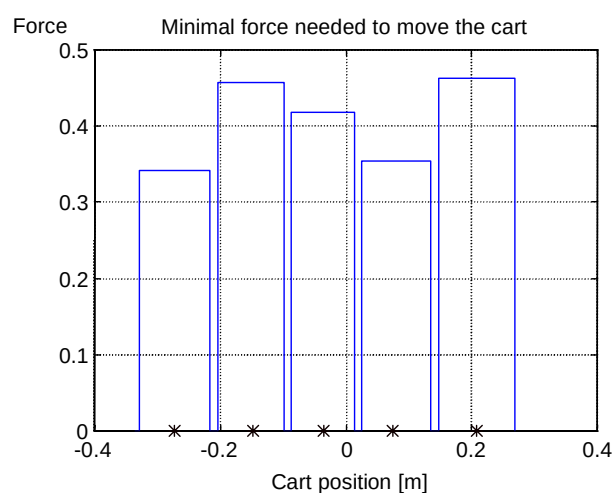


Figure 3-20: Minimal force detection



When the data acquisition is finished, the information appears on the screen(Figure 3-21).

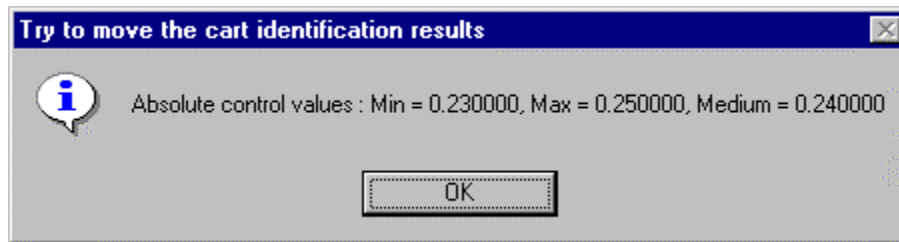


Figure 3-21 Try to move the cart - identification results

At the end of pendulum identification the following message is displayed (Figure 3-22). Make a note of this control value as it will be used in the following experiment, or you can access it from the *Results* block in *Pendulum Identification Menu*. Click the **Yes** button to store identified parameters to the *pendpar.mat* file.

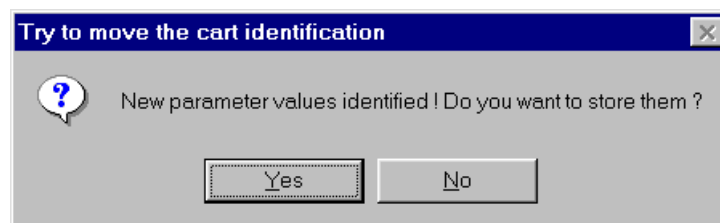
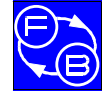


Figure 3-22 End of *Try to move the cart* identification



3.4 CART FRICTION IDENTIFICATION.

Matlab program p_idcf.m

This part of identification procedure should be run if the previous step has been finished successfully, because the average value of the detected force is a basis for this step.

This part of identification is used to detect :

- static friction of the moving cart,
- dynamic friction of the moving cart,
- constant proportionality coefficient M between the control U : $U \in [-1,1]$ produced by the computer interface and the control force F : $F \in [-M, M]$,

The equation of force equilibrium has the form:

$$F_{sf} + f_c(V_0 + a\Delta t) = MU - m_c a$$

where:

$F=MU$ Average value of the detected force from previous experiment [N]

F_{sf} - static friction force of moving cart [kg/s]

f_c - dynamic friction coefficient [kg/s]

$FC=0.1 \cdot f_c$ - dynamic friction coefficient/10 [N]

V_0 - cart velocity in early stage of motion [m/s]

a - acceleration of cart [m/s²]

Δt - time interval when steering force F is active [s]

M - proportionality coefficient (magnitude of control) [N]

U - control

m_c - cart mass without pendulum poles and weights [kg]

The experiment concerns only the cart. Unscrew the arms of pendulum. Leave only the pendulum axis together with the cart. Again the block stoppers are not used. Check the horizontal position of the rail. The edges of the rail having contact with the cart wheels must be clean and free from dust. Be sure that previous practicals are stopped and switch on the DC motor power. Click on the **Cart friction** button in the main identification window to start identification.



Next you will see the following dialog box.

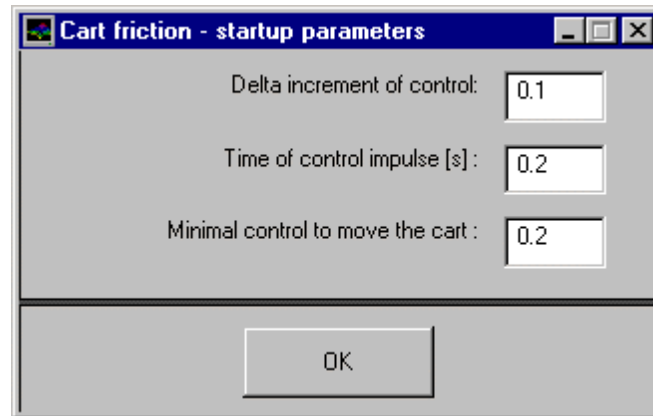
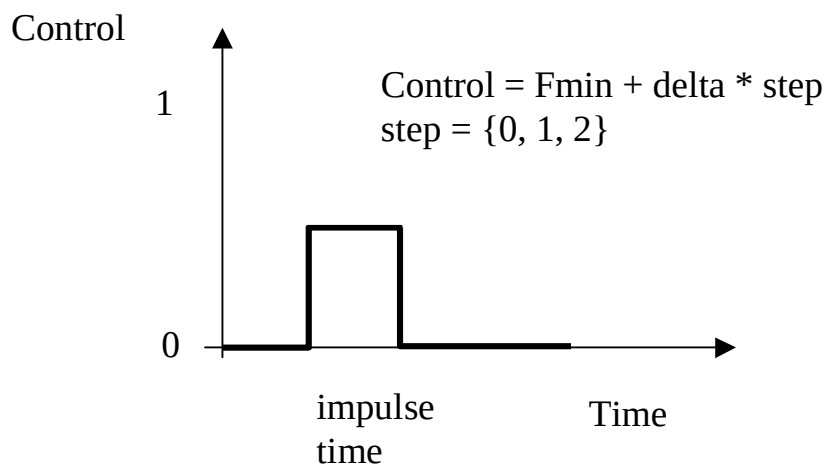


Figure 3-23: Check minimal force

Notice that the minimal control value is calculated in the experiment **“Try to move the cart”**.



The algorithm contains 3 steps. In 2nd and 3rd steps the control value increase.

To change the values, type a new value in the edit window(Figure 3-23). Click on the **OK** button. Next you will be asked about motor supply. You will see the following window.

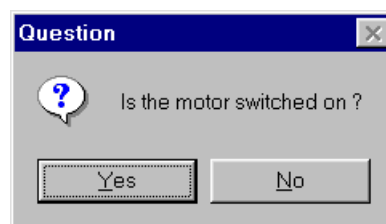


Figure 3-24



CHAPTER 3

IDENTIFICATION

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

If you choose the **No** button you must turn motor on and accept the **OK** button in the following window (Figure 3-25) and you should execute the instruction.

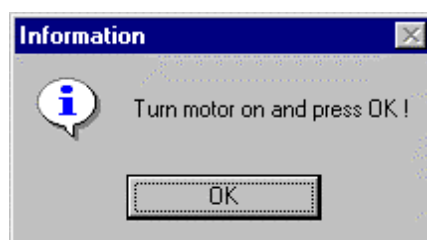


Figure 3-25

Next you will see the following window:



Figure 3-26: Prepare cart position

You must move the cart by hand to the left end of the rail (not overpassing the limit switch). Click on the **OK** button to continue. You see the information in Figure 3-27.

This part of identification consists of three identical experiments. In each of them the steering force is being monotonically increased and the cart motion data are collected. You take part interactively in these experiments by dragging the cart by hand to its initial position and accepting the message (Figure 3-26). During each experiment the following window appears (Figure 3-27).



Figure 3-27: Detection of the cart friction in progress



During the identification procedure the following errors can occur:

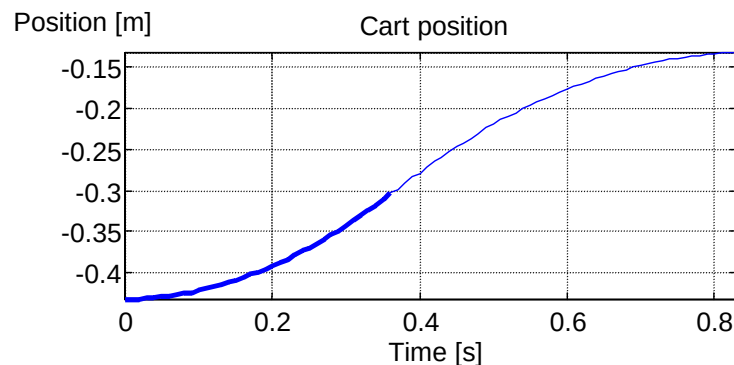
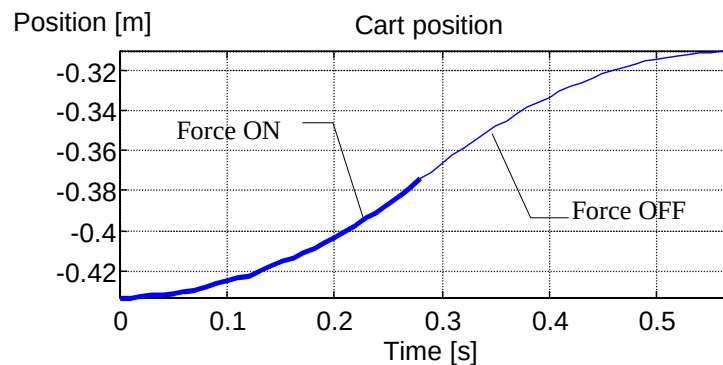


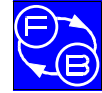
when more than one cart motion is observed, or



when the minimal control value is too small to move the cart.

At the end of these experiments the plot of the cart position is created (Figure 3-28)





CHAPTER 3

IDENTIFICATION

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

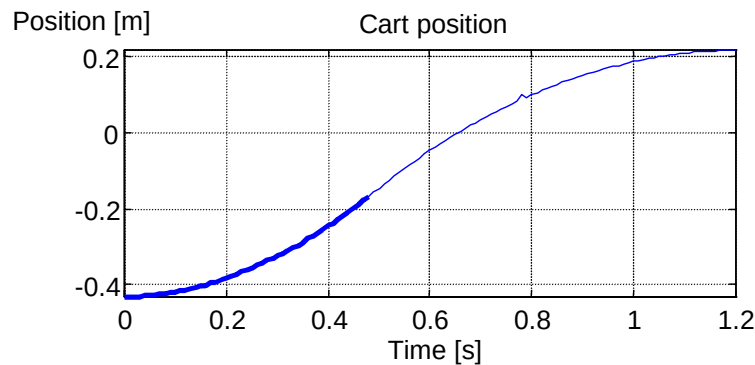


Figure 3-28: Plot containing cart movement data.

Each figure illustrates the cart motion for an increasing control force. The plot represents the cart position for the force ON (thick line) and OFF (thin line). Finally you are informed that the cart friction identification is finished (Figure 3-29).

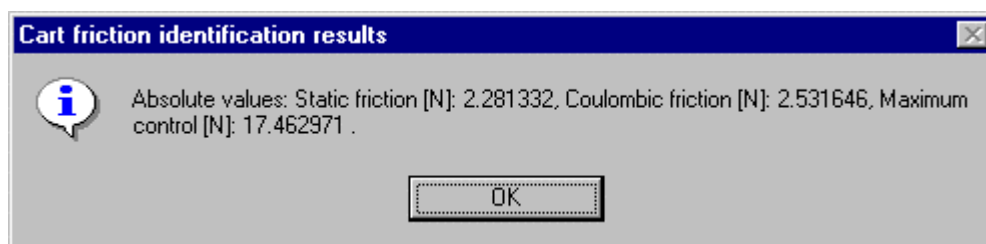


Figure 3-29: Cart friction - identification results

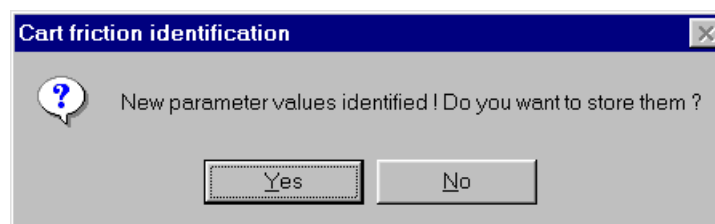


Figure 3-30 End of cart friction identification

3.5 DEAD ZONES IDENTIFICATION DZCV, DZPV

Matlab program p_iddz.m

This step of the identification procedure detects the minimum value of cart velocity such that below it the cart movement is not observed, and the minimum value of the pendulum



angular velocity such that below it the influence of the pendulum on the cart movement is not observed.

Before you run this test you should switch off the DC motor power and take off the cart blocks if the previous step was the identification of pendulum friction or to add the pendulum arms on the cart if the previous experiment was cart friction identification. To run this part of the identification click on the **Dead zone** experiment button in the main identification window. To start the algorithm you must accept the following information (Figure 3-31).

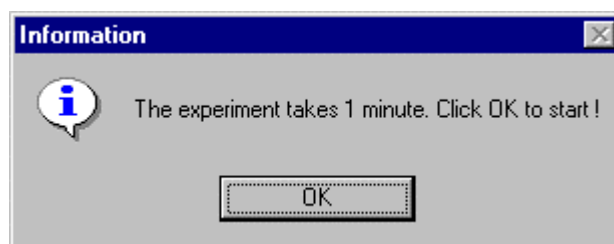


Figure 3-31: Algorithm starts

Next you should place the cart in the centre of the rail and accept the following message (Figure 3-32).

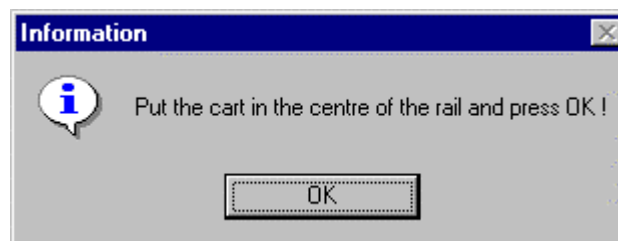


Figure 3-32: Placing the cart in the centre

After that you must raise the pendulum to a horizontal position (angle $\pm \pi/2$) and release it at the same time when you accept the following message box (Figure 3-33).

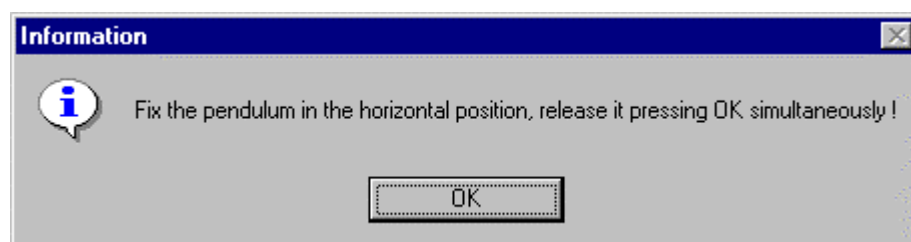


Figure 3-33: Setting pendulum angle



CHAPTER 3

IDENTIFICATION

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

If the value of pendulum angle deviation is too small, hence no motion of the cart, you see the error message (Figure 3-344) and you should repeat this step and raise the pendulum higher.

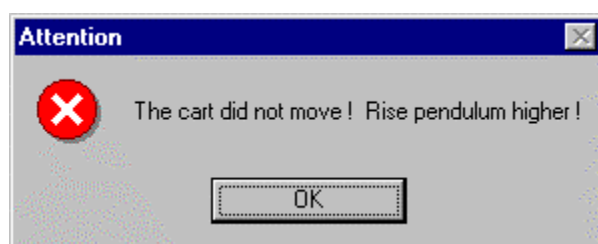


Figure 3-34: Cart not in motion

During collecting data you will be informed about it by the following progress bar window:



Figure 3-35 Dead zone experiment in progress

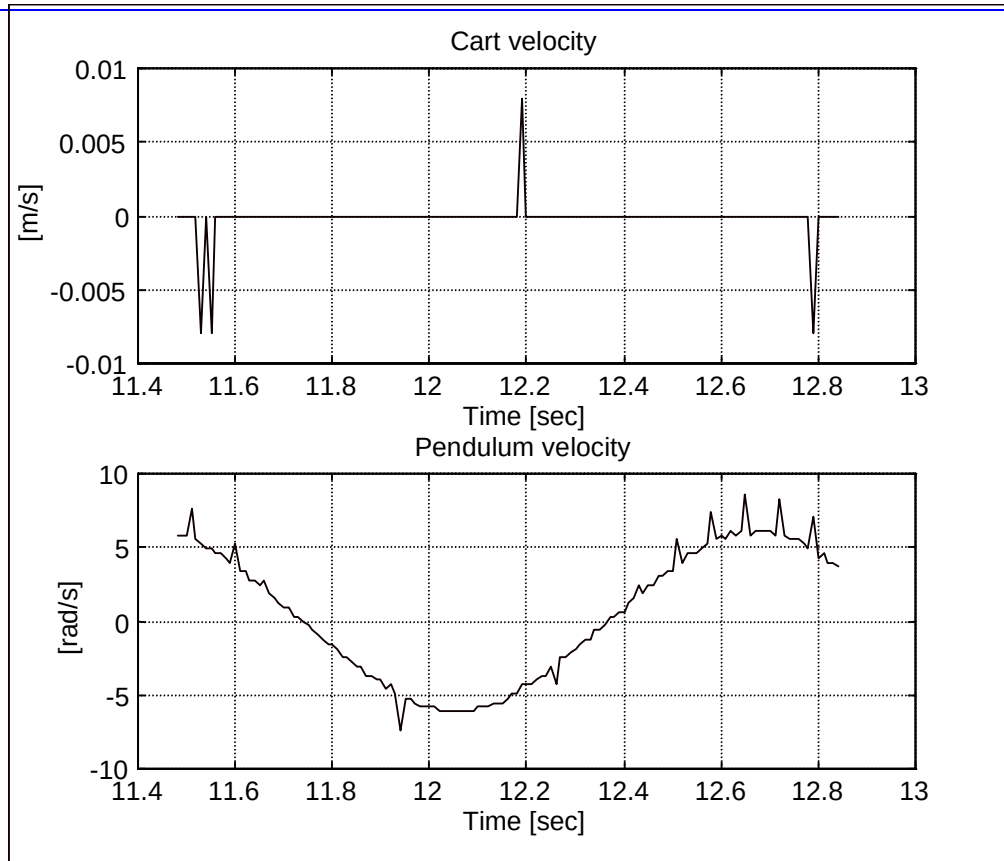


Figure 3-36: Dead zones of cart and pendulum velocities

DZcv and DZpv are the minimum values of cart and pendulum velocities responses. Output of dead zones identification experiment is shown in Figure 3-36 and Figure 3-37.



Figure 3-37 Dead zone experiment results

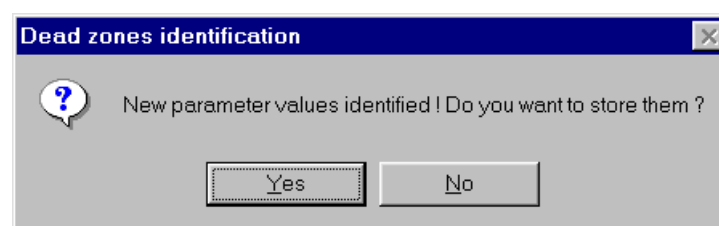


Figure 3-38 End of Dead zone experiment



3.6 IMPULSE EXCITATION

Matlab program p_idie.m

This step of identification shows the pendulum-cart system response due to a short impulse. This step should be performed with the pendulum on the cart. Before you run this step you should click on the **Stop practical** button in the main identification window and next switch on the DC motor power.

To run this part of the identification procedure click on the **Impulse excitation** button in the main identification window. The cart can be positioned at an arbitrary place on the rail, but the pendulum should be in the down position and no motion of it should be observed. The following dialogue box appears (Figure 3-39)

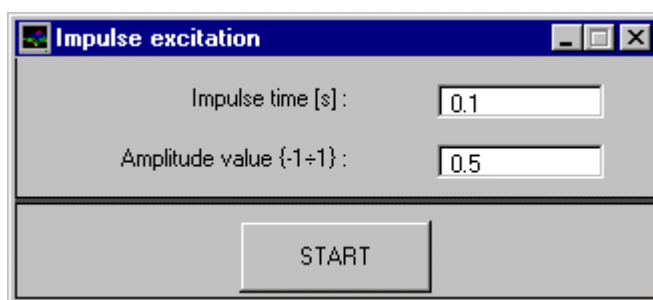


Figure 3-39: Set impulse parameters

In this dialog box you can define the time and amplitude of the control impulse. The amplitude value must be from the range $[|U_{min}|, 1]$, where U_{min} is the minimum value of control needed to move the cart. The time should be less than 1 second. As default parameters the procedure sets : value = 0.2 and time = 0.1;

Next you will be asked about motor supply. You will see the following window (Figure 3-40)

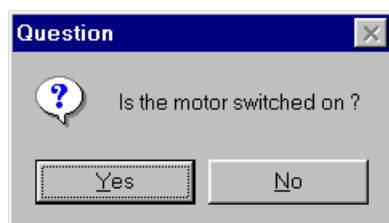


Figure 3-40

If you choose the **No** button you must turn motor on and accept the **OK** button in the following window (Figure 3-41) and you should execute the following instruction.

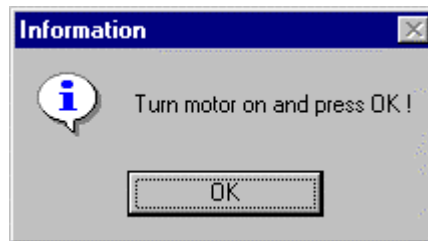


Figure 3-41

You will see the following window with progress bar:



Figure 3-42

Figure 3-42 shows the results of impulse excitation for amplitude 0.2 and time 0.3 second.

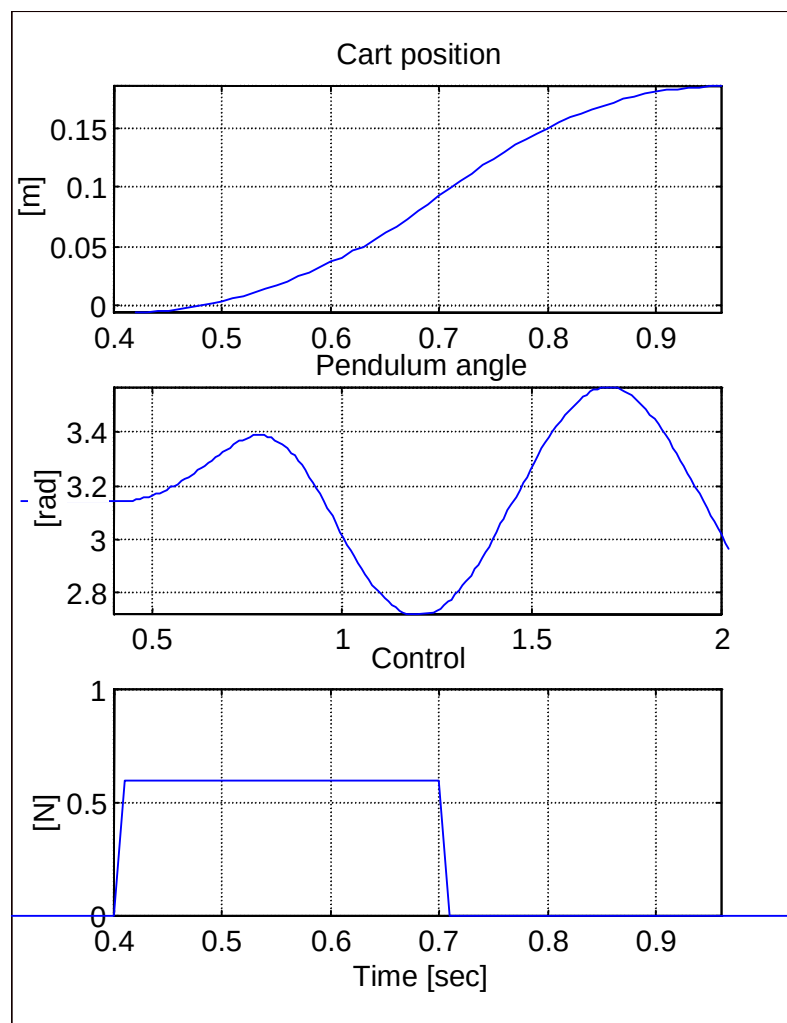


Figure 3-42: Impulse excitation results



CHAPTER 3

IDENTIFICATION

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

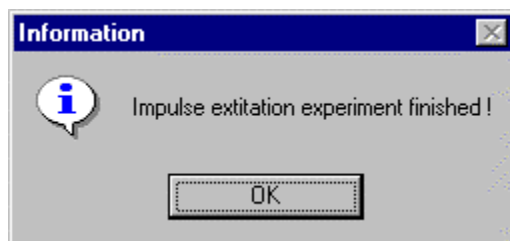


Figure 3-43 End of impulse excitation experiment



3.7 RESULTS OF PENDULUM IDENTIFICATION PROCESS.

Click on the **RESULTS** button in the main identification window to see all pendulum parameters (Figure 3-44). You will be asked about saving all pendulum parameters to MATLAB *pendpar.m* file, which you can call from MATLAB Workspace to load all parameters automatically.

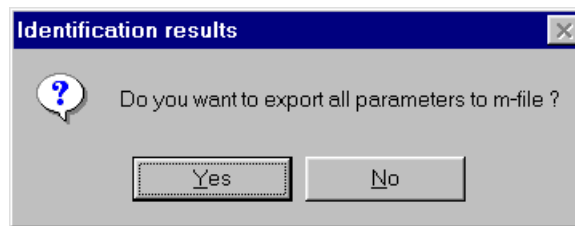


Figure 3-44: Export all parameters

If you click on the **YES** button, all pendulum parameters will be saved in to the *pendpar.m* file. Figure 3-45 shows some actual results from the identification experiments.



CHAPTER 3

IDENTIFICATION

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

Pendulum identification				
PARAMETER DESCRIPTION	SYMBOL	VALUE	ACTUALISED	
Cart mass [kg] :	mc	1.12	1997-09-15	13:27
Cart mass without positioners [kg] :	mc_	1.045	1997-09-15	13:27
Pendulum weight mass [kg] :	mpw	0.095	1997-09-15	13:27
Pendulum stick mass [kg] :	mps	0.025	1997-09-15	13:27
Rail length [m] :	RI	1	1997-09-15	13:27
Pendulum stick length [m] :	lp	0.402	1997-09-15	13:27
Distance lpo [m] :	lpo	0.146	1997-09-15	13:27
Length of load[m] :	lc	0.041	1997-09-15	13:27
Distance lco [m] :	lco	0.347	1997-09-15	13:27
Radius of the load [m] :	rc	0.02	1997-09-15	13:27
Radius of the stick [m] :	rp	0.006	1997-09-15	13:27
Theoretic pend. moment of inertia [kg*m2]:	Jpt	0.0123321	1997-09-15	13:27
Static friction [N] :	FS	2.28133	1997-09-15	12:18
Coulombic friction [N] :	FC	2.53165	1997-09-15	12:18
Maximum control [N] :	M	17.463	1997-09-15	12:18
Minimum control [N] :	DZu	1.37918	1997-09-14	17:22
Minimum cart velocity [m/s] :	DZcv	-0.00793711	1997-09-14	16:33
Minimum pend. velocity [rad/s] :	DZpv	3.37476	1997-09-14	16:33
Pendulum friction [kg*m2/s] :	fp	0.000107443	1997-09-14	16:28
Pendulum period [s] :	T	1.16	1997-09-14	16:28
Pendulum moment of inertia [s] :	Jp	0.0139231	1997-09-14	16:28
Moment of inertia [kg*m2] :	J	0.0135735	1997-09-14	16:28
Distance: axis of rotation-mass centre [m] :	I	0.0167903	1997-09-14	16:28
Close				

Figure 3-45 All system parameters



Summary of all parameters

Name	Description	Unit
m_c	cart mass	[kg]
m_{c-}	cart mass without positioners for pendulum	[kg]
m_{ps}	pole mass	[kg]
m_{pw}	load mass	[kg]
R_l	rail length	[m]
l_p	length of pole	[m]
l_{po}	distance between centre of pole mass and rotation axis	[m]
l_c	length of load	[m]
l_{co}	distance between centre of load mass and rotation axis	[m]
r_c	radius of the load	[m]
r_p	radius of the stick	[m]
J_{pt}	Theoretic pendulum moment of inertia	[kg·m ²]
FS	cart static friction	[N]
FC	cart dynamic friction coefficient	[kg/s]
M	maximal force	[N]
DZ_u	minimal force to move a cart	[N]
DZ_{cv}	dead zone of cart velocity	[m/s]
DZ_{pv}	dead zone of pendulum velocity	[rad/s]
f_p	pendulum friction constant	[kg·m ² /s]
T	pendulum period	[s]
J_p	moment of inertia related to the axis of rotation	[kg·m ²]
J	moment of inertia related to the mass centre	[kg·m ²]
l	distance between axis of pendulum rotation and the centre of pendulum-cart mass	[m]

Table 1 Names of all parameters

All parameters of the pendulum-cart system are listed in Table 1. If you call the file *pendpar.m* from MATLAB command window than all variables will be loaded to workspace.



CHAPTER 3

IDENTIFICATION

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

3.8 IDENTIFIED PARAMETERS.

Examples of parameter values, which were measured and identified during an experiment, are grouped in Table 2.

Parameter name	Value	Unit
m_c	1.120	[kg]
m_{c-}	1.045	[kg]
m_{ps}	0.025	[kg]
m_{pw}	0.095	[kg]
R_l	1.000	[m]
l_p	0.402	[m]
l_{po}	0.146	[m]
l_c	0.041	[m]
l_{co}	0.347	[m]
r_c	0.02	[m]
r_p	0.006	[m]
J_{pt}	0.0136	[kg·m ²]
FS	3.0	[N]
FC	2.0	[kg/s]
M	15.500	[N]
DZu	1.0	[N]
$DZcv$	0.080	[m/s]
$DZpv$	6.800	[rad/s]
f_p	2.1e-5	[kg·m ² /s]
T	1.160	[s]
J_p	0.013922	[kg·m ²]
J	0.0135966	[kg·m ²]
l	0.0167066	[m]

Table 2 Values of all parameters



3.9 VERIFICATION AND TUNING.

There is a group of parameters such as m_c , m_{c-} , m_p , R_l and L . which are measured. There are also parameters which are calculated on the basis of experiments and measurements such as J , I . Other parameters of the system may only be identified by experiments.

Some of these parameters can be successfully identified such as FS , $Dzcv$, $Dzpv$, f_p and T . For example, the pendulum period T may be obtained with great accuracy, whereas the dynamic or Coulomb friction FC , the dead zone of control DZu and the control magnitude M are poorly identified. Due to the random character of these parameters we can only assume the average values of detected parameters in a number of experiments.

In this situation we have to verify the results obtained with the use of other independent identification experiments. These experiments are performed both on the simulation model and real system configured in the same way.

The architecture of the simulated and real experiments is shown in Figure 3-. The following experiments are performed:

- impulse excitation
- closed-loop proportional control with a square wave excitation.

We can apply the pulse generator (Figure 3-47) or the simple proportional controller as shown in Figure 3-48 (use only the proportional cart controller).

The parameters of the simulation model are set as they have been detected previously. The values of poorly identified parameters: FS , FC , M are treated as the starting point in experiments to optimise and find the best parameter values.

A performance index of the optimisation procedure is calculated as the squared difference of the real and simulation transient responses.

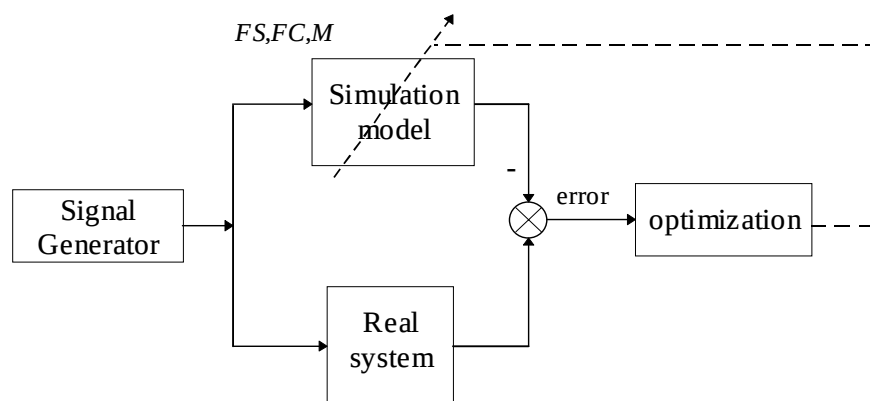
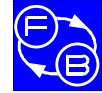


Figure 3-46: Tuning of parameters



CHAPTER 3

IDENTIFICATION

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

One can modify the friction function by tuning the parameters defined in the friction mask: FS , FC , Xc , Yc . The results of tuning performed in the **simulation model** - file ***model_ie.m*** shown in (Figure 3-47) are verified with the results taken from the real system. The simulated system responses are given in Figure 3-48 and Figure 3-49.

The **real data** is obtained after clicking on the **Impulse excitation** button in the main *pendulum identification* window. See Figure 3-42. Note the similarity of the depicted cart position time diagrams in both systems.

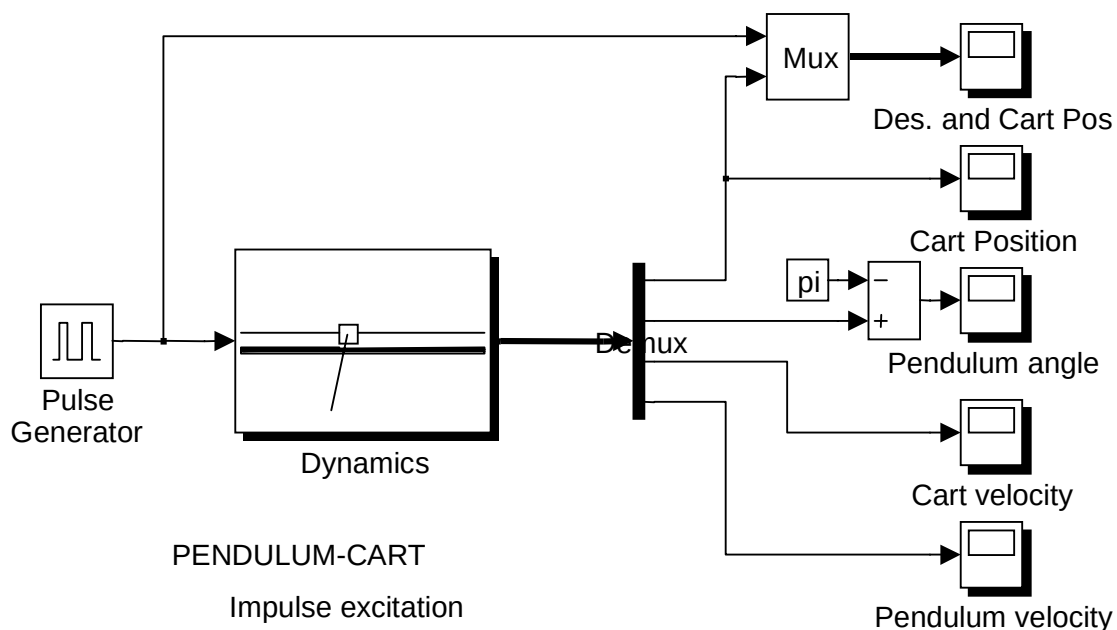


Figure 3-47: Pulse generator *model_ie.m*

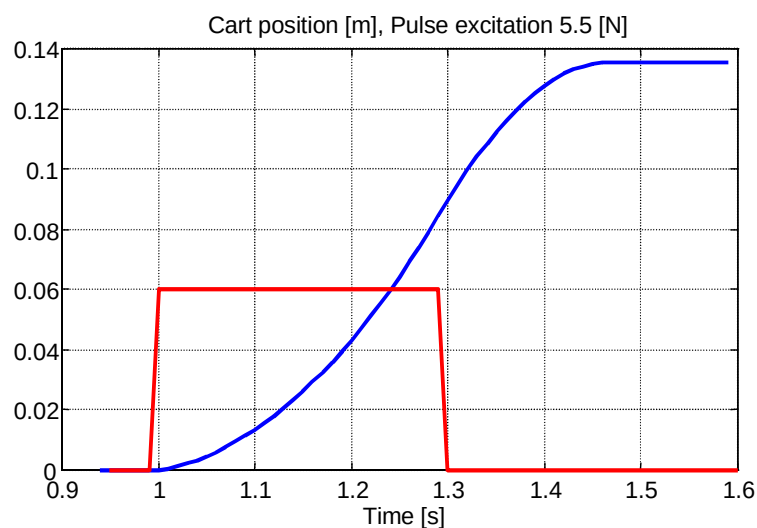


Figure 3-48: Simulated Cart position

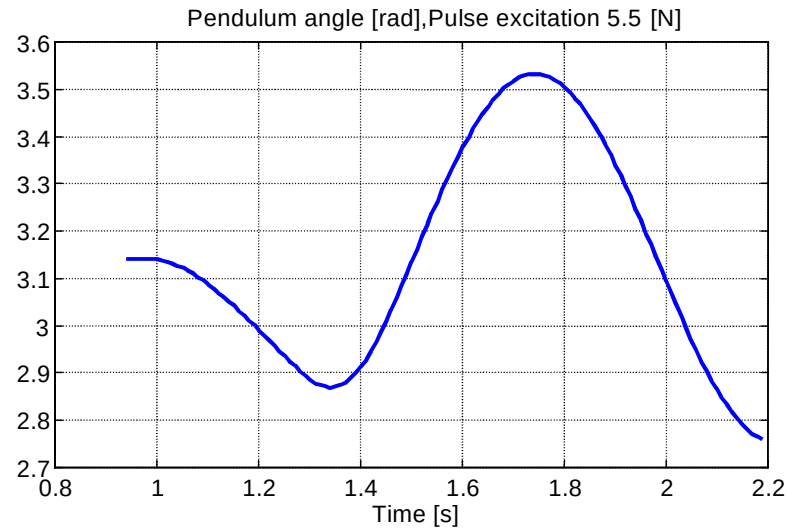


Figure 3-49: Simulated Pendulum angle

In the above case the tuning is done by trial-error method, comparing the real with simulation impulse excitation response.

The second experiment is concerned with a **proportional cart controller** and a square wave excitation. The results of tuning the simulation model shown in Figure 3-50 are verified with the results taken from the **real system** presented in Figure 3-51.

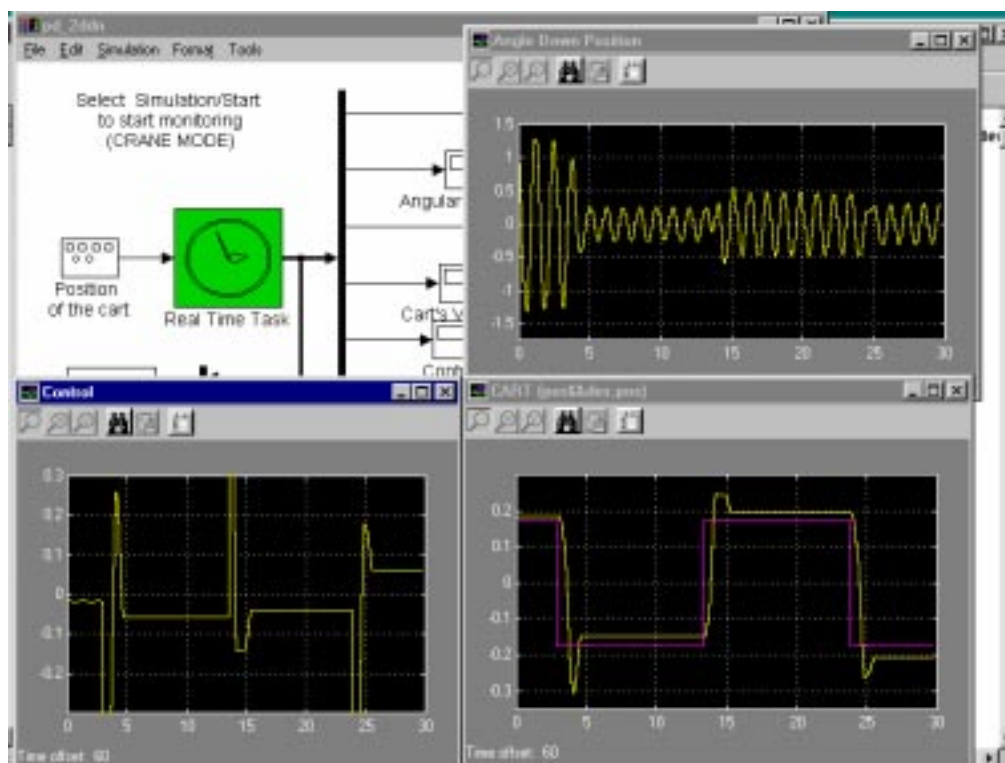
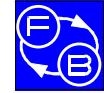


Figure 3-51: Crane mode Proportional cart controller, real system response



CHAPTER 3

IDENTIFICATION

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

To use proportional cart control, access **Crane PID Control** block in the main control window. The following parameters were set: Proportional cart controller with K_p gain equal to 2 (all other PID control parameters set to zero), square wave of frequency equal to 0.2 [rad/s] and peak equal to 0.17, the magnitude of the control is bounded at $|0.8|$.

Notice the similarity of the depicted cart position time diagrams in both systems. The modified parameters obtained in the way described above are shown in Table 3 and are used in the remainder of the manual..

A comparison of experimental data given in Table 2 and the simulated model data in Table 3, shows that the tuning of the poorly identified parameters is necessary.

The **simulation model** for Proportional cart control only can be accessed by typing **modelpid** at the Matlab prompt.

The identified simulation model (Figure 3-50) will be used in the next sections as the model for the PID control. Finally let us note that the identification of the pendulum-cart system can be performed also in a different way including more statistical data processed by the least squares or other statistical method. In this case a greater number of experiments would be required.

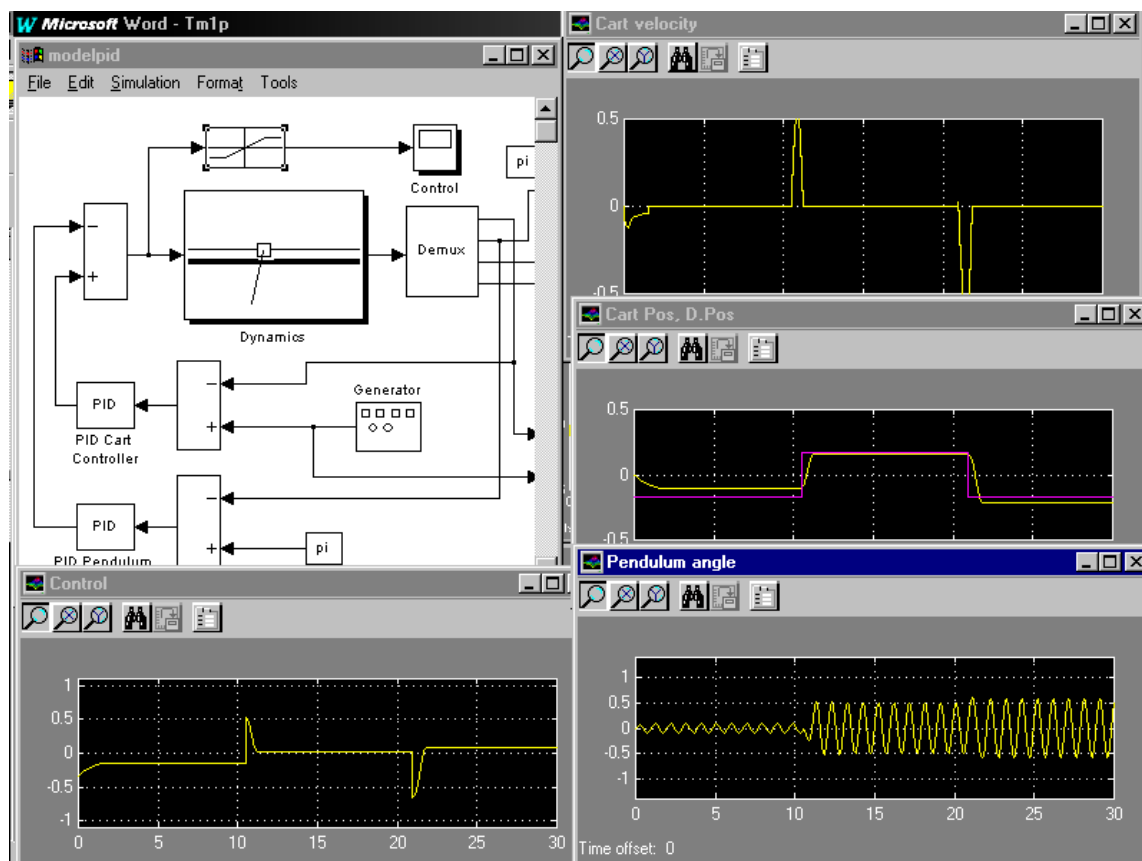
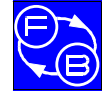


Figure 3-50: Crane mode Proportional cart controller, Simulink model response



Name	Value	Unit
m_c	1.120	[kg]
m_{c-}	1.045	[kg]
m_{pw}	0.095	[kg]
m_{ps}	0.025	[kg]
R_l	1.000	[m]
l_p	0.402	[m]
l_{po}	0.146	[m]
l_c	0.041	[m]
l_{co}	0.347	[m]
FS	3.2	[N]
FC	2.3	[kg/s]
M	11.300	[N]
DZu	1.5	[N]
$DZcv$	0.080	[m/s]
$DZpv$	6.800	[rad/s]
f_p	2.1e-5	[kg·m ² /s]
T	1.160	[s]
J_p	0.012322	[kg·m ²]
J	0.011966	[kg·m ²]
I	0.017066	[m]

Table 3 Values of all measured and identified parameters



3.10 OPTIMISATION OF J_p AND F_p

To start optimisation of two parameters of the pendulum model, J_p and f_p , (the pendulum moment of inertia and the friction of the pendulum) click on the ***Jp & fp Optimisation*** button in *pendulum identification* menu. The window in Figure 3-51 appears. The relations between: a , b , J_p and f_p parameters are shown by the formulae given below:

$$a = \frac{mgr}{J_p} \quad \text{and} \quad b = \frac{f_p}{J_p}$$

or

$$J_p = \frac{mgr}{a} \quad \text{and} \quad f_p = J_p \cdot b$$

The cost function is given by the formula:

$$Q = \langle (y - y_m), (y - y_m) \rangle$$

where:

y - vector approximating the trajectory of pendulum angle. The exact relation between angle and time is given by the equation $\ddot{y} = t_1 \cdot \sin(y) - t_2 \cdot \dot{y}$, where t_2 , t_1 are successive time moments and $t_2 = t_1 + \Delta t$, (Δt is the time step),

y_m - vector of measured trajectory of pendulum angle,

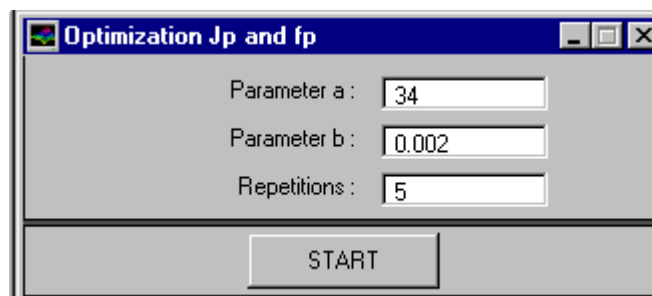


Figure 3-51: Optimisation starting values



Define a number of steps. Enter “a” and “b” values according to your choice. Type the starting values into the “Parameter a” and “Parameter b” edit windows. Type into the Repetitions edit window an integer value which defines the number of calls to the internal optimisation procedure. During the run of the internal optimisation procedure the following progress bar shows the actual state of data transfer (Figure 3-52).

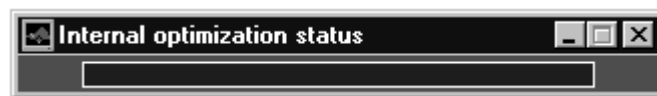


Figure 3-52: Internal optimisation status

The optimisation procedure has a long execution time. Therefore two windows appear and are updated at each iteration to show that the optimisation is in progress.

The first window (Figure 3-53) shows the current status of optimisation procedure. It contains: Parameter a, Parameter b, Cost function, Optimisation progress (%)

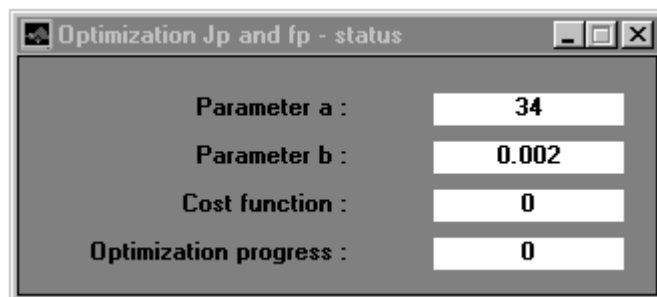


Figure 3-53: Parameters values and total optimisation status

The second window (Figure 3-54) shows two curves:

- current data points to be optimised and,
- the results of optimisation.

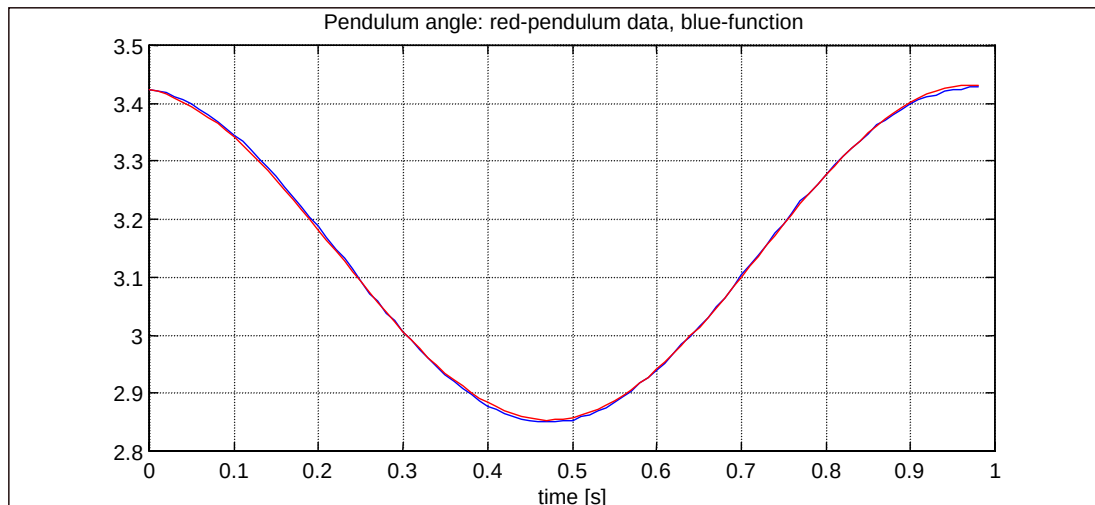


Figure 3-54: Pendulum angle

After optimisation procedure the following window appears (Figure 3-55).

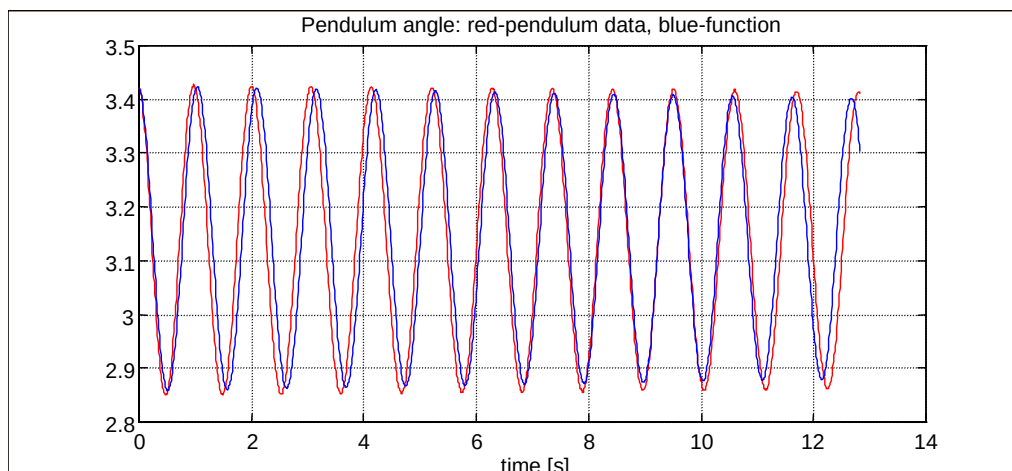


Figure 3-55: Optimisation results

Before clicking on the *OK* button examine values of the optimised parameters. When you click on the *OK* button the status window and status bar disappear. Results are simultaneously displayed in MATLAB Command window as *a*, *b*, and *Q*. The parameters *a* and *b* are identified with the least squares method. The method is applied in an iterative way, with the time interval *T_n* increasing in successive iterations. The value of *T_n* is increased by 6 oscillation periods in each of iterations 1, 2, 3, and 4. In further iterations the increment of *T_n* is equal to 15 oscillation periods. The iterations are repeated until the full observation time has been covered. If the Cost function value in the current step is greater than the previous value you see the message (Figure 3-56).

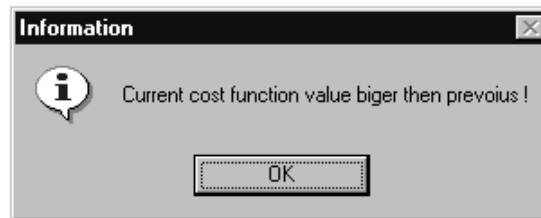


Figure 3-56: Cost function overshoot

If the number of repetitions is too big and the optimisation procedure has no more data to operate on, you see the following message (Figure 3-57).



Figure 3-57: No more data



CHAPTER 3
IDENTIFICATION

DIGITAL PENDULUM SYSTEM
TEACHING MANUAL 1

Notes



4 PID CONTROLLERS.

The PID control rule is very common in control systems. It is the basic tool for solving most process control problems. The PID controllers are usually standard building blocks for industrial automation. An industrial PID control algorithm is equipped with several modifications. For example, the proportional part acts only on a fraction of the desired value, the integral action is modified with "anti-windup" operation which stops integrating when the control saturates, and the derivative action is reduced at high frequencies.

There is a number of tuning methods for PID controllers. Some of them are based on transient response experiments e.g., the Ziegler-Nichols step response tuning rule, other are based on relay feedback when the parameters of a PID controller are determined from features of the limit cycle of the closed-loop system, some other are based on frequency analysis.

We shall consider the pendulum-cart control system containing two PID controllers. The first controls the angle of the pendulum. The second is used to control the position of the cart. The outputs of the PID controllers are added to produce the final control value for the D/A converter. The PID double control system of the pendulum-cart system is shown in Figure 4-1.

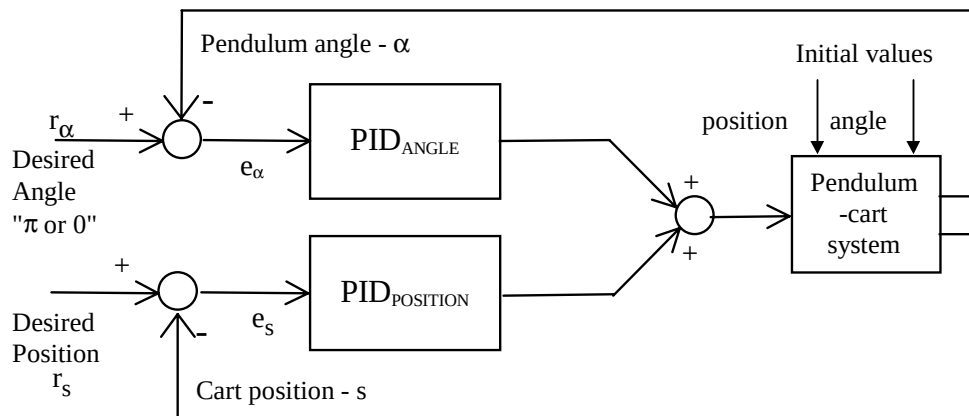
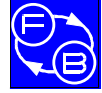


Figure 4-1: Block diagram of PID double control system

The double controller is described by the following equations:



CHAPTER 4

PID CONTROLLERS

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

The proportional parts:

$$e_{\alpha} = r_{\alpha} - \alpha,$$

$$e_s = r_s - s,$$

where:

e_{α}, e_s - errors of angle and position,

r_{α}, r_s - reference values of angle and position,

α, s - current values of angle and position.



The integrators:

$$I_{\text{ANGLE}}(t) = \begin{cases} I_{\text{ANGLE_SAT}}, & I_A(t) > I_{\text{ANGLE_SAT}} \\ I_A(t), & -I_{\text{ANGLE_SAT}} \leq I_A(t) \leq I_{\text{ANGLE_SAT}} \\ -I_{\text{ANGLE_SAT}}, & I_A(t) < -I_{\text{ANGLE_SAT}} \end{cases},$$

$$I_A(t) = K_{I_ANGLE} \int_0^t e_\alpha(\tau) d\tau$$

$$I_{\text{POSITION}}(t) = \begin{cases} I_{\text{POSITION_SAT}}, & I_P(t) > I_{\text{POSITION_SAT}} \\ I_P(t), & -I_{\text{POSITION_SAT}} \leq I_P(t) \leq I_{\text{POSITION_SAT}} \\ -I_{\text{POSITION_SAT}}, & I_P(t) < -I_{\text{POSITION_SAT}} \end{cases},$$

$$I_P(t) = K_{I_POSITION} \int_0^t e_s(\tau) d\tau$$

where:

$K_{I_ANGLE}, K_{I_POSITION}$ are gains of the integration parts,

$I_{\text{ANGLE_SAT}}, I_{\text{POSITION_SAT}}$ are saturations of the integrators.

The outputs from the $\text{PID}_{\text{ANGLE}}$ and $\text{PID}_{\text{POSITION}}$ are equal to:

$$U_{\text{ANGLE}}(t) = K_{P_ANGLE} e_\alpha(t) + I_{\text{ANGLE}}(t) + K_{D_ANGLE} \frac{de_\alpha(t)}{dt},$$

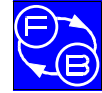
$$U_{\text{POSITION}}(t) = K_{P_POSITION} e_s(t) + I_{\text{POSITION}}(t) + K_{D_POSITION} \frac{de_s(t)}{dt},$$

$K_{P_ANGLE}, K_{D_ANGLE}, K_{P_POSITION}, K_{D_POSITION}$

are gains of the proportional and differentiation parts of the controllers, respectively.

Finally, the control for the DC drive $U(t)$ is given by:

$$U(t) = \begin{cases} U_{\text{MAX}}, & U_{AP}(t) > U_{\text{MAX}} \\ U_{AP}(t), & -U_{\text{MAX}} \leq U_{AP}(t) \leq U_{\text{MAX}} \\ -U_{\text{MAX}}, & U_{AP}(t) < -U_{\text{MAX}} \end{cases},$$



CHAPTER 4

PID CONTROLLERS

DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

$$U_{AP}(t) = U_{ANGLE}(t) + U_{POSITION}(t)$$

where:

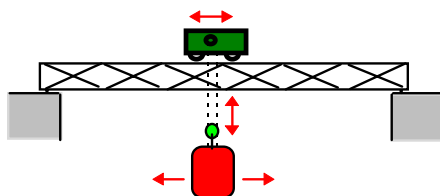
U_{MAX} is the maximum absolute value of control.

The controller uses the error between the desired value and the actual value for the position of the cart and the angle of the pole. If the desired value of the angle is equal to zero the controller stabilises the pole in the upright position. If the desired value is equal to $\pm\pi$ the cart-pole system works as a crane.

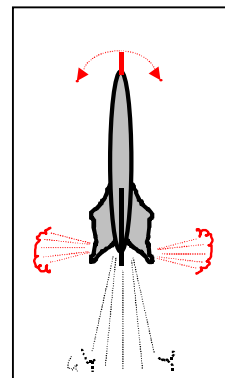
Note that only these values for the pole's desired angle are admissible: 0 and $\pm\pi$.

4.1 SIMULINK MODELS FOR PID DOUBLE CONTROLLER

There are two SIMULINK models available - one for stabilising the pole in the down position – the crane problem (C) and another for stabilising the pole in the upright position - self erecting problem (SE) (Figure 4-1).



crane control (C)



self erecting control (SE)

Figure 4-1: Two linear control problems



A human or a computer may operate as a controller. A human operator may be fairly successful in the **C** problem, but completely fails in the **SE** problem which requires a fast operating controller. It is obvious that **C** and **SE** modes require different sets of controller parameters. A few different parameter sets have been prepared for both control modes and they are available in SIMULINK models.

The names of SIMULINK models for **C** and **SE** modes are *pd_2ddn.m* and *pd_2dup.m*, respectively. These files are stored on the installation diskette.

4.2 CRANE MODE (2 PID for Pendulum Angle And Cart Position)

The SIMULINK model for monitoring the real-time experiments operating in **C** mode is shown in Figure 4-1.

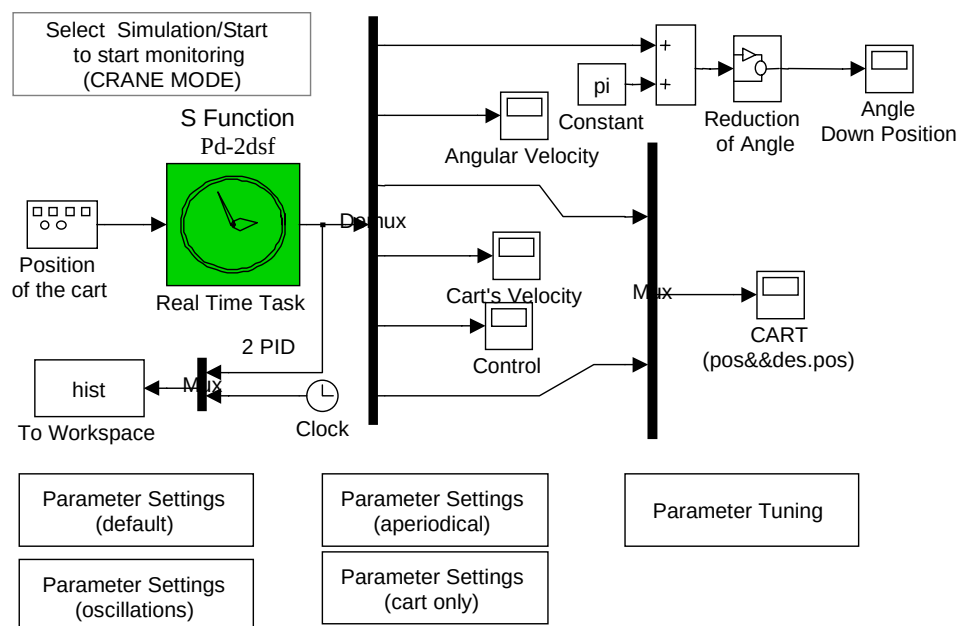


Figure 4-1: External interface of real system for C mode, invoked by the *pd_2ddn* command

It can be run by typing *pd_2ddn* in the MATLAB Command Window. The main goal of this model is not to simulate the pendulum-cart system but to transfer data to and from the Real Time Kernel. This is the main difference between this model and typical SIMULINK models.



The main block of this SIMULINK model is the Real Time Task block which controls the data exchange between MATLAB and the real-time kernel. The Real Time Task is excited by the signal generator which is the source of reference position of the cart. The output of the Real Time Task block is a vector which contains information about the angle of the pendulum, position of the cart, angular velocity of the pendulum, velocity of the cart, reference position of the cart and control value of the DC drive. The values of the output vector are shown on SIMULINK scopes and are transferred to MATLAB workspace by the *To Workspace* block. In this way off-line calculations and other visualisations of the experiment results can be made.

Table 4

		Parameter s Setting (default)	Parameters Setting (aperiodical)	Parameters Setting (oscillations)	Parameters Setting (cart only)
Pend.	KpAngle	7.0	5.0	6.0	0.0
	KiAngle	0.0	0.0	0.0	0.0
	KdAngle	0.0	0.0	0.0	0.0
Cart	KpPosition	10.0	10.0	16.0	10.0
	KiPosition	2.8	1.0	2.0	1.0
	KdPosition	3.0	5.0	1.0	5.0
	IMaxAngle	0.8	0.8	0.8	0.8
	IMaxPosition	0.8	0.8	0.8	0.8
	UMax	0.8	0.8	0.8	0.8
	TSample	0.01	0.01	0.01	0.01
	RefAngle	pi	pi	pi	pi
	RefPosition	0.0	0.0	0.0	0.0

The blocks : *Parameters Setting (default)*, *Parameters Setting (aperiodical)*, *Parameters Setting (oscillations)* and *Parameters Tuning* are used to select different sets of controller parameters. Double clicking the mouse over one of the first four blocks changes the behaviour of the system when executing the SIMULINK model. The values of parameters in the sets are shown in Table 4. The *Parameter Tuning* block can be used to freely change parameter values by using mouse controlled sliders. After double clicking the window shown in Figure 4-2 opens.



Figure 4-2: Tuning parameters of the *pd_2ddn* controller.

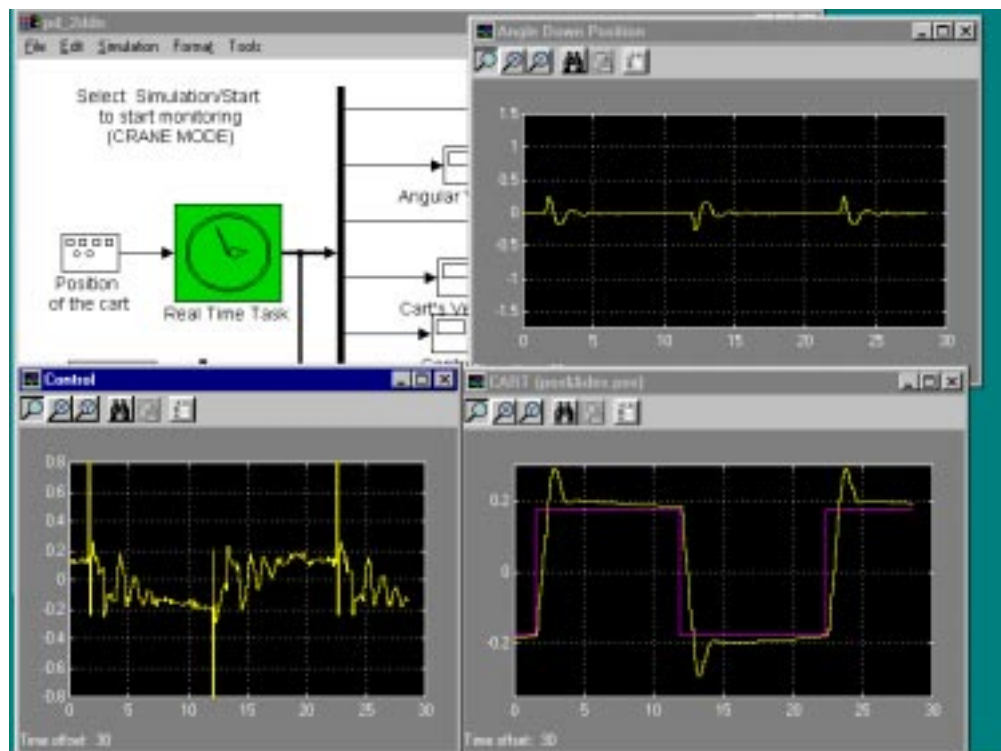


Figure 4-3: Default parameters

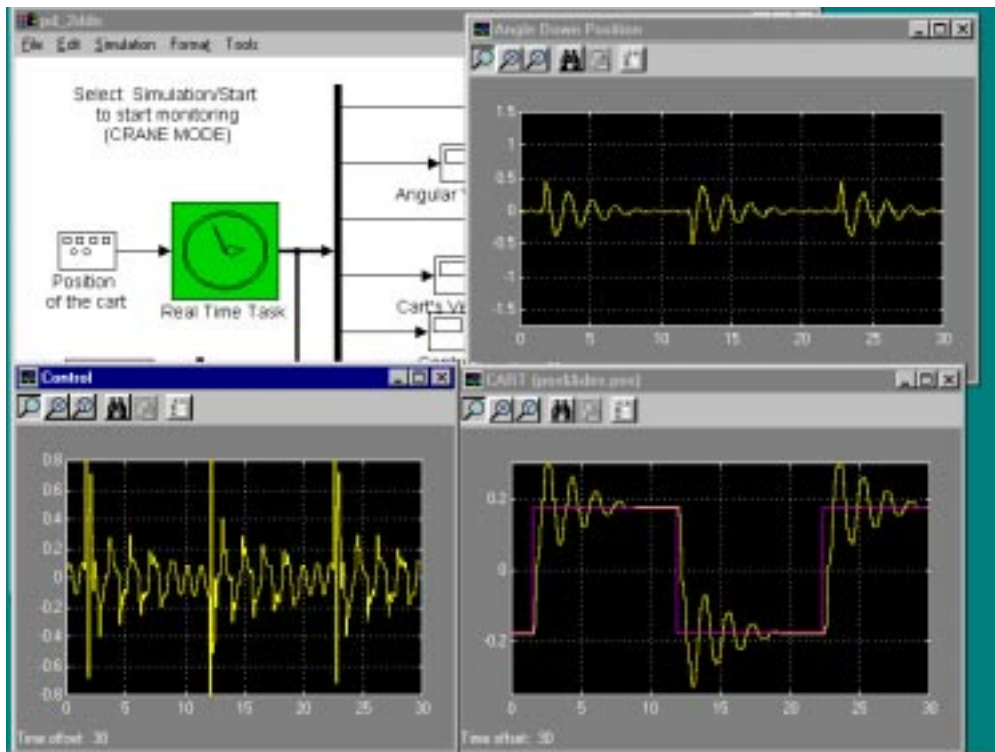


Figure 4-4: Parameters Setting (oscillations)

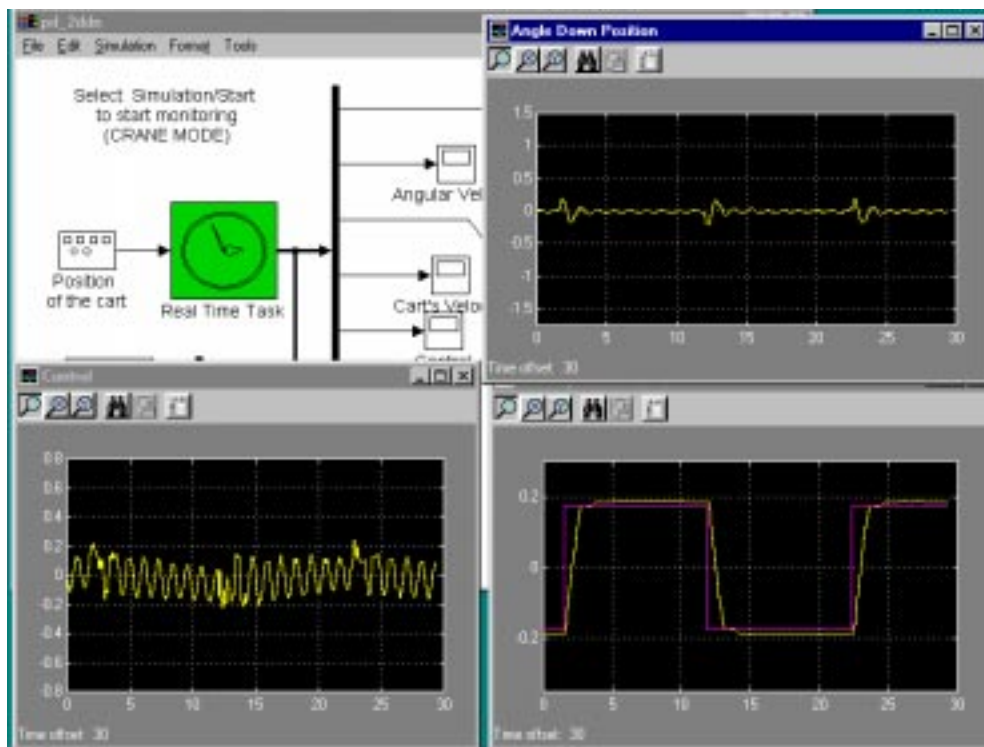


Figure 4-5: Parameters Setting (aperiodical)

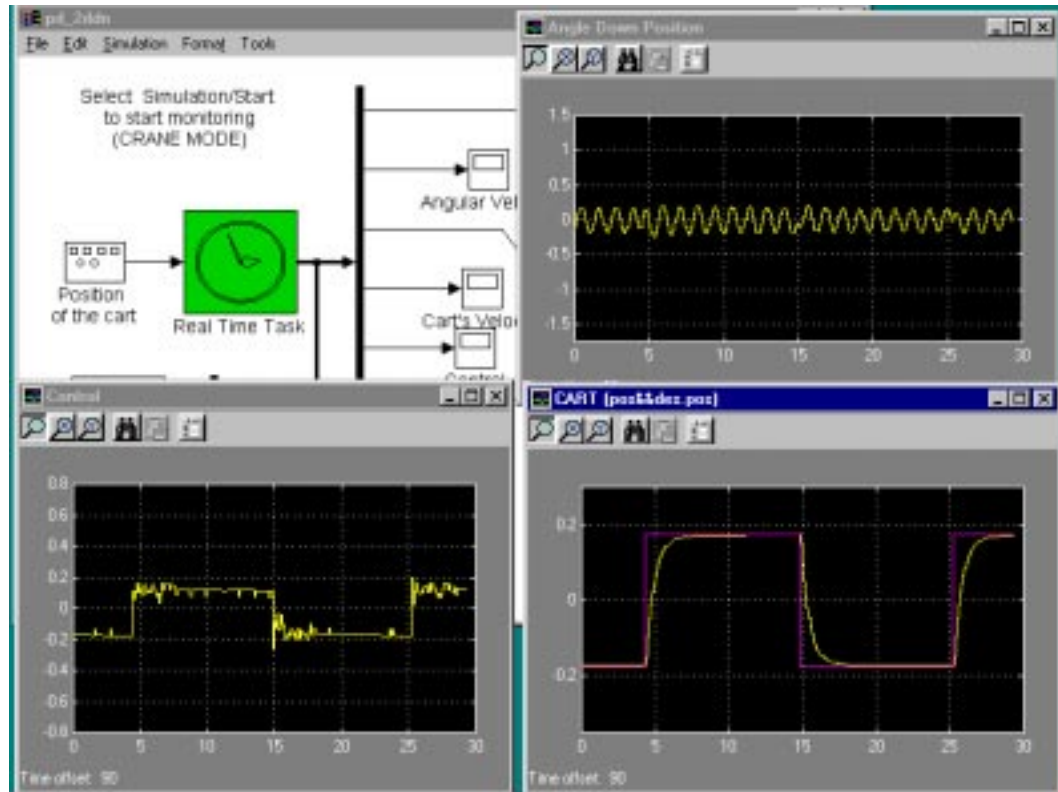


Figure 4-6: Parameters Setting (cart only)

4.3 SELF ERECTING MODE

This is activated by typing *pd-2dup* at the MATLAB prompt.

The SIMULINK model for the **SE** mode is similar to the SIMULINK model for the **C** mode and is presented in Figure 4-1.

Note that the controller cannot swing up the pole from every position to the upright position. The experiments must start with the pendulum placed in the neighbourhood of the upright equilibrium point. If the pendulum falls down during experiments, it must be raised up manually.

The main differences between the controller for the **C** mode and the controller for the **SE** mode are in the sets of parameters. The parameters for **SE** available in the SIMULINK model (blocks *Parameters setting (default)*, *Parameters setting (cart pos. stabilisation)* and *Parameters setting (cart pos. neglected)*, are presented in Table 5.



DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

CHAPTER 4

PID CONTROLLERS

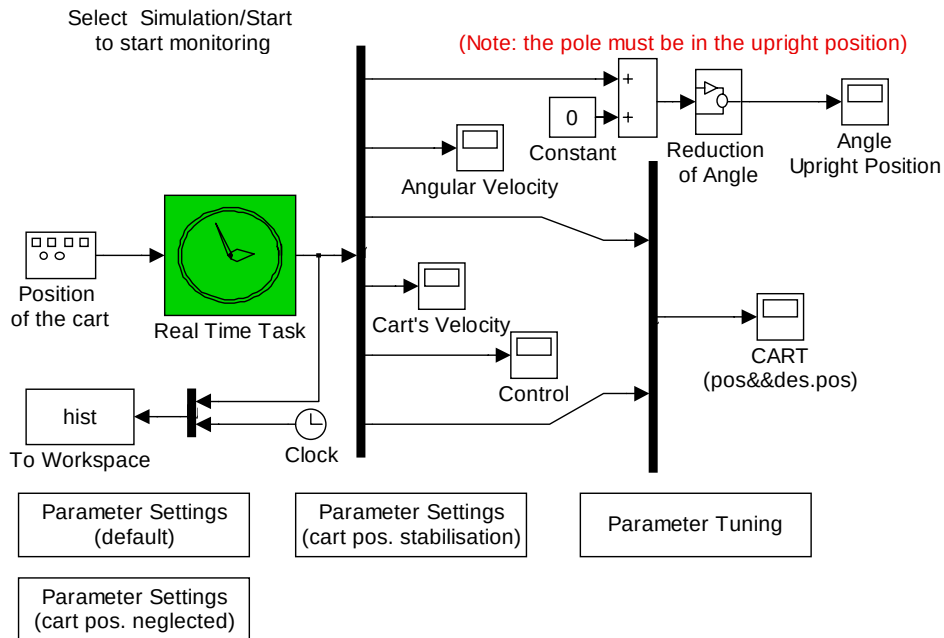


Figure 4-1: External interface of real system for SE mode, invoked by the *pd_2dup* command

Table 5

	Parameters Setting (default)	Parameters Setting (cart. pos. stabilisation)	Parameters Setting (cart pos. neglected)
KpAngle	-40.0	-8.0	-8.0
KiAngle	0.0	0.0	0.0
KdAngle	-1.0	-0.6	-0.6
KpPosition	-10.0	-3.0	-0.8
KiPosition	0.0	0.0	0.0
KdPosition	-2.0	-1.4	-1.4
IMaxAngle	0.8	0.8	0.8
IMaxPosition	0.8	0.8	0.8
UMax	0.8	0.8	0.8
TSample	0.01	0.01	0.01
RefAngle	0.0	0.0	0.0
RefPosition	0.0	0.0	0.0



The history of the experiment is transferred to the MATLAB workspace as the *hist* variable. The statements below display the history of the experiment. The parameters were changed during data acquisition, so you can notice different system behaviour at the beginning and at the end of the experiment. The results of an experiment are presented below together with the MATLAB commands used for making the plots.

```
plot( hist( :, 7 ), hist( :, 1 ) ); grid; title( 'Angle' ) - Figure 4-2
```

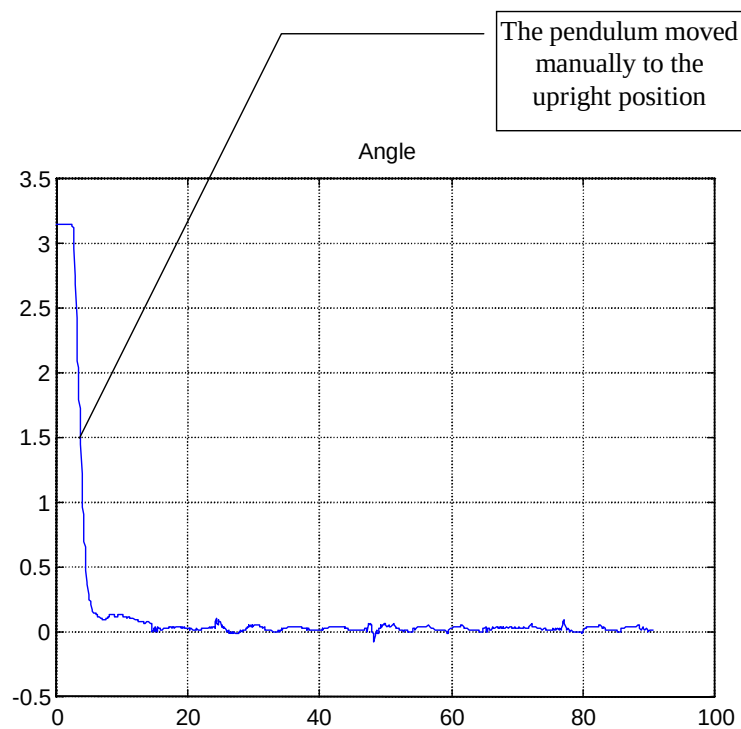


Figure 4-2



```
plot( hist( 200:900, 7 ), hist( 200:900, 1 ) ); grid; title('Angle' ) - Figure 4-3
```

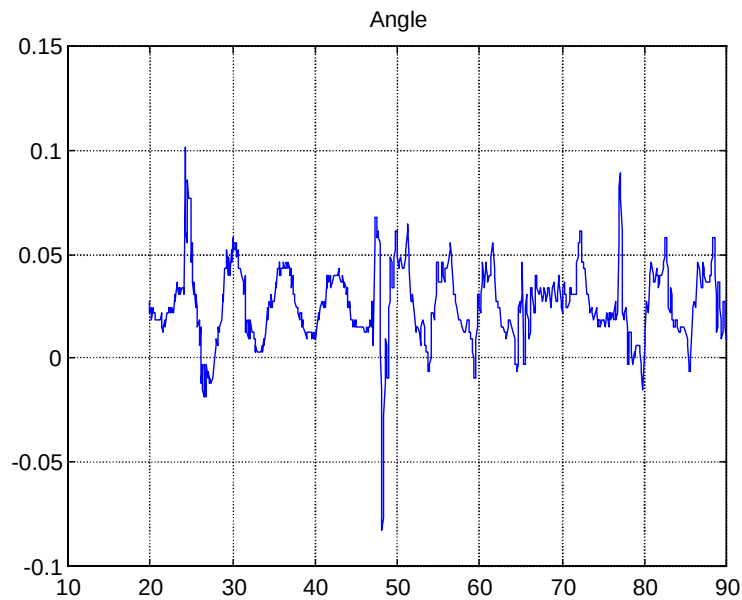


Figure 4-3

```
plot( hist( :, 7 ), hist( :, 3 ), hist( :, 7 ), hist( :, 6 ) ); grid; title( 'Position and Reference  
Position' ) - Figure 4-4
```

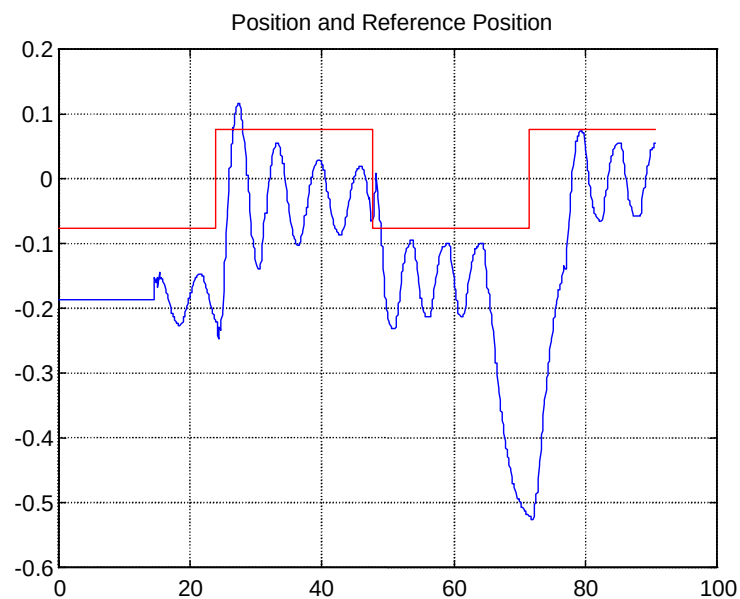


Figure 4-4



```
plot( hist( :, 7 ), hist( :, 6 ) ); grid; title( 'Control' ) - Figure 4-5
```

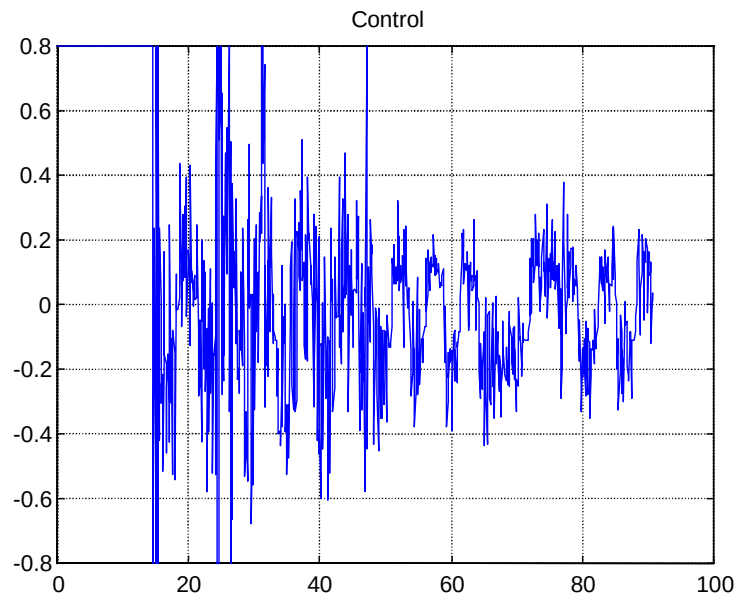


Figure 4-5



4.4 EXAMPLES OF PID CONTROLLERS, USED IN THE SIMULATION MODEL AND REAL SYSTEM.

The closed-loop **model** with PID controllers built in SIMULINK is shown below (Figure 4-1). It can be run by typing **modelpid** in MATLAB Command Window.

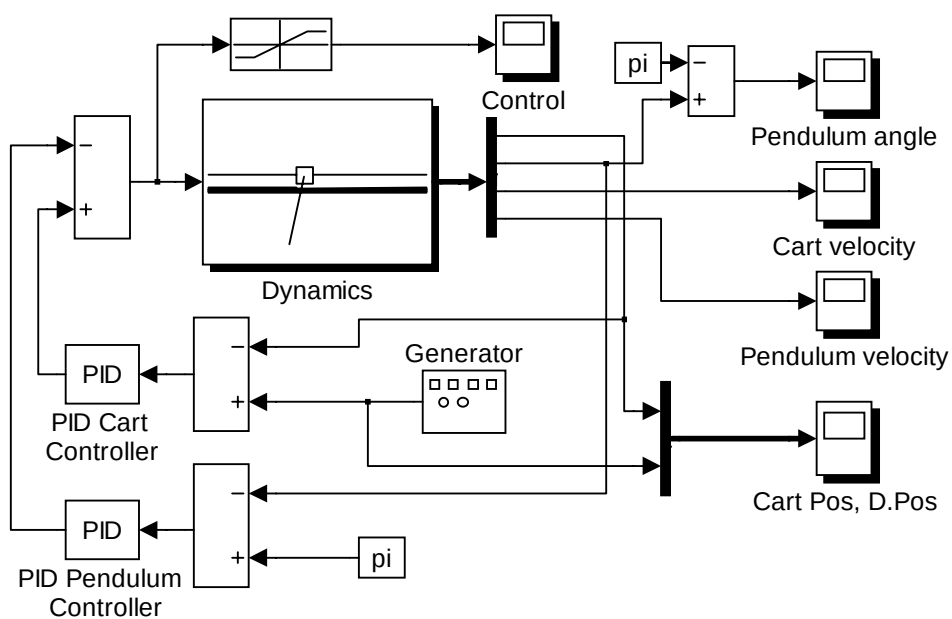


Figure 4-1: PID controllers (*modelpid.m*)

In order to get a closed representation of your real system, all the identification parameters calculated in the previous experiments need to be set up in the “Dynamics Block”.

Before starting the simulation of the PID control use the following parameter setup values.

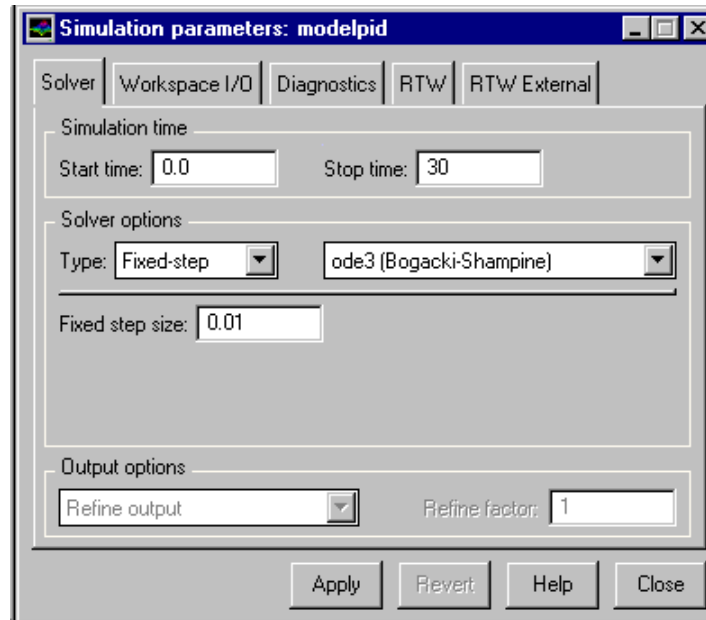


Figure 4-2: Simulation setting of *modelpid.m*

Notice the different integration method (Ode3 - Bogacki-Shampine) for the model with PID controllers (Figure 4-1).

Three examples with different PID parameters are tested in which the PID controller works in the default, aperiodic and oscillatory mode. The parameters are given in Table 6.

Table 6

	PID Cart Controller			PID Pendulum Controller		
mode	K_p	K_i	K_d	K_p	K_i	K_d
default	10	2.8	3	7	0	0
aperiodic	10	1	5	5	0	0
oscillatory	16	2	1	6	0	0

PID controller parameters for different modes



Obtaining the modes mentioned above?

There are several methods, concerned with different control goals and dynamic system behaviour, which result in appropriate parameters setting. In this manual we only wish to show three different modes of operation for the closed loop system. Let us start from defining an aperiodic mode.

The following procedure is recommended:

The *modelpid.m* file is used. We introduce variable *names* instead of constant parameter values inside the *PID* blocks. The model is additionally equipped with an *NCD* block taken from the NCD MATLAB Toolbox. This model is shown in Figure 4-3 and is called “modelncd.mdl”.

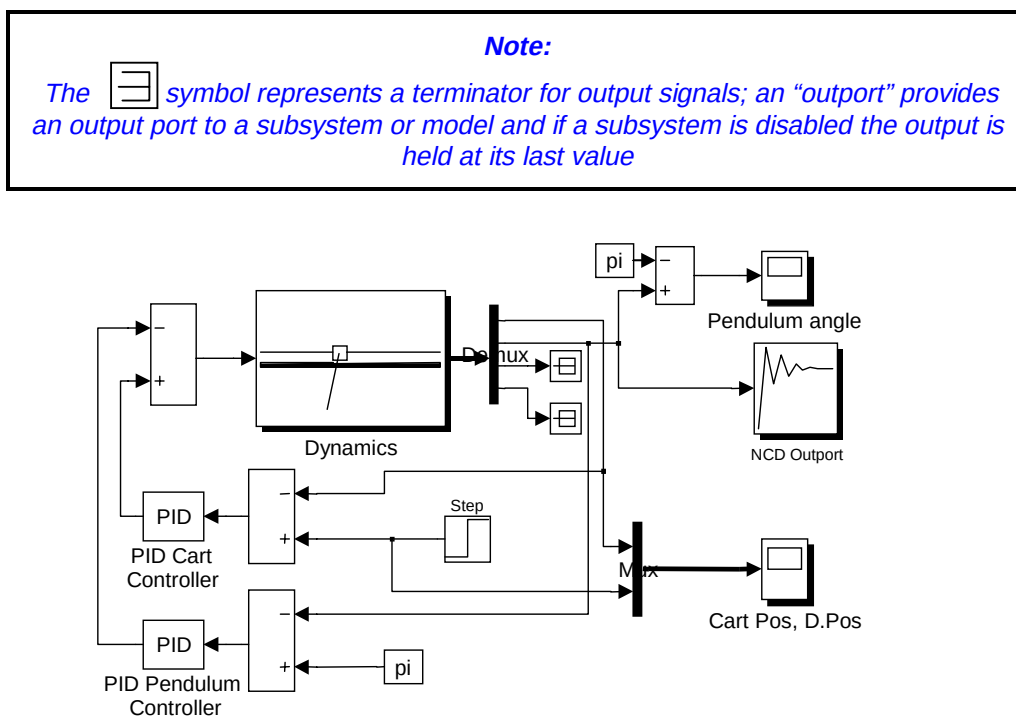


Figure 4-3: PID controllers (*modelpid.m* – [same as *modelncd.m*]) but equipped with *NCD* block

Assume a PID controller for the pendulum with the following initial values:

$$K_p = 6, \quad K_i = 0, \quad K_d = 0.$$

Assume a PID controller for the cart with the following initial values

$$K_p = 1, \quad K_i = 1, \quad K_d = 1.$$



Open the *NCD* window, assume that K_p , K_i , K_d are varying parameters, assume upper and lower boundaries of the system are a step response and click the **start** button to start the optimisation sequence.

The results are shown in Figure 4-4



Figure 4-4: *NCD* optimisation results

The following final numerical optimisation results are obtained and stored in the MATLAB workspace:

PID controller for the cart $K_p = 9.8$, $K_i = 0.98$, $K_d = 5$.

PID controller for the pendulum $K_p = 5$, $K_i = 0$, $K_d = 0$.

Next, an oscillatory mode is assumed.

Assume a PID controller for the pendulum with the following initial values:

$K_p = 6$, $K_i = 0$, $K_d = 0$.

Assume a PID controller for the cart with the following *constant* values:

$K_i = 1$, $K_d = 1$ and K_p varying from 8-17

The eigenvalues of the linearised closed model for K_p varying from 8 to 17, are calculated and plotted using the program listed below



```
lambda=[];  
Kp=7; Ki=2; Kd=1;  
for i=1:10,  
Kp=Kp+1;  
[A,B,C,D]=linmod('m_pid_2',[0 pi 0 0 0 0],0);  
lambda=[lambda eig(A)];  
[t,x,Y]=rk45('modelpid',30);  
end  
plot(lambda,'o'); - Figure 4-5
```

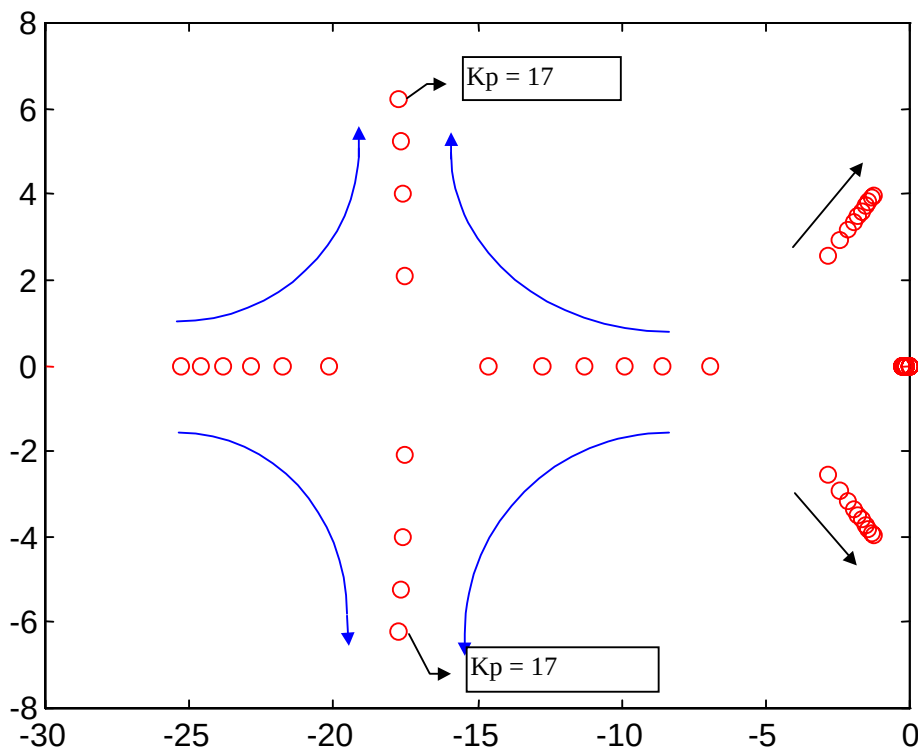


Figure 4-5: Root loci for the linearised model

Notice, that new complex eigenvalues appear which correspond to $K_p \geq 14$.
We set $K_p = 16$ in the real time oscillatory mode experiment.



The time diagrams of: the cart position signal, the pendulum angle under the control signal generated by the PID controllers for the simulation model are presented in Figure 4-6, Figure 4-7 and Figure 4-8 for the default, aperiodic and oscillatory modes. The corresponding time diagrams for the real laboratory system are given in Figure 4-9, Figure 4-10 and Figure 4-11.

Refer to Table 4 to set up controller parameters for each of the three system modes – default, aperiodic and oscillatory.



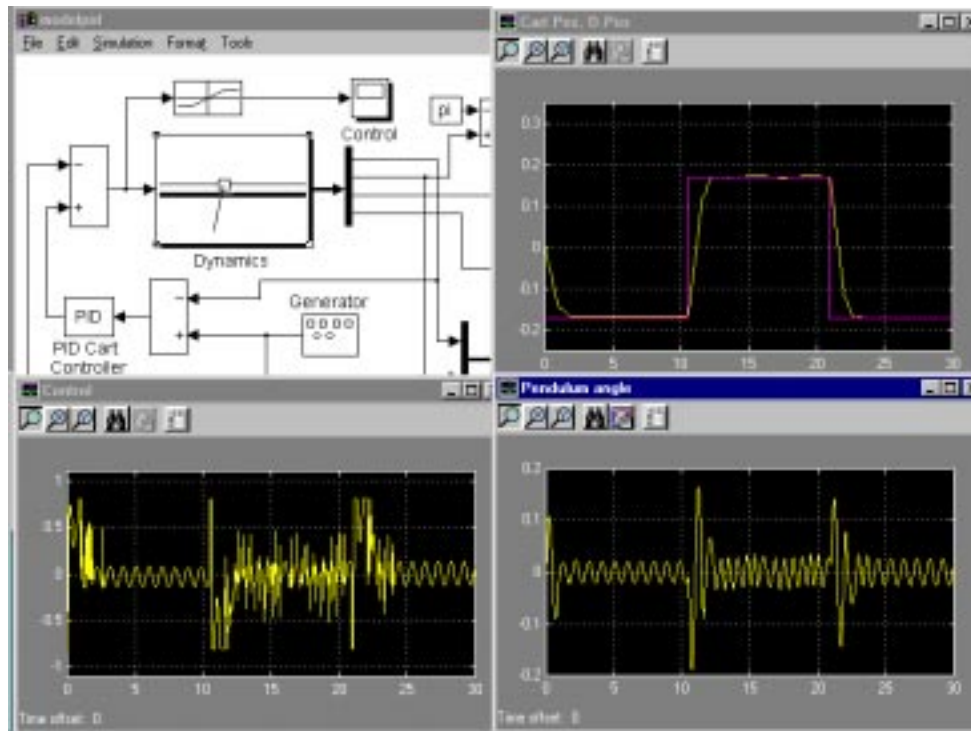


Figure 4-7: Aperiodic mode for the simulation model

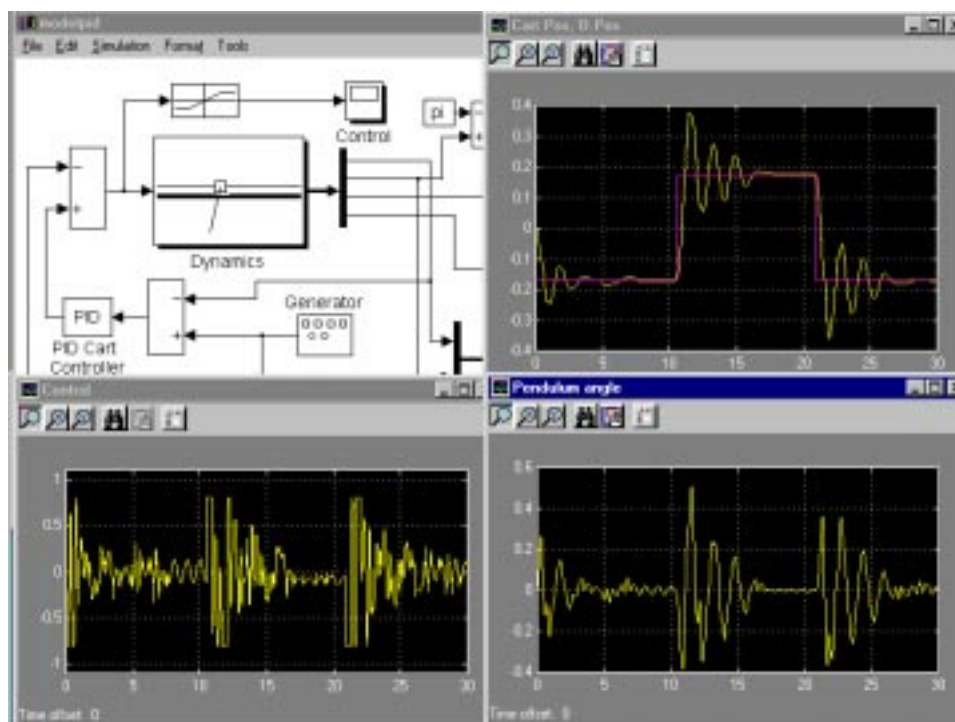


Figure 4-8: Oscillatory mode for the simulation model



DIGITAL PENDULUM SYSTEM TEACHING MANUAL 1

CHAPTER 4 PID CONTROLLERS

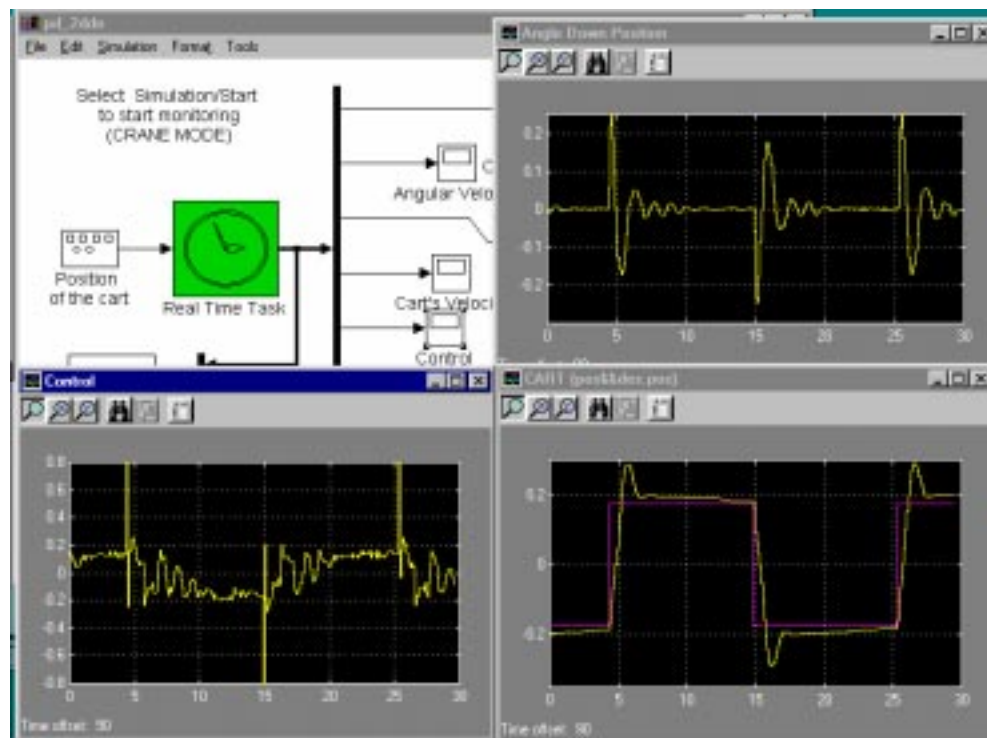


Figure 4-9: Default mode for the real model

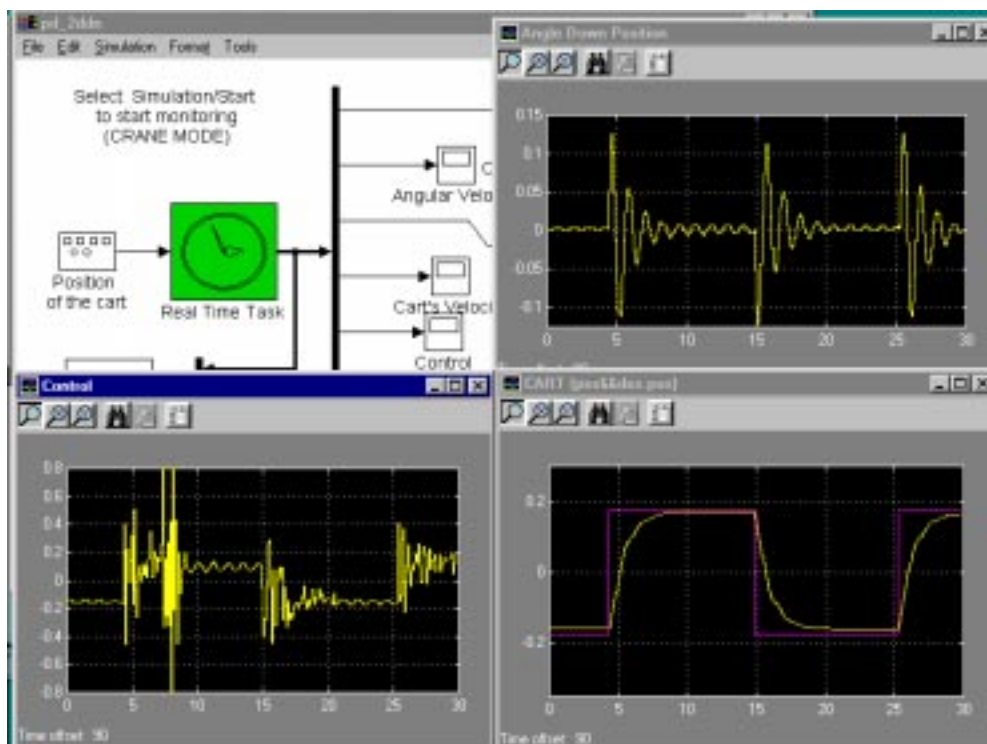


Figure 4-10: Aperiodic mode for the real model

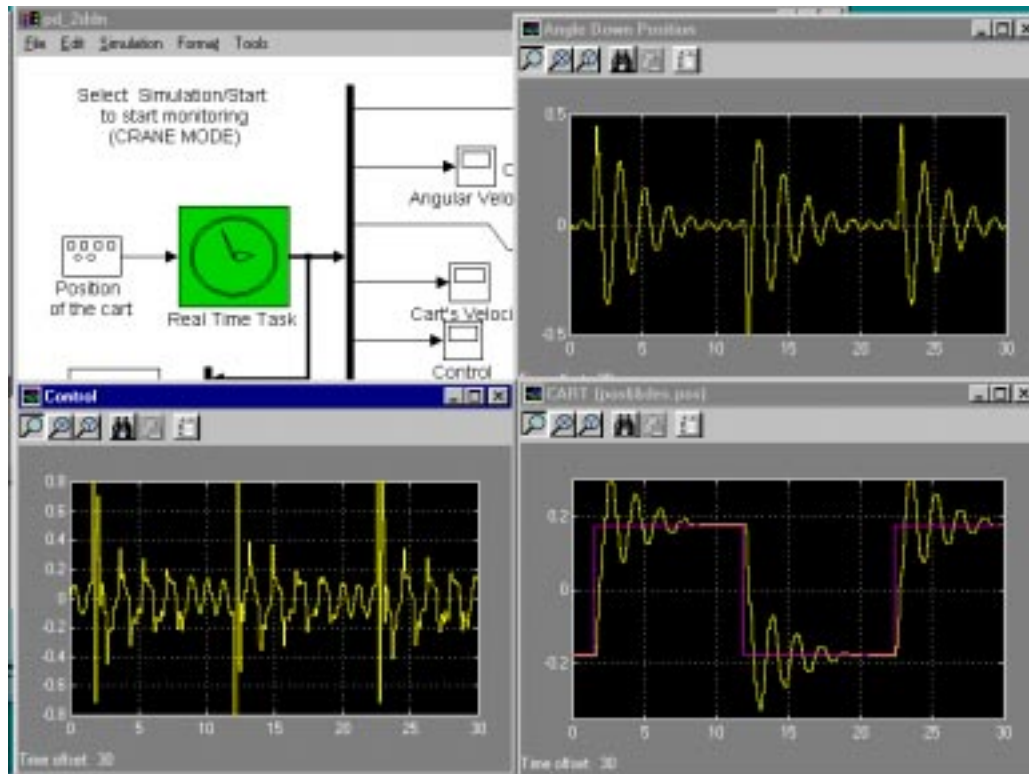


Figure 4-11: Oscillatory mode for the real model

Good quantitative and qualitative compatibility characteristics of the simulation and real system responses are observed. We can formulate some general remarks related to both model responses.

- The simulation model responses are characterised by smooth shapes.
- Insignificant differences of the control time responses are visible in the steady state time intervals. This is due to the random character of friction.
- It is recommended to repeat the identification procedure from time to time, with the inclusion of the PID or LQ controllers as it is well known that system parameters can vary over long periods of time.



5 LINEAR-QUADRATIC CONTROLLERS

The linear-quadratic problem (LQ problem) is a central one in the theory and applications of optimal control. There are two versions of the LQ problem: the open-loop and the closed-loop optimal control problems. Either the optimal control is given as an explicit function of time for fixed initial conditions, or the optimal controller is synthesised. Further, only the second case is considered. The main result of the finite-dimensional linear-quadratic theory is that under suitable assumptions the optimal feedback controller is linear with respect to the state, and constant with respect to time.

The cart-pendulum system is a nonlinear one. It can be approximated by a linear model in the neighbourhood of equilibrium points. The closed-loop system with an LQ controller has an infinite number of periodic equilibrium points. In practice they are reduced to three equilibrium points: $x=[-x_1, \pi, 0, 0]$, $x=[+x_1, \pi, 0, 0]$ and $x=[0, 0, 0, 0]$. The first and second correspond to the **C** (Crane) problem (see Section 4.1), and the third to the **SE** (Self Erecting) problem. LQ controller design for both problems is performed in the same way.

5.1 LQ CONTROL PRINCIPLES

The synthesis of the continuous LQ controller is presented below. For a very small value of the sampling time the response of the discrete system converges to the response of the corresponding continuous system. It is just our case. The most important question for the designer of a control system is how to select the weighting factors in the cost function.

The linearised dynamical model of the cart-pendulum system is described by the linear differential equations:

$$\frac{dx}{dt} = Ax + Bu \quad (5.1)$$

$$y = x,$$

where the state matrices A and B are obtained using the *linmod* MATLAB function (see the next section). The desired output time history of the state vector is assumed as $yd = [yd_1, yd_2, yd_3, yd_4]$. Hence, the error vector e is defined:

$$e = yd - y \quad (5.2)$$

A typical quadratic cost function has the form:



$$J(u) = \frac{1}{2} \int_0^{\infty} [e^T(\tau) Q e(\tau) + u^T(\tau) R u(\tau)] d\tau \quad (5.3)$$

where:

- Q is a nonnegative definite matrix $Q \geq 0$, $Q = Q^T$,
- R is a positive definite matrix $R > 0$, $R = R^T$, in our case R is a scalar,
- The pair (A, B) is controllable.

The weighting matrices Q and R are selected by the designer but they must satisfy the above conditions. This is most easily accomplished by choosing Q to be diagonal with all diagonal elements positive or zero.

The values of elements of matrix Q and scalar R (5.3) are weakly connected with the performance specification. A certain amount of trial and error is required with a simulation program to achieve satisfactory result. A few guidelines can be given. For example, if all states are to be kept under close regulation and Q is diagonal with entries so selected that a fixed percentage change of each variable makes an equal contribution to the cost. The matrix R is also diagonal. If the maximum deviations of the system outputs are: y_{1max} , y_{2max} , y_{3max} , y_{4max} and the maximum deviation of control is: u_{max} , then the cost is:

$$\int_0^{\infty} (Q_{11}e_1^2(t) + Q_{22}e_2^2(t) + Q_{33}e_3^2(t) + Q_{44}e_4^2(t) + R u^2(t)) dt$$

where the weighting matrices fulfil the following equations:

$$Q_{11} = \frac{1}{y_{1max}^2}, \quad Q_{22} = \frac{1}{y_{2max}^2},$$

$$Q_{33} = \frac{1}{y_{3max}^2}, \quad Q_{44} = \frac{1}{y_{4max}^2}, \quad R = \frac{1}{u_{max}^2} \quad (5.4)$$

This rule can be modified to satisfy desired root locations and transient response for selected values of weights. One must avoid saturation effects both of outputs and control.

The optimal scalar control u^* and the optimal trajectory vector y^* are given then by:

$$u^* = -K(yd - y^*) \quad (5.5)$$



where K is the feedback matrix.

The optimal control problem is now defined as follows: find the gain K such that the feedback law (5.5) minimises the cost function (5.3) subject to the state equation (5.1). The calculation of the control variable which minimises the criterion (5.3) is a dynamic optimisation problem. The procedure returns the optimal feedback matrix K , and the matrix S , the unique positive definite solution to the associated matrix Riccati equation:

$$SA + A^T S - SBR^{-1}B^T S + Q = 0 \quad (5.6)$$

Because of the quadratic appearance of S , there is more than one solution, and S must be positive definite to select the correct one.

The vector K can be calculated by a numeric iterative formula on the basis of the Riccati equation (5.6). The associated closed-loop system:

$$\frac{de}{dt} = (A - BK)e \quad (5.7)$$

is asymptotically stable (under the controllability assumption of the pair A, B).

To solve the LQ controller problem the *lqry* function can be used from the *MATLAB Control System Toolbox*. The syntax of *lqry* is: $[K, S, E] = \text{lqry}(A, B, C, D, Q, R)$.

In this case the matrix Q weights the outputs y instead of the state x . D is the row matrix with four zero elements. The function *lqry* computes the equivalent Q , R and calls *lqr*, the other function from the same toolbox. The procedure returns also the eigenvalue vector E :

$$E = \text{eig}(A - BK)$$

The control u is not subject to constraints. Such an assumption cannot be satisfied in a real physical system. One must remember that if the control u is saturated during the practical then it does not satisfy the LQ principles. To return to the LQ problem the amplitude of the u signal should be diminished. In such a case the designer must increase the weight R in (5.3). It is then recommended to use the simulation tool.



5.2 DESIGN OF LQ CONTROLLERS (CRANE PROBLEM)

All the crane demos presented in this section deal with the same task: i.e. move the cart to an arbitrary chosen position (Figure 5-1) and stabilise the pendulum in the down position there.

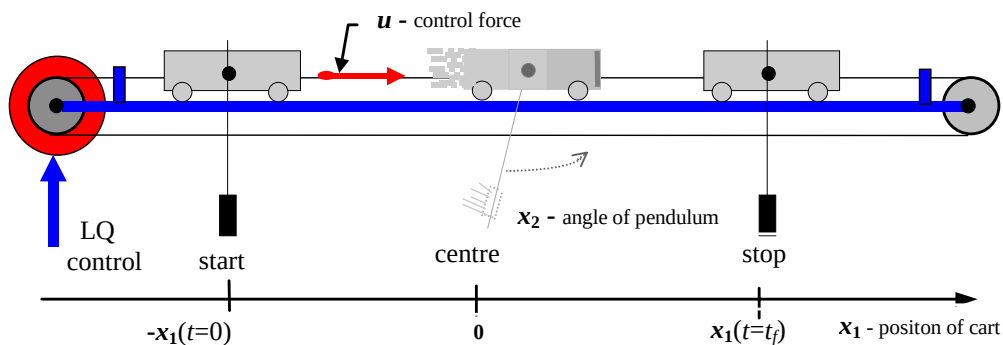


Figure 5-1: LQ control for the crane problem.

We are concerned with two equilibrium points **start** or **stop** for the **C** problem as shown in Figure 5-1. These two points can be replaced by the **centre** point $x_d=[0, \pi, 0, 0]$. One can notice that the right hand sides of equations (2.1) do not include x_1 variable and for an arbitrary cart position the same linear model is obtained. The similar remark is valid for the **SE** problem. Hence, we are concerned only with one equilibrium point e.g. $x_u=[0, 0, 0, 0]$.

In order to obtain the model of the crane in the form of (5.1) we have to linearise the pendulum dynamics model around x_d and x_u equilibrium points. The model introduced in section 2, stored in the *model.m* file, must be simplified to be linearised. All unnecessary blocks are removed. Removing these blocks simplifies the calling syntax of the *linmod* function. The reduced form of the model is given in Figure 5-2. We can use the MATLAB function *linmod* from the *Control Toolbox* to obtain linear models from nonlinear sets of ordinary differential equations (ODEs). Notice, that the model is equipped with the *Inport* block to obtain a non empty matrix B.

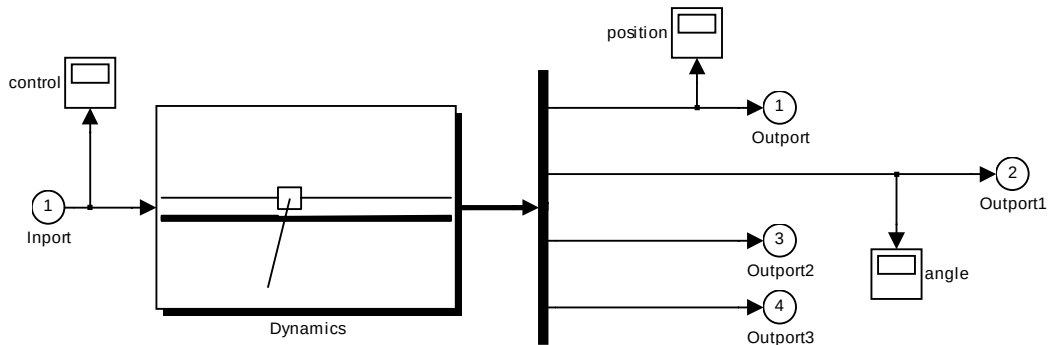
We simply call the function *linmod* in the MATLAB Command Window:

```
[Au, Bu, Cu, Du]=linmod('model4lq',[0,0,0,0],0);
```

The execution of this command results in obtaining the state-space linear described in the S-function *model4lq*. The calling syntax of *linmod* requires the state variables and input to be specified. In our case they are set to zero.



A linear model (the matrices A_u , B_u , C_u , D_u) is obtained at $x_u=[0, 0, 0, 0]$, $u=0$ operating point. The calculated matrices correspond to the unstable upright equilibrium point. We denote A_u , ... for the unstable, and later A_s for the stable point, to distinguish sets of matrices for both equilibrium points.



PENDULUM-CART SYSTEM

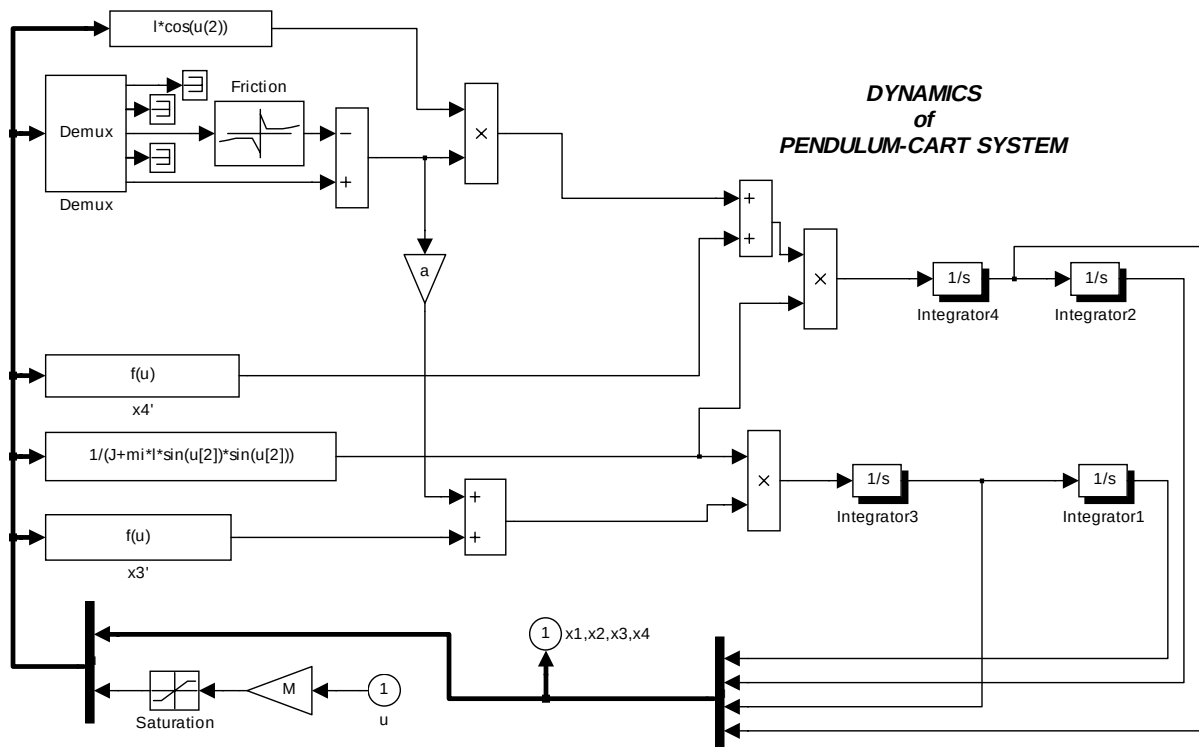


Figure 5-2: The reduced model *model4lq.m*



$$A_u = \begin{bmatrix} 0 & 0 & 1.0000 & 0 \\ 0 & 0 & 0 & 1.0000 \\ 0 & 0 & -34.9727 & -0.0001 \\ 0 & 43.42113 & -145.7195 & -0.0054 \end{bmatrix}$$

$$B_u = [0 \ 0 \ 9.1803 \ 38.2514]$$

$$C_u = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$D_u = [0 \ 0 \ 0 \ 0]$$

The eigenvalues of the matrix A_u can be obtained by typing in the MATLAB Command Window:

```
Eu = eig(Au)
```

```
Eu = [0 6.4216 -6.3515 -35.04820]'
```

As one can expect, there is one positive value. The system is unstable in its upright position. The system equipped with an appropriate linear feedback will become stable.

The same procedure can be applied to obtain a linear model at the second (down) stable equilibrium point. After calling:

```
[As, Bs, Cs, Ds]=linmod('model4lq',[0,pi,0,0],0);
```

we obtain:



$$A_s = \begin{bmatrix} 0 & 0 & 1.0000 & 0 \\ 0 & 0 & 0 & 1.0000 \\ 0 & 0.6111 & -34.9727 & -0.0001 \\ 0 & 43.4213 & -145.7195 & -0.0054 \end{bmatrix}$$

$$B_s = [0 \ 0 \ 9.1803 \ -38.2514]'$$

$$C_s = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$D_s = [0 \ 0 \ 0 \ 0]'$$

$$E_s = \text{eig}(A_s)$$

$$E_s = [0 \ -0.0379 - 6.3997i \ -0.0379 + 6.3997i \ -34.9024]'$$

The system is stable. Having two sets of linearised models we are ready to apply LQR controller. The SIMULINK model of the LQR controller is given in Figure 5-3.

The next step is to assume the weighting matrices Q and R . It can be done according to LQR principles. For example, we write in MATLAB Command Window:

$$Q = \text{eye}(\text{size}(A_s));$$

$$Q(1,1) = 50;$$

$$Q(2,2) = 100;$$

$$Q(3,3) = 1;$$

$$Q(4,4) = 1;$$

$$R = 1;$$



$[K, S, E] = \text{lqr}(A_s, B_s, Q, R);$

$K = [7.0711 \quad -11.7190 \quad 2.0251 \quad -0.2106]$

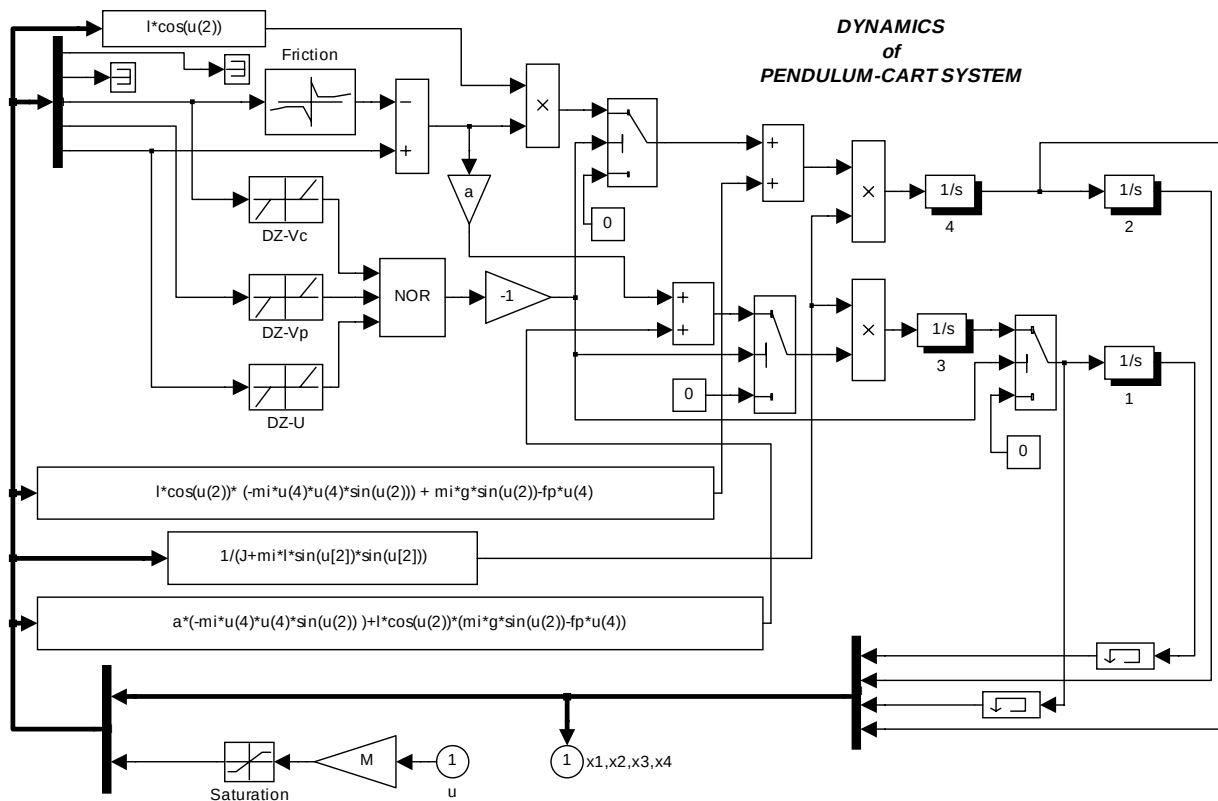
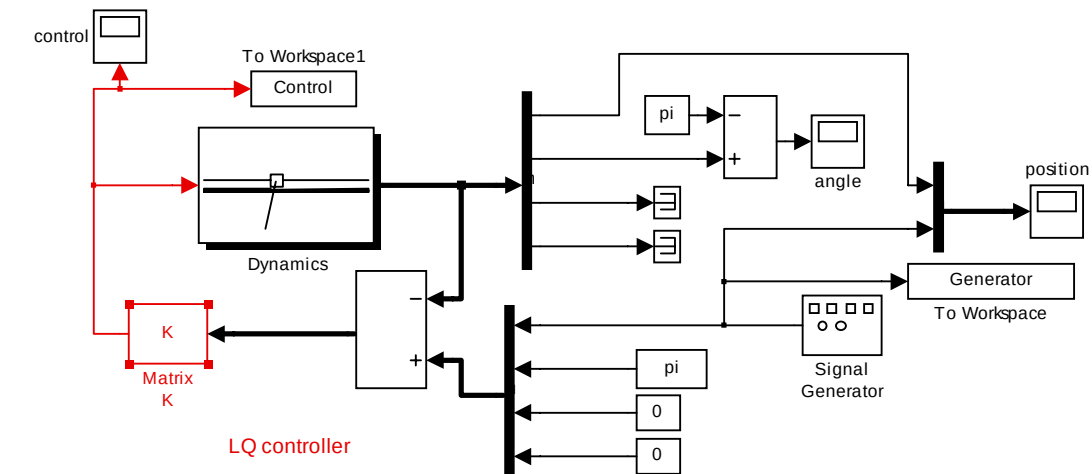


Figure 5-3: The SIMULINK model of the LQR controller *model_lqr.m* and the interior of *Dynamics* block



If we call the integration procedure *ode4* from the command line we must remember to check first the ordering of states. Usually an ordering automatically done by SIMULINK differs from the ordering assumed in the model menu. Type in the MATLAB Command Window:

```
[sizes,x0,string] = model_lq
```

to check the SIMULINK ordering. This command results in:

```
sizes = [ 4  0  0  0  0  0  2 ]'
```

```
x0 = [ 0  3.1416  0  0 ]' (0   $\pi$   0  0)
```

```
string =
```

```
/model_lq/Dynamics/1
```

```
/model_lq/Dynamics/2
```

```
/model_lq/Dynamics/4
```

```
/model_lq/Dynamics/3
```

The ordering of states is stored in the variable *string*. Notice, that the initial value of the pendulum angle must be put at first position. You can write the m-file to perform integration of *model_lq* using *ode4* procedure or you can start a simulation straight from the SIMULINK model menu not forgetting to define the parameters.

The calling syntax for integration invoked from the command line (m-file) is given below. The simulation time is set to 50 [s], the initial state vector [0 pi 0 0] relates to the pendulum at its down position with the cart at the centre of the rail, both velocities are equal to 0, the vector *opt* of integration parameters is defined by a *simset*:

```
opt=simset('Solver','ode4',...  
          'FixedStep',0.01,...  
          'OutputPoints','specified',...  
          'OutputVariables','x',...  
          'InitialState',x_ini);
```



It results in the following structure:

```
Solver: 'ode4'  
RelTol: []  
AbsTol: []  
Refine: []  
MaxStep: []  
InitialStep: []  
MaxOrder: []  
FixedStep: 0.0100  
OutputPoints: 'specified'  
OutputVariables: 'x'  
MaxRows: []  
Decimation: []  
InitialState: [0 3.1416 0 0]  
FinalStateName: []  
Debug: []  
Trace: []  
SrcWorkspace: []  
DstWorkspace: []  
ZeroCross: []
```

```
[T,X,Y]=sim('model_lq',50,opt);  
subplot(311);  
plot(T,X(:,1),'g-');  
hold on  
plot(T,Generator,'m-');  
hold off  
ylabel('[m]');
```



```
grid;  
title('Green->Cart pos. Mag->Des. pos, Red->Pendulum angle,Blue->Control');  
subplot(312);  
plot(T,X(:,2)-pi,'r-');  
ylabel('rad');  
grid;  
subplot(313);  
plot(T,Control,'b-');  
ylabel('N');  
grid;  
xlabel('Time [s]');
```

The MATLAB Graphical Window shown in Figure 5-4 is created.

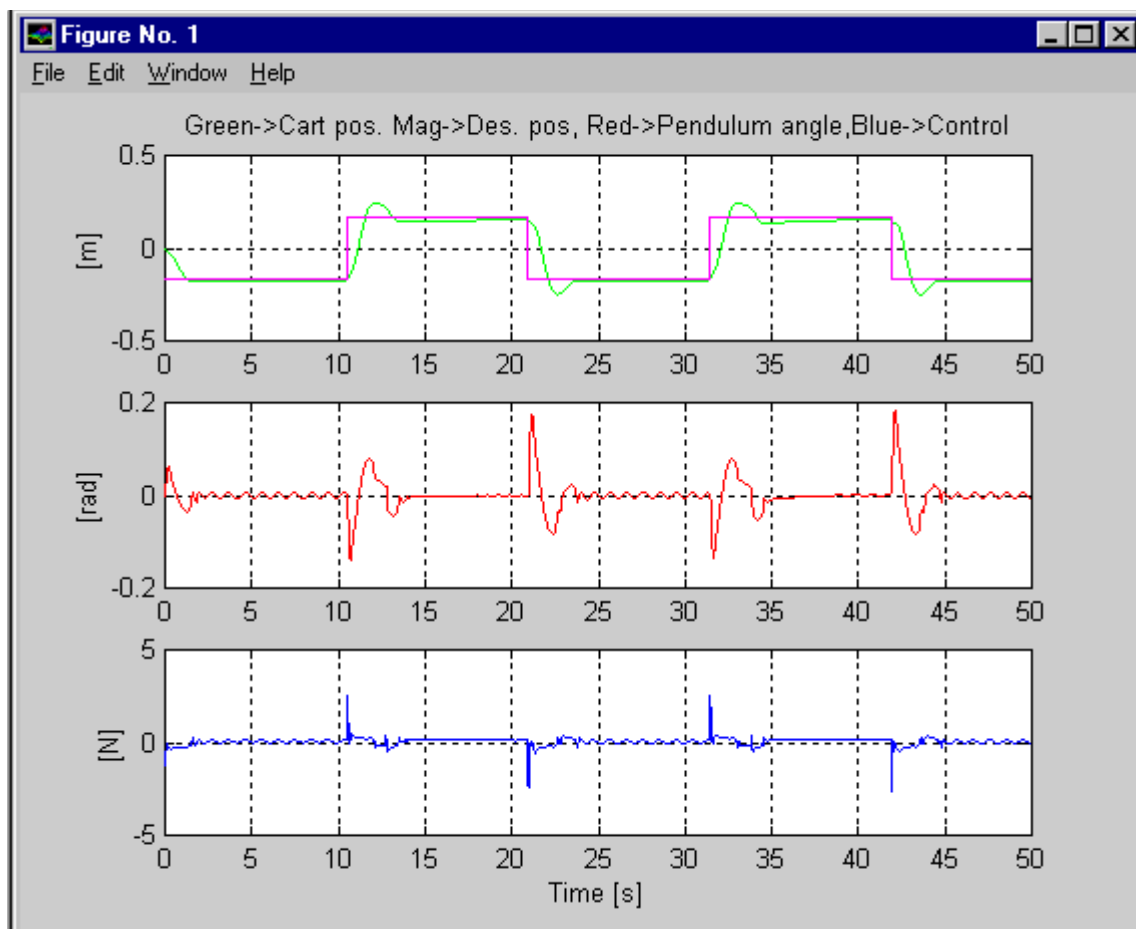


Figure 5-4: Simulation of LQ controller



5.3 LQ CONTROL EXPERIMENTS.

LQ control experiments can be performed in two ways :

1. From the *pendulum* main menu window by invoking *Crane LQ Controller* demo.
2. From MATLAB Command Window by invoking the *pd_2ddn.m* file.

The first method (*pendulum.m*).

After clicking on the *Crane LQ Controller* block the following window opens (Figure 5-1). The *Desired Position Generator* is set to generate a square wave signal.

Start real experiment from Simulation –Start top bar menu. While this is controlling using default parameters , return to *main control menu* and select *Parameters of the Controller* block. Here you can apply the calculated vector K for the real laboratory system. Click on *Set* button in order to acknowledge all changes to RTK. Return to *pd_lqr* file and see the changes to the system by opening the scope blocks.

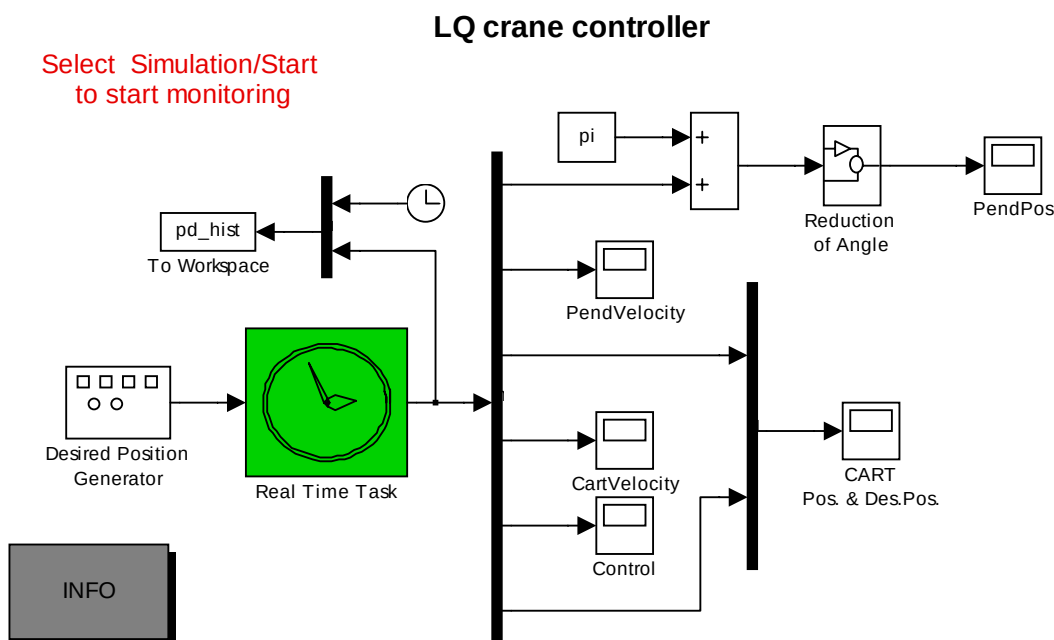


Figure 5-1: LQ crane controller

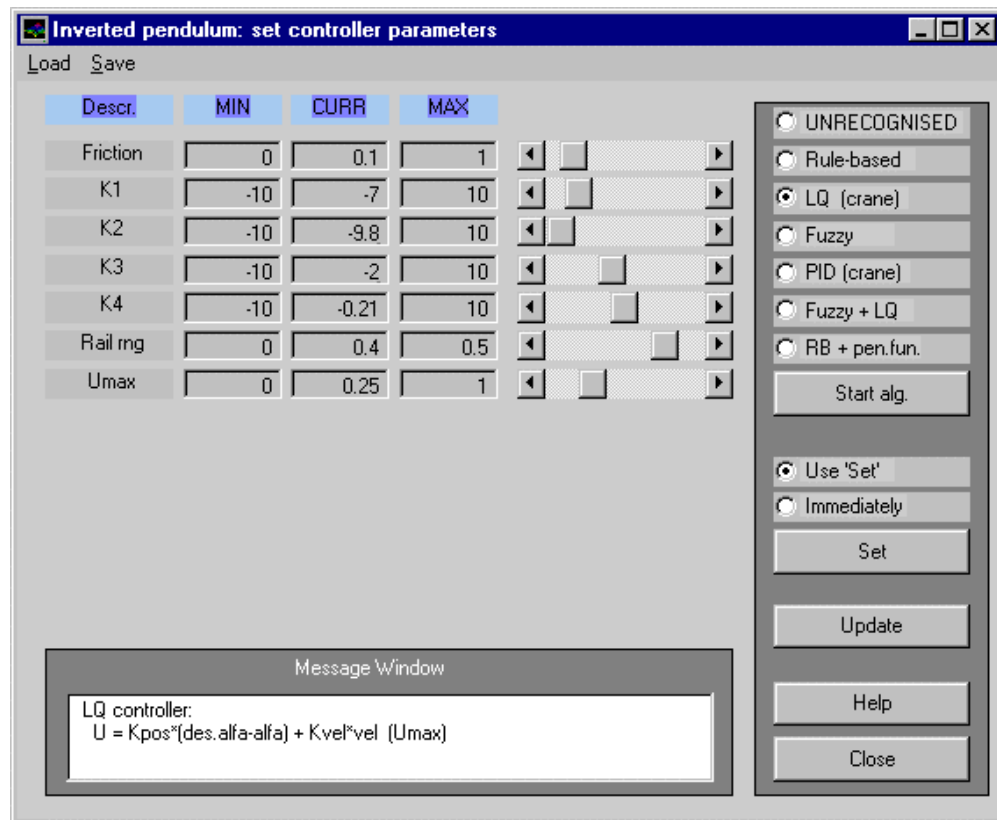


Figure 5-2: Tuning of K

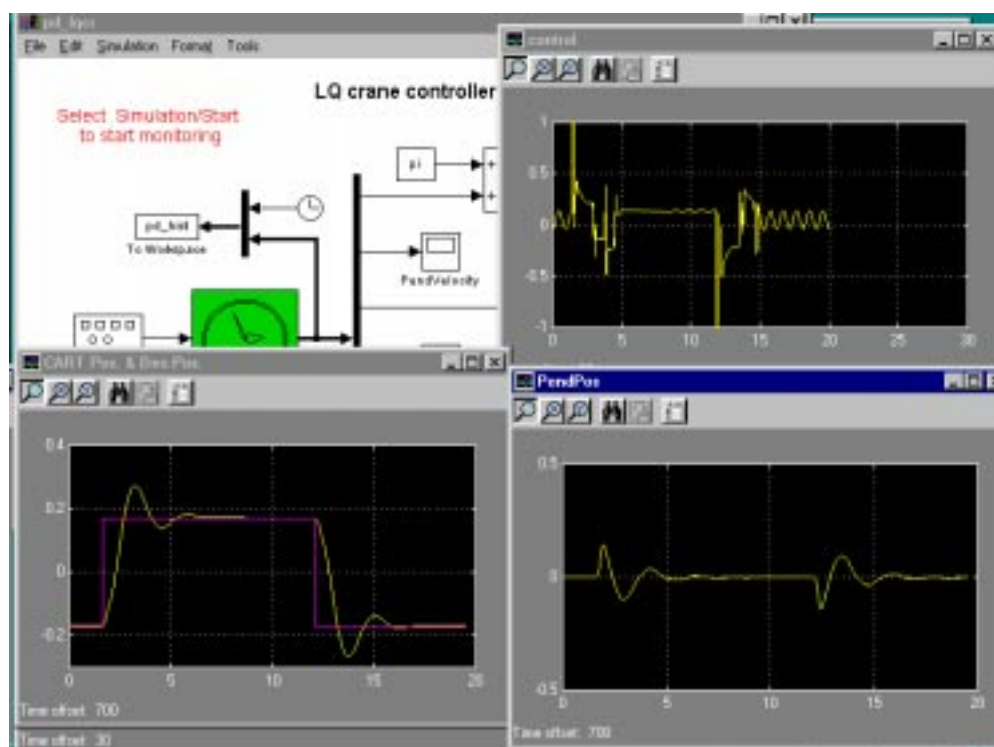


Figure 5-3: Responses of the laboratory model for the first experiment



LINEAR QUADRATIC CONTROLLERS

Similar experiments can be performed for the upright stabilisation of the pendulum. In this case we must use the feedback vector K calculated for the $A_u B_u C_u D_u$ matrices of the unstable linearised model.

The following steps are sequentially called by the *lqpnew.m* file: linearisation, order checking, LQR problem solving and simulation of the closed-loop system with the calculated LQ controller feedback. The *model4lq.m* is linearised at [0 pi 0 0] equilibrium point by the *linmod* procedure. It results in the *As Bs Cs Ds* matrices of the linearised system (not displayed in the MATLAB Command Window). The new file *model_j.m* is introduced (Figure 5-4).





Next, Q and R weighting matrices are defined. Remember that the default values of Q and R are defined in the *lqpnew.m* file. Put values of your own choice into the *lqpnew.m* file! The feedback vector K is calculated by the *lqr* procedure. The *ode4* procedure integrates the *model_j.m* SIMULINK model. The *lqpnew.m* procedure is paused two times to allow checking if the initial state ordering in *ode4* calling syntax and in *model_j.m* correspond to each other. It is important to know the structure and ordering of blocks generated automatically by SIMULINK. If the model is called from a command line the ordering of states is essential for a proper integration.

Remember, the *linmod* procedure from the CONTROL Toolbox calls the model from a command line. In this case the model cannot be called from the menu!

The *model_j.m* S-function is enlarged with one state to calculate the performance index. Following these remarks we open the *lqpnew.m* file and define Q and R :

```
Q = [50 0 0 0; 0 100 0 0; 0 0 0 0; 0 0 0 0];  
R = 1;
```

This modification of the *lqpnew.m* file is necessary if you like to have the Q and R matrices of your choice. Now we are ready to call the *lqpnew.m* file which automatically executes all mentioned above functions. In MATLAB Command Window we simple write:

```
lqpnew
```

The numerical and graphical results are displayed one by one in MATLAB Command and Graphical windows.

We obtain in MATLAB Command Window:

```
sizes = [4 0 4 1 0 1 2]  
x0 = [0 3.1416 0 0]
```



string =

'model4lq/Dynamics/Integrator1'

'model4lq/Dynamics/Integrator2'

'model4lq/Dynamics/Integrator3'

'model4lq/Dynamics/Integrator4'

Check sequence of states !

SIZES(1) number of continuous states

SIZES(2) number of discrete states.

SIZES(3) number of outputs

SIZES(4) number of inputs

SIZES(5) number of roots that the system has.

SIZES(6) set to 1 if the system has direct feed-through of its inputs
(used for systems within systems).

SIZES(7) number of sampling periods

$\text{eig}(AS) = [0 \quad -0.0379 + 6.3997i \quad -0.0379 - 6.3997i \quad -34.9024]$

$Q = \begin{vmatrix} 50 & 0 & 0 & 0 \\ 0 & 100 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{vmatrix}$

Sizes = [5 0 0 0 0 0 2]'

$x_0 = [0 \ 3.1416 \ 0 \ 0]'$



string =

'model_j/Dynamics/1'

'model_j/Dynamics/2'

'model_j/Dynamics/4'

'model_j/Dynamics/3'

'model_j/Integrator'

Check sequence of states in "model_j" !

Matrix K = [7.0711 -9.8961 1.7055 0.1227]

This is the end of information displayed in MATLAB Command Window. Similarly, we obtain the following figure in MATLAB Graphical Window (Figure 5-5).

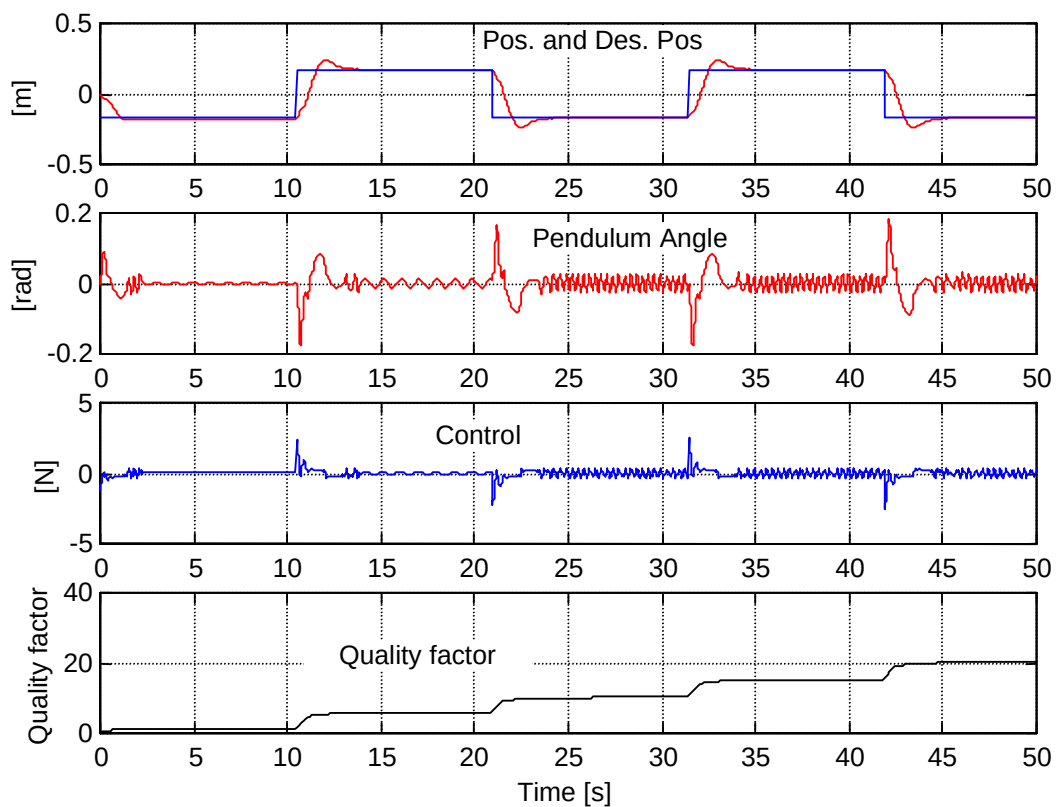


Figure 5-5: Simulation of LQ control



This is the end of information displayed in MATLAB Graphical Window.

We are ready for real system LQ experiments. The matrix K is applied in the menu of the *pd_2ddn.m* file.

To introduce the above optimal parameters in real control, after Simulation-Star command, open block called *Parameter Tuning* situated on the right-hand corner of this *pd_2ddn* file. We use the PID controller template for the LQ controller purpose. We assume:

K_i coefficients are zero

K_p Angle-Pendulum Angle relates to K(2)

K_p Position-Cart Position relates to K(1)

K_d-Pendulum Velocity relates to K(4)

K_d -Cart Velocity relates to K(3)

This ordering correspond to the {1 2 4 3} ordering of states in the *model_j.m* file.

An update of parameters is shown in Figure 5-6.



Figure 5-6: The K vector tuning

The real system response is given in Figure 5-7. The K vector is optimal as far as the Q and R matrices in the quadratic performance index are concerned.

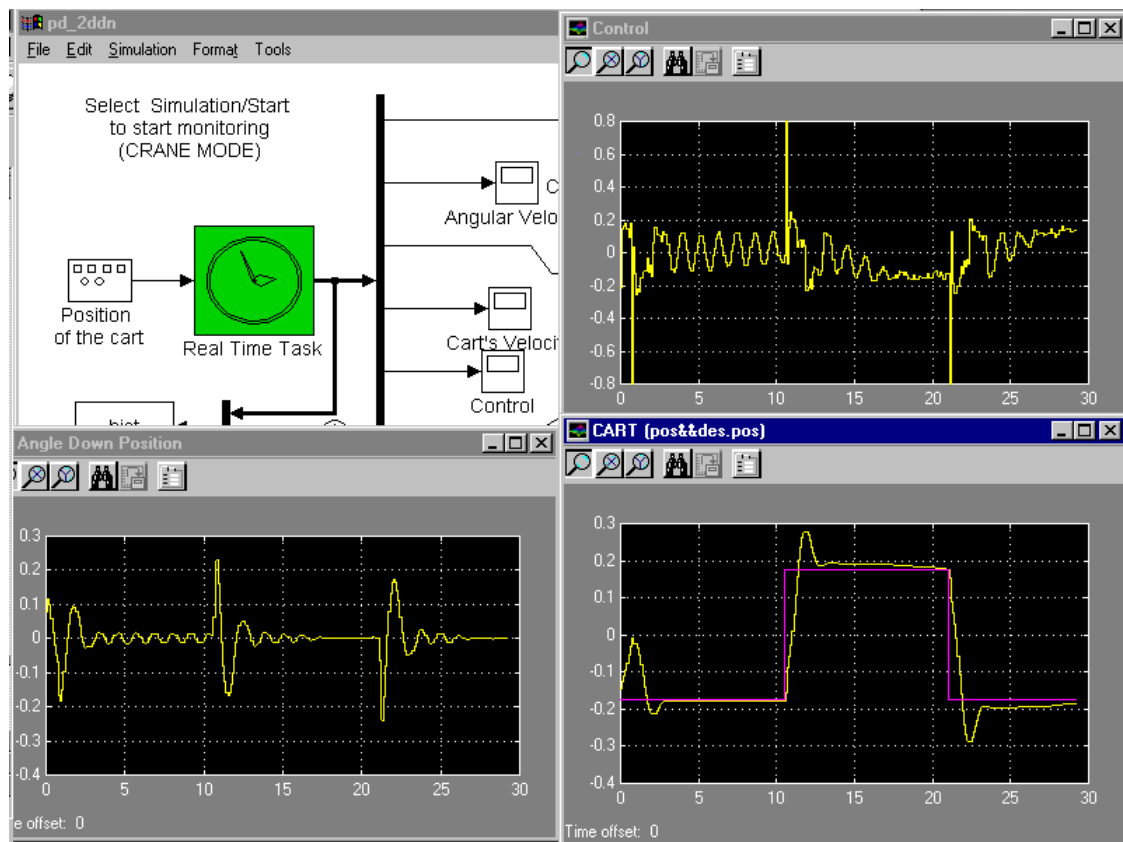


Figure 5-7: LQ control experiment

It is easy to show that the minimal value of the quadratic performance index is obtained for the optimal value of K . It is illustrated in Figure 5-8. Three simulations are performed for three different vectors K :

$$K = \begin{bmatrix} 7.0711 & -9.8961 & 1.7055 & 0.1227 \end{bmatrix} \text{ (optimal)}$$

$$K = \begin{bmatrix} 3.0 & -9.8961 & 1.7055 & 0.1227 \end{bmatrix}$$

$$K = \begin{bmatrix} 12.0 & -9.8961 & 1.7055 & 0.1227 \end{bmatrix}$$

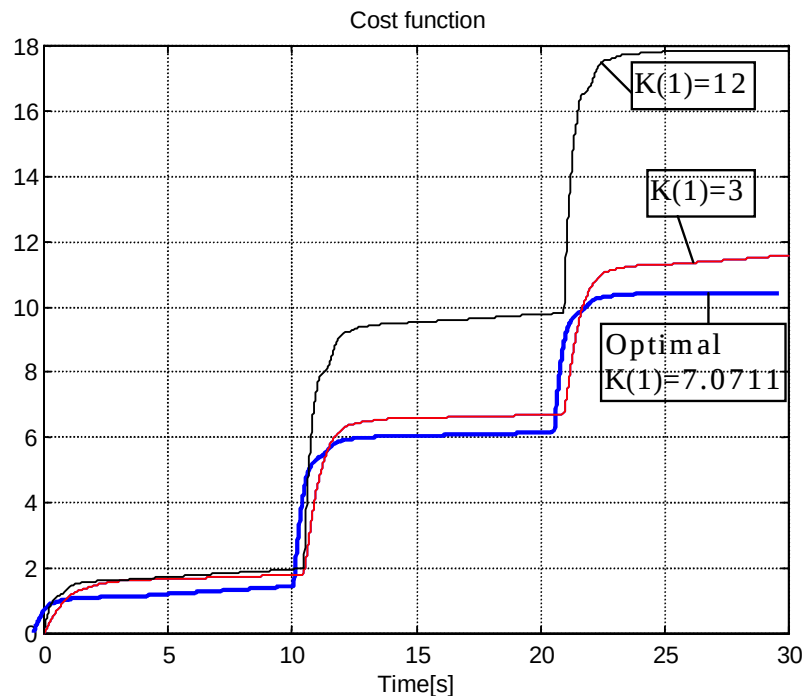


Figure 5-8: Costs vs. time

A next example concerns a different choice of the R weight. We introduce into the *lqpnew.m* file:

```
R=10;
```

After typing in MATLAB Command Window :

```
lqpnew
```

we obtain:

Matrix $K = [2.0000 \quad -2.3394 \quad 0.1941 \quad -0.0827]$

The following Graphical Window opens (Figure 5-9).

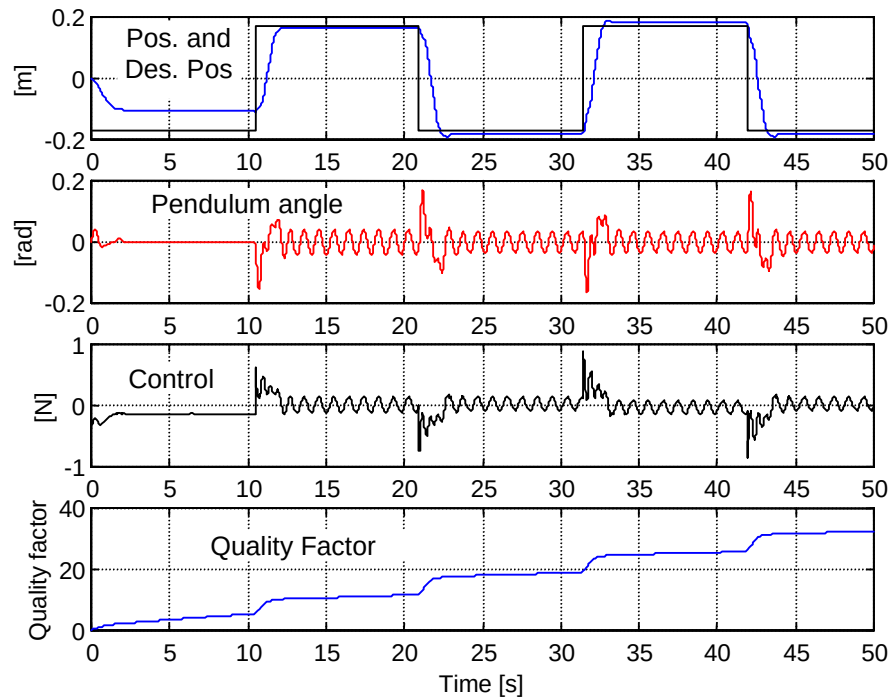


Figure 5-9: LQ controller simulation

We obtain the new vector K . It can be used again in the *pd_2ddn.m* file

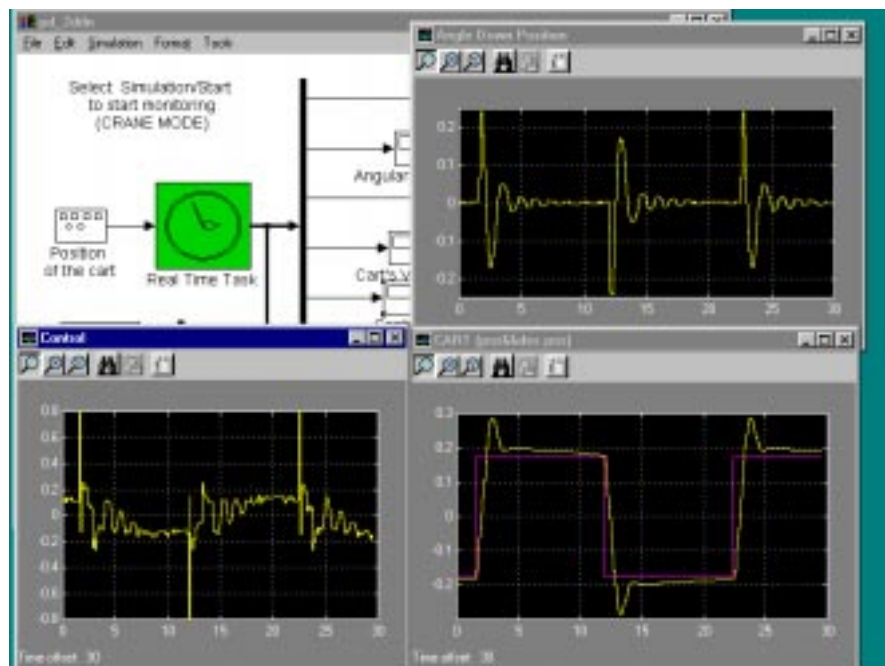


Figure 5-10: LQ controller practical

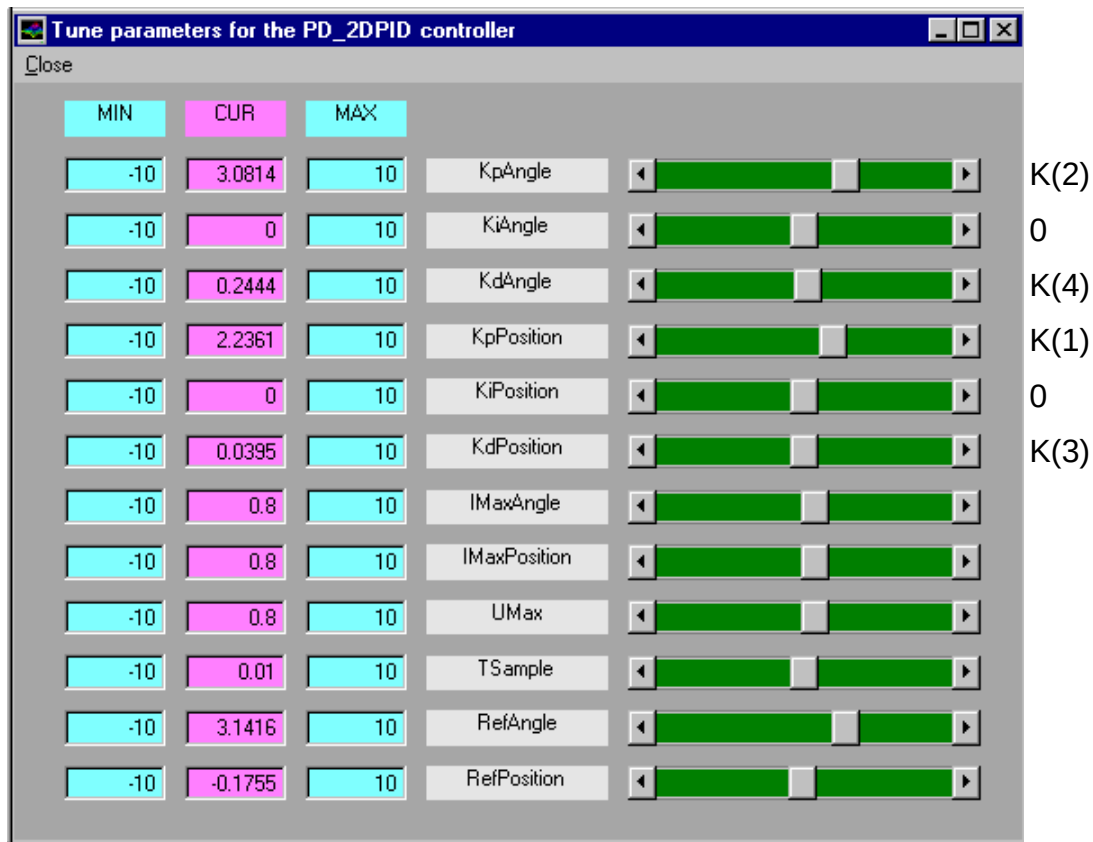


Figure 5-11: LQ controller parameters

The data taken from the laboratory system are presented in (Figure 5-10). The LQ controller parameters are given in Figure 5-11.

5.4 DISCRETE LQ CONTROLLERS

You can formulate also a discrete LQ problem. There are tools in the MATLAB and CONTROL Toolbox to be applied in the discrete case. The continuous linear model can be transferred to the discrete model. One can use the *c2d* procedure. If you type in the MATLAB Command Window:

```
[As, Bs, Cs, Ds] = linmod('model4lq',[0,pi,0,0],0);
```

```
[Phi, Gamma] = c2d(As,Bs,0.1)
```

The *c2d* procedure converts the continuous-time system:

$$\frac{dx}{dt} = As \cdot x + Bs \cdot u$$



into the discrete-time system:

$$x[n+1] = \text{Phi} \cdot x[n] + \text{Gamma} \cdot u[n]$$

assuming a zero-order hold on the inputs and sampling time 0.1[s].

$$\text{Phi} = \begin{bmatrix} 1.0000 & 0.0012 & 0.0278 & 0.00001 \\ 0 & 0.7978 & 0.2867 & 0.0931 \\ 0 & 0.0148 & 0.0336 & 0.0012 \\ 0 & -3.8672 & 3.5384 & 0.7973 \end{bmatrix}$$

$$\text{Gamma} = [0.0189 \quad -0.0753 \quad 0.2537 \quad -0.9288]$$

The eigenvalues of the Phi matrix are:

$$\text{eig}(\text{Phi}) = [1.0000 \quad 0.7991 + 0.5949i \quad 0.7991 - 0.5949i \quad 0.0305]$$

Assuming the same weighting matrices as in the continuous case:

$$Q = \text{eye}(\text{size}(\text{As}));$$

$$Q(1,1) = 50;$$

$$Q(2,2) = 100;$$

$$Q(3,3) = 1;$$

$$Q(4,4) = 1;$$

$$R = 1;$$

we apply the *dlqr* - linear quadratic regulator for discrete time systems:

$$[K, S, E] = \text{dlqr}(\text{Phi}, \text{Gamma}, Q, R)$$

$$K = [3.1813 \quad -4.5895 \quad -0.9920 \quad -0.4047]$$

$$E = [0.0170 \quad 0.6332 + 0.2032i \quad 0.6332 - 0.2032i \quad 0.8153]$$



The SIMULINK model and responses of the discrete LQ controller is shown in Figure 5-1.

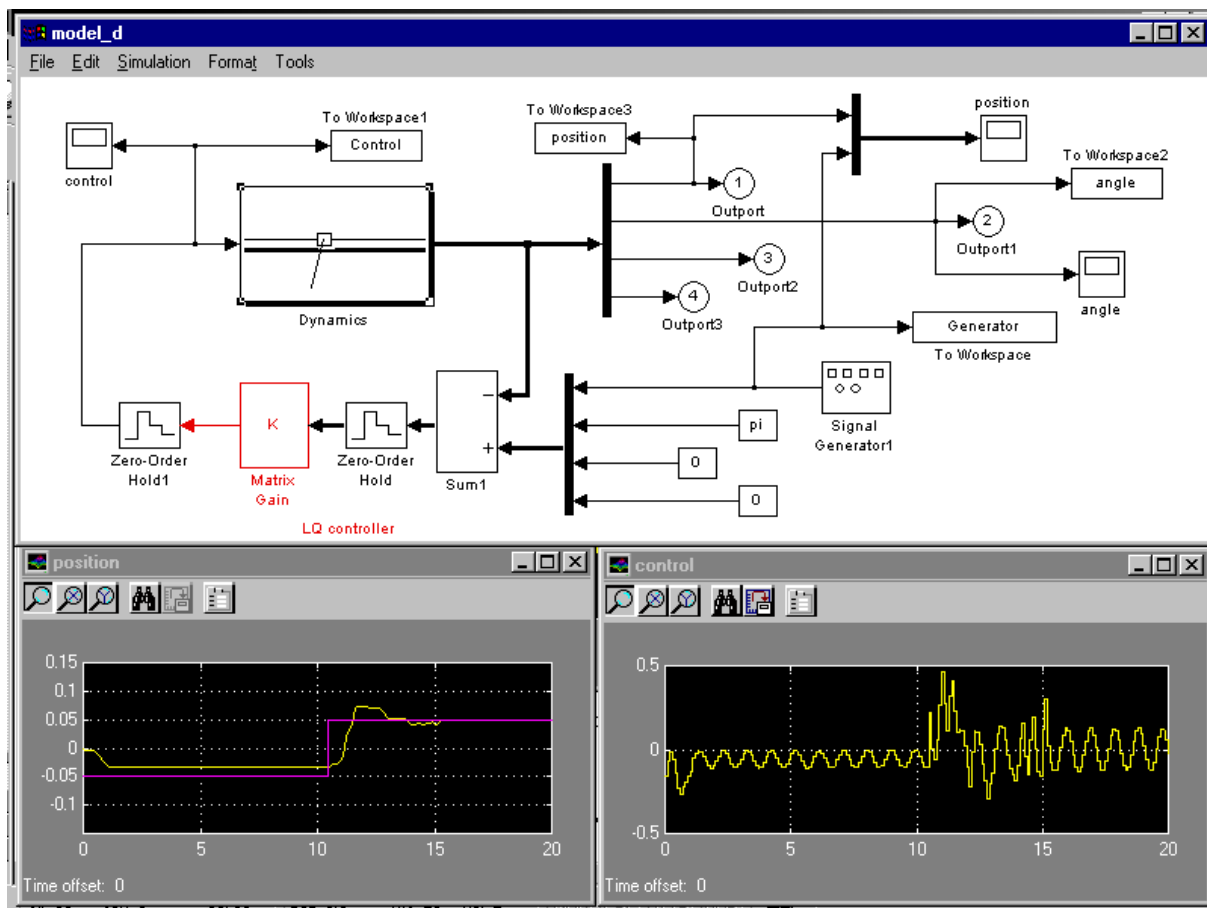
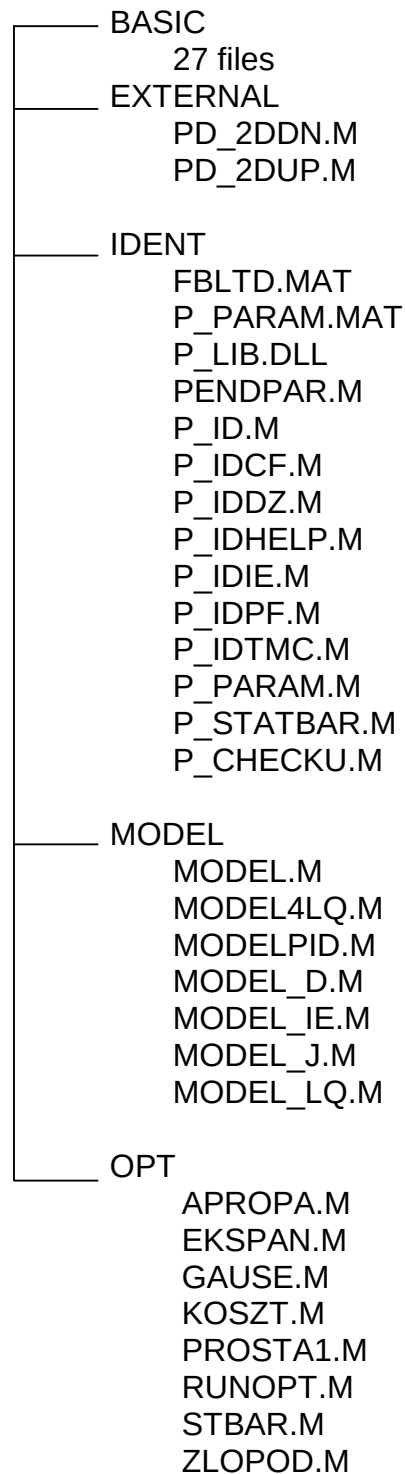


Figure 5-1: Discrete LQ controller *model_d.m* and its responses



6 LIST OF DIRECTORIES AND FILES :

The Teaching Manual I software consists off the following files. The subdirectories tree is shown below:





A short description of the main files is given below:

\\EXTERN

- *pd_2ddn.m* - external crane PID controller (built in SIMULINK),
- *pd_2dup.m* - external self-erecting PID controller (built in SIMULINK),

\\IDENT

- *fb ltd.mat* - Feedback Ltd. window (mat-file)
- *pendpar.m* - parameters of pendulum (m-file) (exported from *p_id.m*, ready to load to MATLAB WorkSpace,
- *p_helpf.m* - help
- *p_id.m* - identification procedures (m-file),
- *p_idcf.m* - identification of cart friction (static, dynamic) and detection of maximal force
- *p_iddz.m* - minimal velocity value for cart and pendulum
- *p_idhelp.m* - help
- *p_idie.m* - short impulse response
- *p_idpf* - pendulum friction and oscillation period
- *p_idtmc.m* - try to move the cart on the rail
- *p_lib.dll* - identification library (dll-file).

\\MODEL

- *model.m* - Pendulum-Cart model (S-file),
- *model4lq.m* - reduced *model.m* (S-file) for linearisation,
- *model_ie.m* - *model.m* equipped with pulse generator (S-file) for verification and tuning
- *model_lq.m* - *model.m* equipped with LQ controller (S-file),
- *model_j.m* - *model.m* equipped with LQ controller and cost function (S-file),
- *lqpnew.m* - sequence of commands (S-file) for: linearisation and states order checking (*model4lq.m*), LQR problem solving and simulation of the closed-loop model with LQ controller (*model_j.m*),
- *model_d.m* - *model.m* equipped with discrete LQ controller (S-file),
- *modelpid.m* - *model.m* equipped with PID controllers (S-file),



\OPT

- *aproxa.m* -parabolic approximation
- *ekspan.m* - expansion procedure
- *gause.m* - Gause optimisation procedure
- *koszt.m* -cost function
- *prosta1.m* - the line search
- *runopt.m* - main optimisation procedure
- *zlopod.m* - golden section procedure

Data are saved to the following files:

- Data related to **cart friction**
 - pend_cf1.dat
 - pend_cf2.dat
 - pend_cf3.dat
- Data related to **dead zones**
 - pend_dz.dat
- Data related to **impulse excitation**
 - pend_ie.dat
- Data related to **pendulum friction**
 - pend_pf.dat (to be used in optimisation)
- Data related to the experiment **“try to move cart”**
 - pend_tmc.dat (cart position and force are saved).



Notes